Theory of Computation
(three parts)

Computability	① Are there problems that a computer can not solve?
Complexity	① Any systematic way to classify problems as 'easy' or 'hard' which can be solved by the computer?
Automata	① How to define a computer? ② What can we compute with very little space?

Computability:
What problems can a computer solve?
Not all problems !!!

Ex. ① Given a C-program, we can not check if it will not crash.
② Verification of the correctness of programs is hence impossible.
③ Even checking whether a C-program will halt/terminate is not possible.

```
input n;
assume n > 1;
while (n != 1) {
  if (n is even)
      n := n/2;
  else
      n := 3*n + 1;
}
```

No one knows whether this terminates on all inputs!

17, 52, 26, 13, 40, 20, 10, 5, 16, ...

II. Complexity:

1. Focus on problems solvable by computers
2. Time complexity: How difficult w.r.t time requirement.
   - ↳ P, NP, NP-Complete
3. Space complexity: How difficult w.r.t space requirement
   - ↳ PSPACE, NL
4. problems that are outside the above class (so that they need much more time or space or other resources to solve)

III. Automata theory:

(1) study very simple "computer" called automaton (plural: automata)

(2) Though very simple, they can solve some decision problems
Like, is an input sequence of 0's and 1's, such that 10111010101, representing a binary number divisible by 5 ?

(3) Study what kind of decision problems can be solved or can not be solved by automaton ?

Grammar:
Rules for defining which strings over an alphabet are in a particular language

Automaton (plural: Automata)
A mathematical model of a computer which can determine whether a particular string is in the language?

Note: see next slides for the exact meaning of terms like, alphabet, language, etc.

Grammar:
Mathematically
Grammar is (N, T, P, S)

N = Non-empty (NE) set of Non-terminals
T = NE set of terminals
P = NE set of productions
S = Basically a non-terminal but it's the start point of derivation.

Ex: Grammar to generate valid identifier for 'C' programming language

Identifier → Non_digit | Identifier Non_digit | Identifier Digit
Non_digit → a | b | c | d | ... | z | A | B | C | ... | Z | _
Digit → 0 | 1 | 2 | 3 | ... | 9

↑ identify (N, T, P, S) ?

the language L generated (or defined) by the grammar $G=(N,T,P,S)$ is defined by

$$L=L(G)=\{x \mid S \overset{*}{\Rightarrow} x, \text{ and } x \text{ is in } \Sigma^*\}$$

↓

refers to derivation tree

(Deriving string x after starting from 'S')

Alphabet (Σ):

↳ Finite set of symbols

String:

↳ Finite sequence of symbols over Σ

Language:

↳ A formal language L is a set of strings over some finite set of symbols (Σ)

OR

An arbitrary subset of Σ^*.

More on string:

- length of the string w, is denoted by $|w|$
- Empty string is denoted by ϵ (Epsilon)

$$|\epsilon| = 0$$

- Formally, a string is
  - ↳ string $= \in$
  - ↳ string $= ax$, if $a \in \Sigma$ and x is a string
- length of string w is
  - ↳ $|w| = 0$, if $w = \in$

  else $|w| = 1 + |x|$ if $w = ax$

Meaning of $\Sigma^n, \Sigma^*, \Sigma^+$

Σ^n = set of all strings over Σ of length exactly equals to n

$\Sigma^0 = \{\in\}$

$\Sigma^n = \Sigma \circ \Sigma^{n-1}$ if $n > 0$

Σ^* = set of all finite length strings

$$\Sigma^* = \bigcup_{k=0}^{\infty} \Sigma^k$$

Σ^+ = set of all non-empty finite length strings

$$\Sigma^+ = \bigcup_{k=1}^{\infty} \Sigma^k$$

Note:

Qn: How long is the longest string in Σ^*

Ans: No longest string

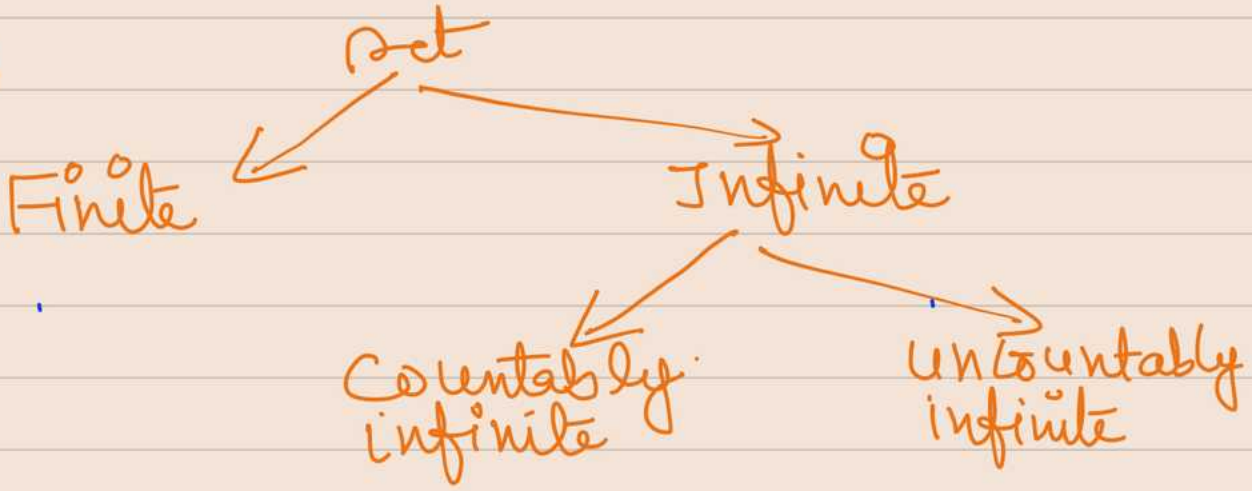
Qn: How many infinitely long strings are there in Σ^*

Ans: NONE

Definition of V^* from Wiki:

if V is a set of strings then V^* is defined as the smallest superset of V, that contains the empty string $\in$ and is closed under string concatenation operation.

Note:



Something on nothingness:

(1) $\in$ is a string containing no symbol (it's not a set) → Not a language

(2) $\{\in\}$ is a set containing one string (empty string $\in$) and it is not a string → Not a string

(3) ϕ is the empty set, it contains no string. → empty language

Note: There are uncountably many languages but each language has countably many strings.

Chomsky classification:

Note: Here, we consider Grammar

$$G \equiv (V_N, \Sigma, P, S)$$

Type 0 : (unrestricted form)

$$\alpha \rightarrow \beta$$

where

$$\alpha \in (V_N \cup \Sigma)^* V_N (V_N \cup \Sigma)^*$$

$$\beta \in (V_N \cup \Sigma)^*$$

Type 1 : (context sensitive)

$$\alpha \rightarrow \beta$$

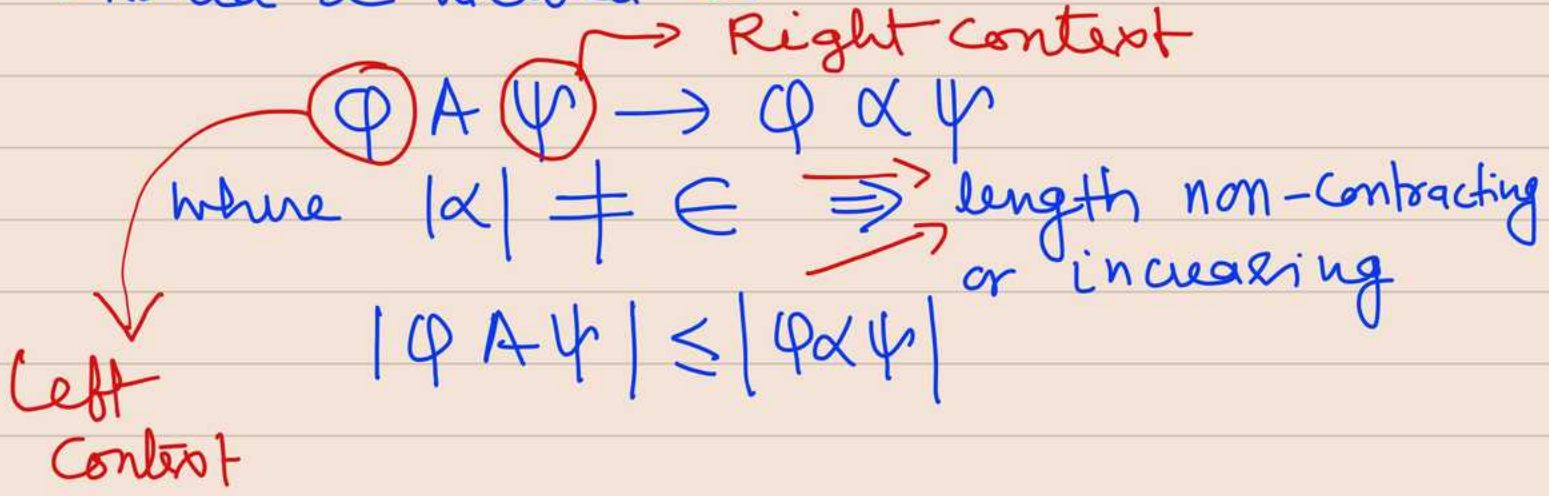
where

$$\alpha \in (V_N \cup \Sigma)^* V_N (V_N \cup \Sigma)^*$$

$$\beta \in (V_N \cup \Sigma)^+ \text{ and } |\alpha| \leq |\beta|$$

Note: In the Type 1 (context sensitive grammar) Null production of the form $S \rightarrow \in$ is allowed, however, then S should not occur in the R.H.S of any production.

Alternatively, in Type-1, a production should be viewed as



Type 2 (context free):

$$\alpha \rightarrow \beta$$

where $\alpha \in V_N$, $\beta \in (V_N \cup \Sigma)^*$.

Type 3 (Regular):

Productions of the following forms are allowed.

$A \rightarrow a$ where $A \in V_N$

$A \rightarrow bB$ $a, b \in \Sigma$

$A \rightarrow \in$

Note: In Type-3, productions can also be viewed as

$$A \to w$$
$$A \to wB, \text{ where } w \in \Sigma^*$$

because

$A \to a_1 a_2 a_3 \cdots a_K B$ can be split in to

$$A \to a_1 B_1,\ B_1 \to a_2 B_2, \cdots B_{K-1} \to a_K B$$

by using few more non-terminals

$B_1, B_2, \cdots, B_{K-1}$

Also Note:

$A \to wB \mid w$

Right linear

linear = single Non-terminal in the R.H.S

Right = Extreme right here, $w \in \Sigma^*$.

$A \to Bw \mid w$

Left linear

Note: this left linear can be viewed as Right linear

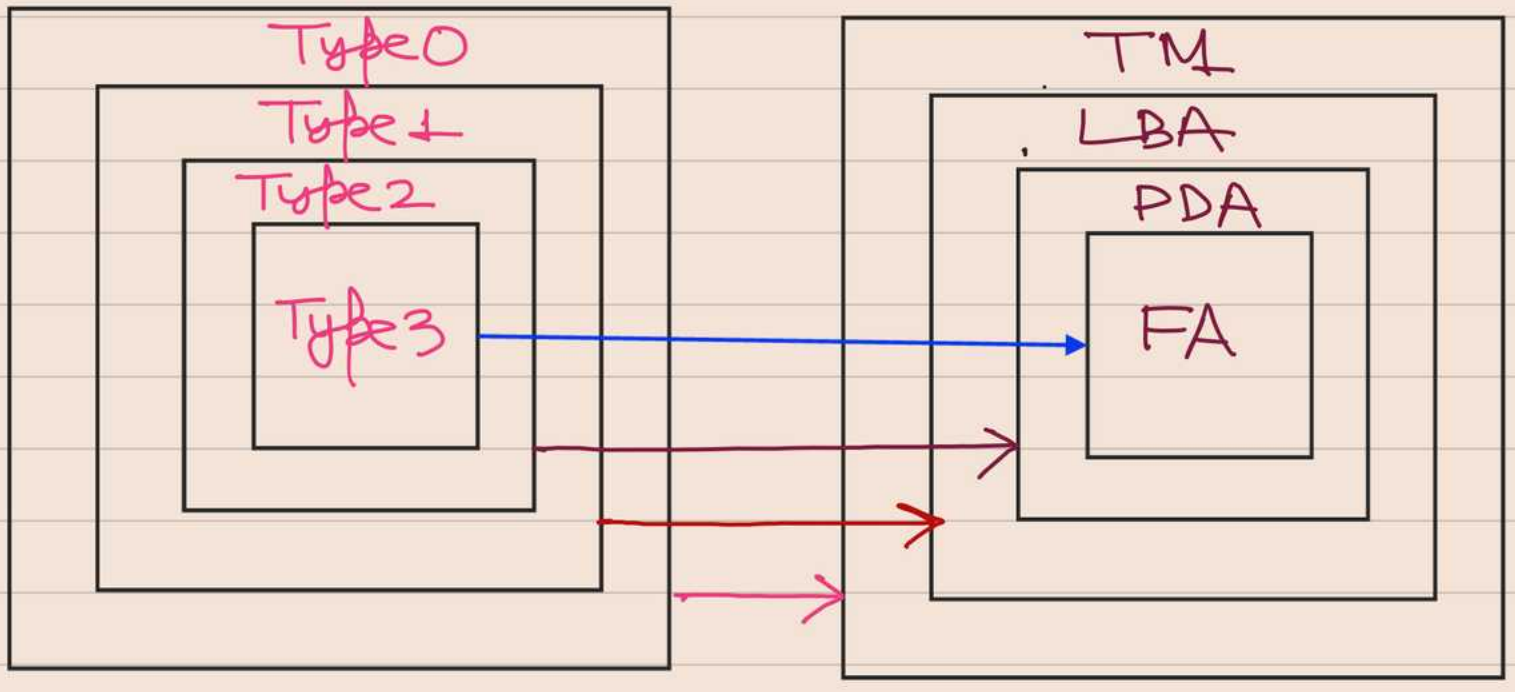
$$A \to w^R B \mid w^R$$

One more clarification regarding Type-3

Do not mix Right linear with left linear, such a grammar may not be a Regular grammar

Ex:
$S \to 0A$
$S \to 1B$
$S \to \in$
$A \to S0$
$B \to S1$

This grammar generate language $L = \{ww^R \mid w \in \{0,1\}^*\}$ which is NOT regular.



Type3: Regular grammar
Type2: Context free grammar
Type1: Context sensitive grammar
Type0: Unrestricted

FA: Finite Automata
PDA: Push down Automata
LBA: Linear Bounded Automata
TM: Turing M/C

Ex: ① $S \to abA$
$A \to baB$
$B \to aA$
$B \to bb$
$\Rightarrow$ Regular grammar

Ex: ② $S \to abB$
$A \to aaBb$
$B \to bbAa$
$A \to \in$
$\Rightarrow$ Context free grammar

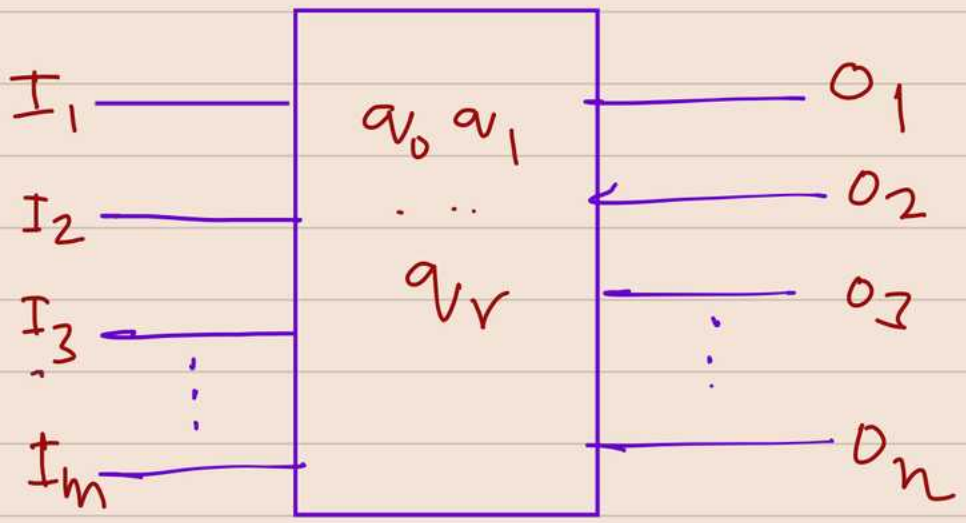
Ex: ③ $S \to aSb$
$S \to SS$
$S \to \in$
$\Rightarrow$ Context free grammar

Ex: ④ $S \to aSa$
$S \to bSb$
$S \to \in$
Context free grammar

Ex: ⑤
$S \to aTb \mid ab$
$aT \to aaTb \mid aC$
Context sensitive grammar

Automata (singular form "Automaton")

General Automaton:



Key ingredients:

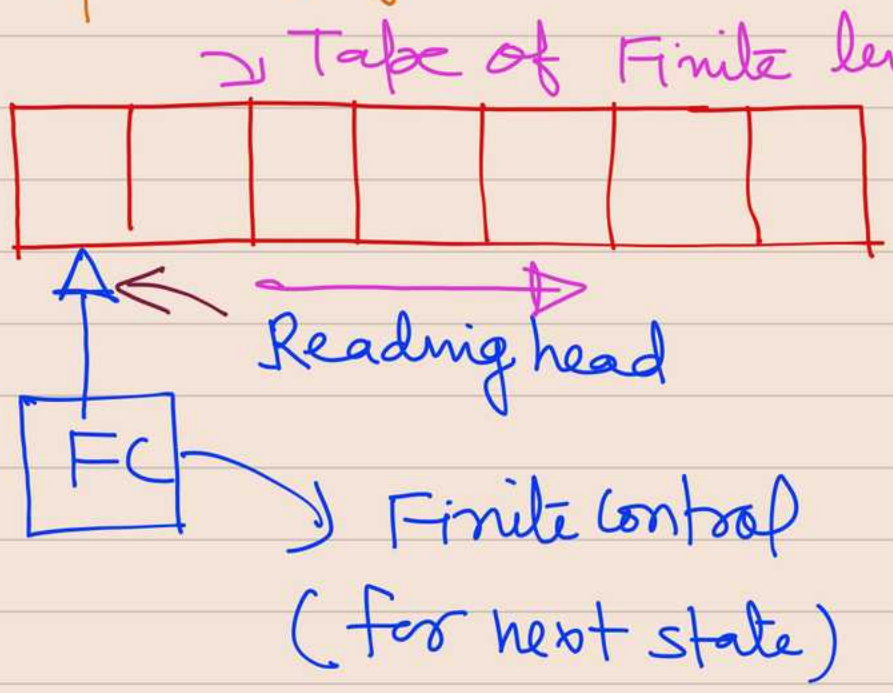
1. Set of inputs $\Sigma = \{I_1, I_2, \ldots I_m\}$
2. Set of outputs $A = \{O_1, O_2, \ldots O_n\}$
3. Set of states $Q = \{q_0, q_1, \ldots q_r\}$
4. State relation (or transition fn)
5. Output relation

"Automaton is a system where Energy, material, of information gets processed / transformed without Human intervention"

Finite Automata

" Schematic diagram of finite automata

[Note: this is only for the understanding of the functioning of finite automata (FA) Otherwise Finite automata (FA) is an abstract mathematical Model that recognizes Type 3 (Regular) Grammar.]



(1) Tape is of finite length
(2) Reading head can only read symbol under the head (CAN NOT WRITE)
(3) Reading head can only move towards Right (NO LEFT MOVE)

Finite Automata (FA):

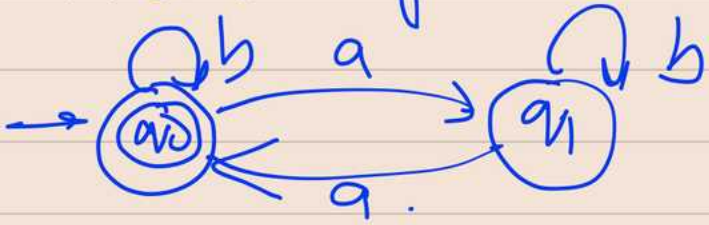
Mathematically, FA is a five-tuple

$(Q, \Sigma, \delta, q_0, F)$

- q_0 → Initial/start state, $q_0 \in Q$.
- F → Finite states, $F \subseteq Q$
- Q → Finite set of states
- Σ → alphabet of input symbols.
- δ → transition function, $\delta : Q \times \Sigma \to Q$.

Note: representing finite automata = representing transition function.

→ Transition diagram



Transition Table

	a	b
q_0	q_1	q_0
q_1	q_0	q_1

Design Guidelines of finite automata:

(Note: these are only guidelines and NOT rules)

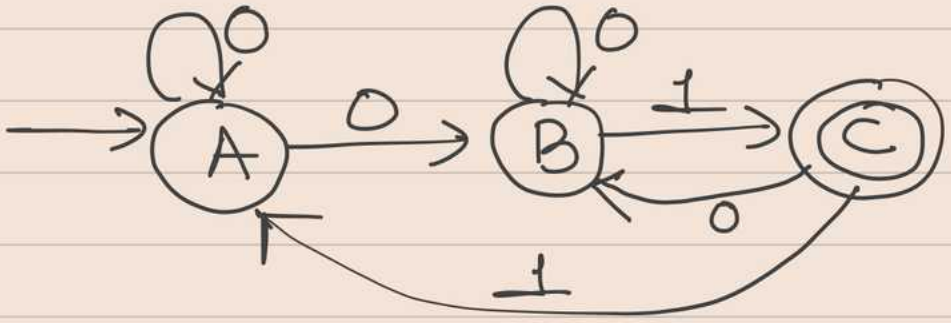
1. Design skeleton and consider minimum string of acceptance / rejection.
2. Explore all possibilities at every node. (May consider "sink / Trap / dead" state also)

Ex:1

Design a FA to recognize

$L = \{ x \mid x \text{ ends with } 01, x \in \Sigma^* \}$,

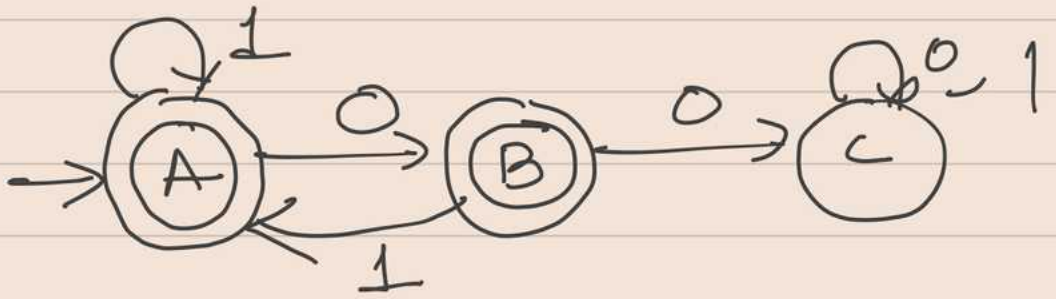
$\Sigma = \{0, 1\}$



Ex2:

Design a FA to recognize.

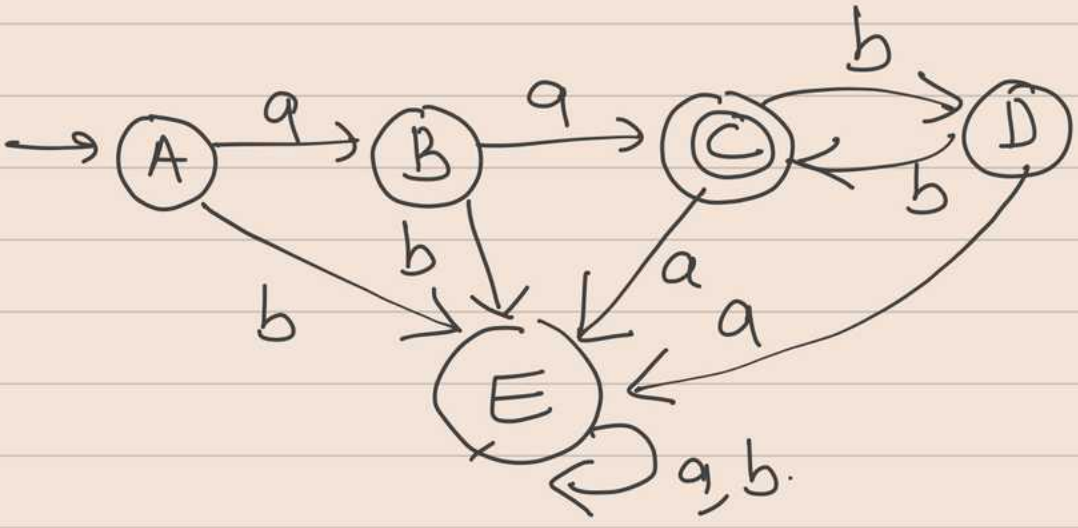
$L = \{x \mid x$ does not contain 00, $x \in \Sigma^*, \Sigma = \{0, 1\}\}$



Ex3:

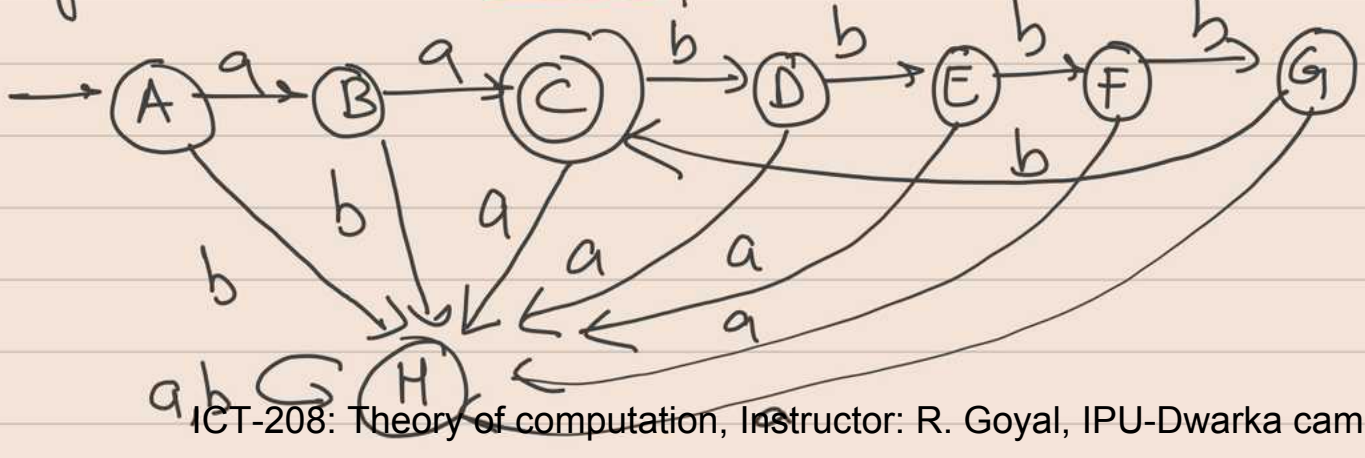
Design a FA

for $L = \{a a b^n \mid n$ is divisible by 2$\}$



Ex4:

Design FA for $L = \{a a b^{5i} \mid i \geq 0\}$



Accepting the Union, Intersection, or difference of two languages :

Suppose $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$ and

$M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$ are finite automata accepting L_1 and L_2 respectively.

Let M be the FA $(Q, \Sigma, \delta, q_0, F)$, where

$Q = Q_1 \times Q_2$ ← Note

$q_0 = (q_{01}, q_{02})$ ← Note

and transition function δ is defined by the formula

$\delta((p,q),a) = (\delta_1(p,a), \delta_2(q,a))$ ← Note

$\forall\, p \in Q_1, \forall\, q \in Q_2$, and $\forall\, a \in \Sigma$

Then

1. if $F = \{(r,s) \mid r \in F_1 \text{ or } s \in F_2\}$
   then M accepts $L_1 \cup L_2$

2. if $F = \{(r,s) \mid r \in F_1 \text{ and } s \in F_2\}$
   then M accepts $L_1 \cap L_2$

3. if $F = \{(r,s) \mid r \in F_1 \text{ and } s \notin F_2\}$
   then M accepts $L_1 - L_2$

Ex: let L_1 and L_2 be the languages

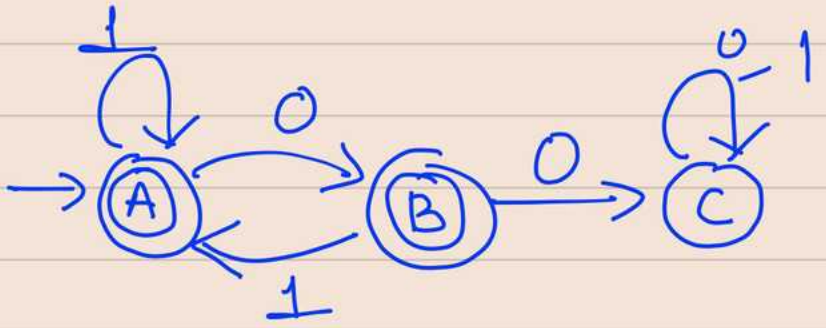
$L_1 = \{ x \in \{0,1\}^* \mid x \text{ does not contain '00'} \}$

$L_2 = \{ x \in \{0,1\}^* \mid x \text{ ends with 01} \}$

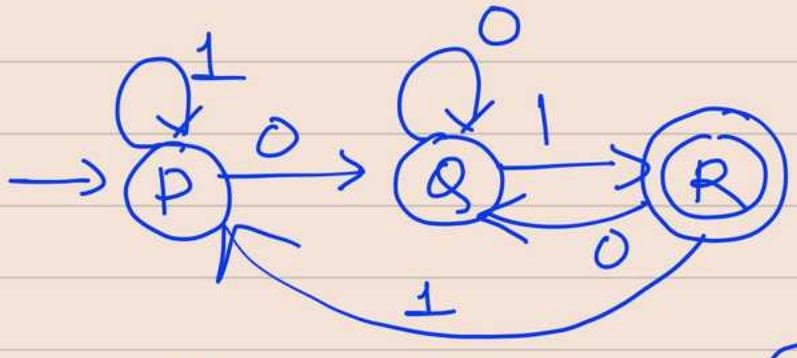
Design FA to recognize $L_1 \cap L_2$

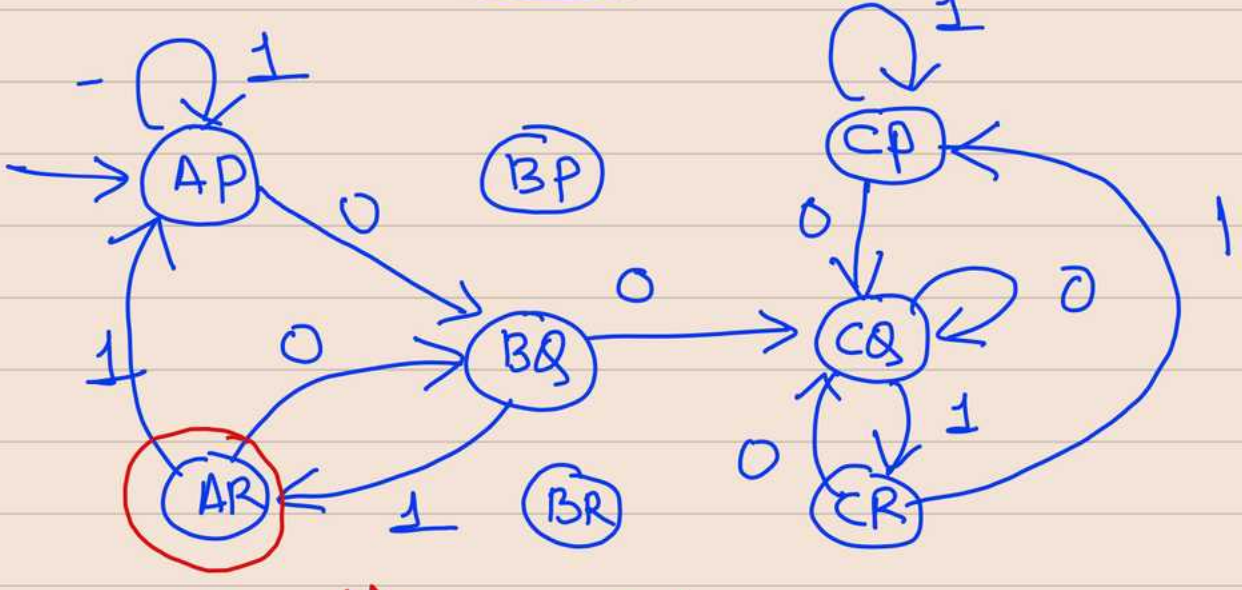
Soln

M_1:

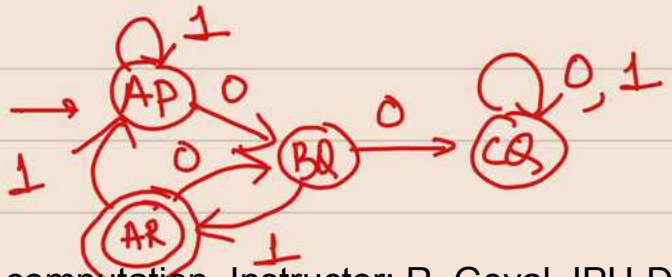


M_2:





⇒



Examples (Try yourself):

1. Design a FA to accept L

$L = \{ x \in \{a,b\}^* \mid$ In x, No. of a's is divisible by 2 and No. of b's is divisible by 3 $\}$

2. Let $L_1 = \{ x \in \{a,b\}^* \mid x$ contains the substring 'ab' $\}$

$L_2 = \{ x \in \{a,b\}^* \mid x$ contains the substring 'bba' $\}$

Construct a FA to accept $L_1 \cup L_2$, $L_1 \cap L_2$, and $L_1 - L_2$.

Non-deterministic Finite Automata :
(NDFA or NFA)

Ex: Design a FA for

ThirdLast 1 = $\{ w \in \{0,1\}^* \mid$ the third from last character is a 1 $\}$



Non-deterministic Finite Automata :
(NFA)

Mathematically NFA is a 5-tuple

$(Q, \Sigma, \delta, q_0, F)$

Q: finite set of states

Σ: finite set of symbols, input alphabet

δ:

$$Q \times (\Sigma \cup \{\epsilon\}) \longrightarrow P(Q)$$ → Powerset of Q

OR

$$Q \times (\Sigma \cup \{\epsilon\}) \longrightarrow 2^Q$$

Note: $\Sigma \cup \{\epsilon\}$ means Σ augmented by ϵ. This indicates that machine can move forward without an input symbol.

$q_0 \in Q$: initial state

$F \subseteq Q$; set of accept states

Note about δ:

(1) Range of δ is a set of states, so that, given a state and a symbol, the machine can go to several states. We assume that it will always take the correct move. Note

(2) Move can be made without using the input. These are $\in$-transitions and they can be made before or after an input symbol is used. Note

Note:

NDFA 'M' accepts $w \in \Sigma^*$ iff there exists at least one path (sequence of moves) leading to acceptance.

More on NFA :

NFA is like a DFA but with three extra features:

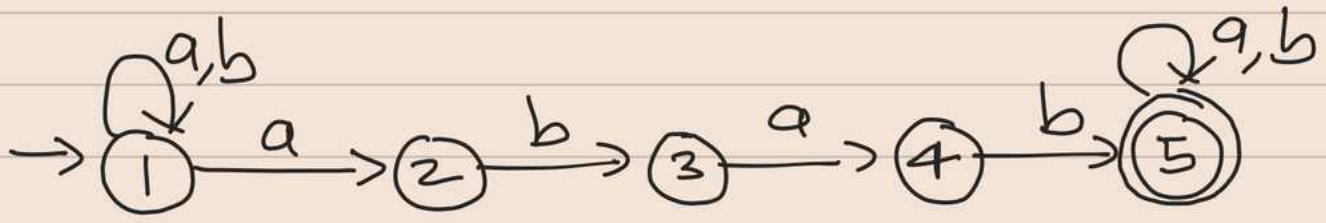
1. Easier to construct because they can be composed in a modular fashion.
2. Easier to read and they tend to be much smaller, therefore easy to describe as well.
3. Computationally, they are equivalent to DFA's, in the sense that they recognizes the same languages.

NFA's important features

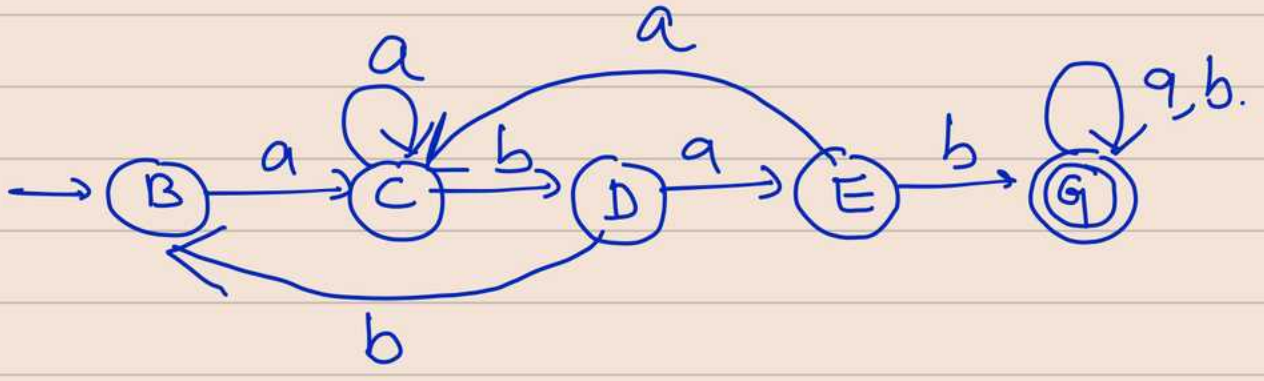
1. Epsilon transition possible.
   ↓ Easy to represent optional things
2. Missing transitions
   ↓ All possibilities need not be explored.
3. Multiple transitions
   ↓ Need to guess which path will accept the string.

Ex:

the automata (NFA) below accepts string containing the substring abab.



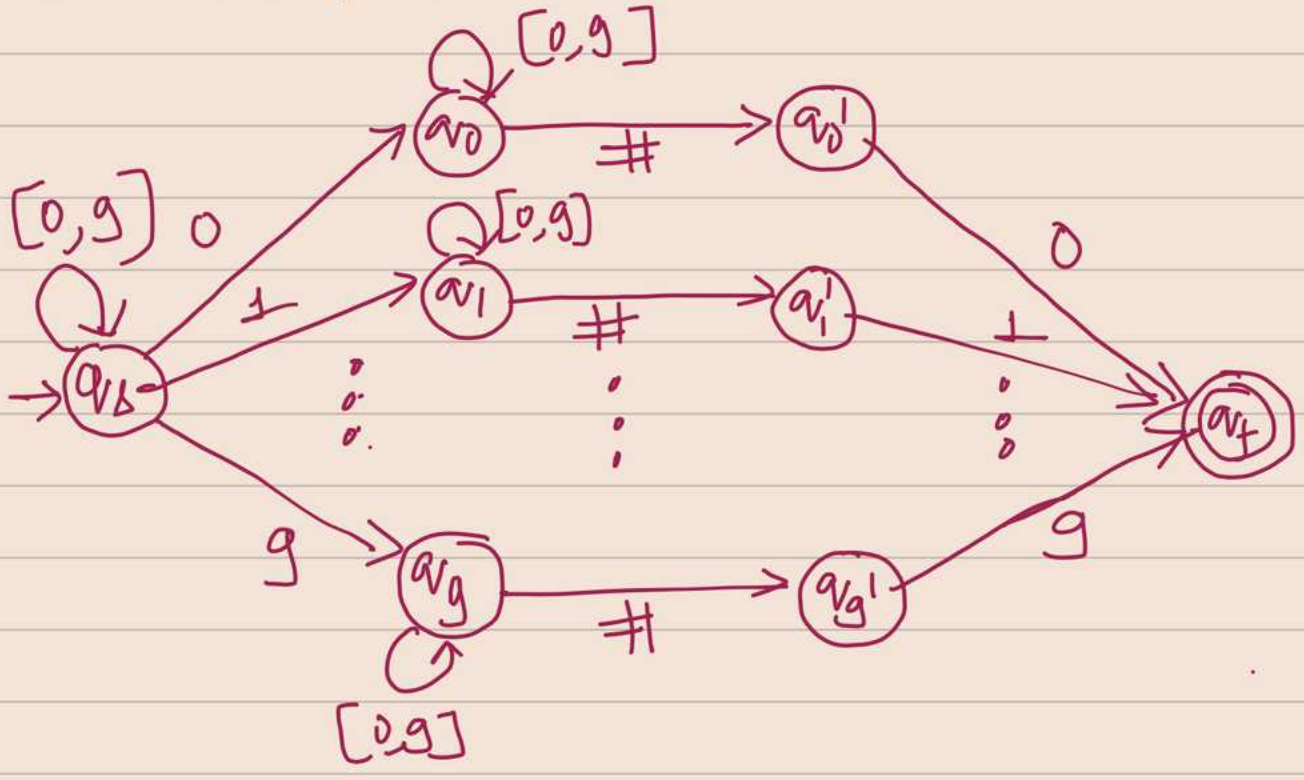
the respective DFA, shown below, needs a lot more transitions and is somewhat harder to read.



Let $\Sigma' = \{0,1,2,\ldots 9\}$. Let us denote this as $[0,9]$ in short form. Let

$$L = \{w\#c \mid c \in \Sigma',\ w \in \Sigma'^{*} \text{ and } c \text{ occurs in } w\}$$

For example, the word $314159\#5$ is in L, and so is $314159\#3$. But the word $314159\#7$ is not in L.



Relationship between NFA and ϵ-NFA :

Qn: Are there languages only accepted by ϵ-NFA's ?

Answer: No, every ϵ-NFA (A) can be converted to an NFA (A') without ϵ-transitions so that $L(A) = L(A')$

Construction: Let $A = (Q, \Sigma, \delta, q_0, F)$

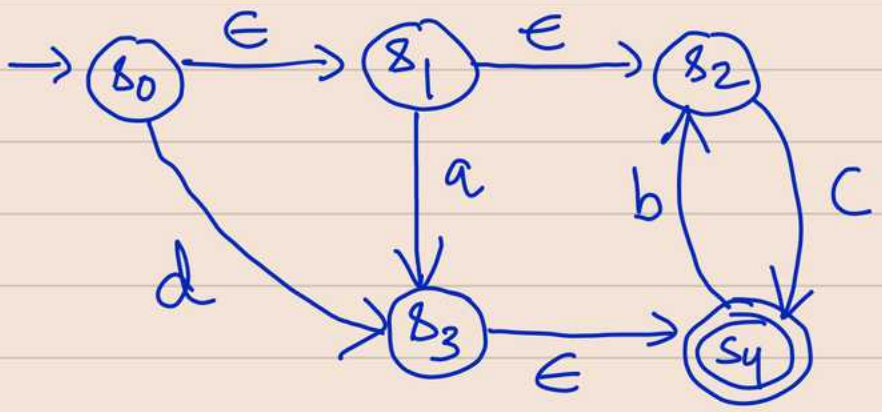
and $A' = (Q, \Sigma, \delta', q_0, F')$

where

- Make $q \in Q$ an accepting state in A' if q can reach an accepting state in A by ϵ-transitions

$$F' = \{ q \in Q \mid \epsilon\text{-closure}(q) \cap F \neq \phi \}$$

- $\delta'(q, a) = \epsilon\text{-closure}\left(\delta(\epsilon\text{-closure}(q), a)\right)$



ϵ-closure (s_0) = $\{s_0, s_1, s_2\}$
OR
ϵ^*

Note: each state is there in the ϵ^* of itself.

$\epsilon^*(s_1) = \{s_1, s_2\}$

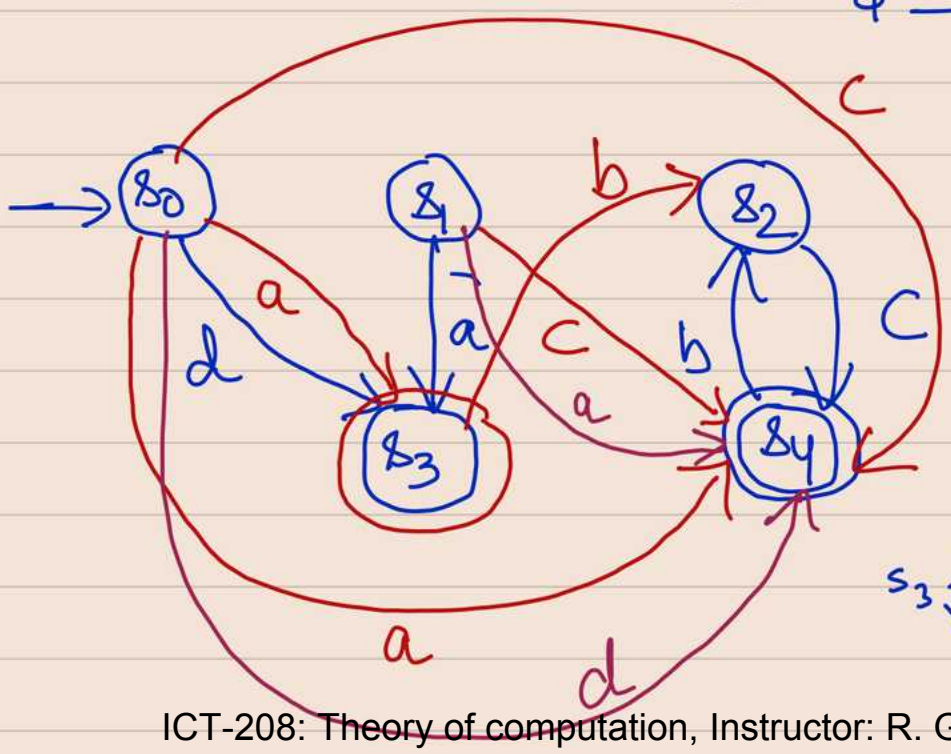
$\epsilon^*(s_2) = \{s_2\}$

$\epsilon^*(s_3) = \{s_3, s_4\}$

$\epsilon^*(s_4) = \{s_4\}$

s_0 on a :

	ϵ^*	a	ϵ^*	
s_0 →	s_0	ϕ	ϕ	
	s_1	s_3	s_3	s_3, s_4
			s_4	
	s_2	ϕ	ϕ	



	ϵ^*	c	ϵ^*
s_0 →	s_0	ϕ	ϕ
	s_1	ϕ	ϕ
	s_2	s_4	s_4

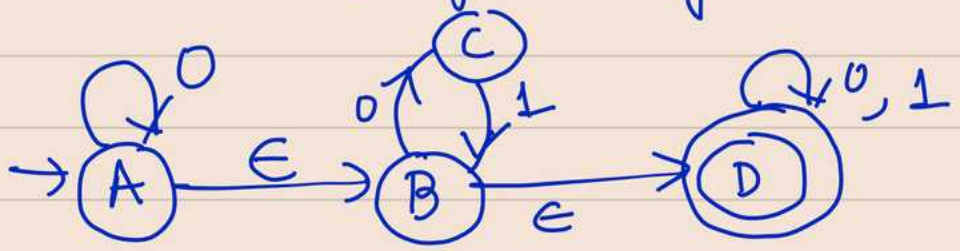
⋮

Similarly other calculation

	ϵ^*	b	ϵ^*
s_3	s_3	ϕ	ϕ
	s_4	s_2	s_2

Ex:-

Convert following ∈-NFA into NFA

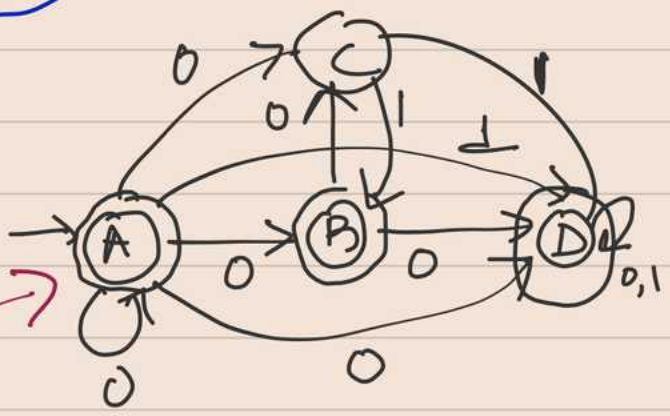


$\in^*(A) = \{A, B, D\}$

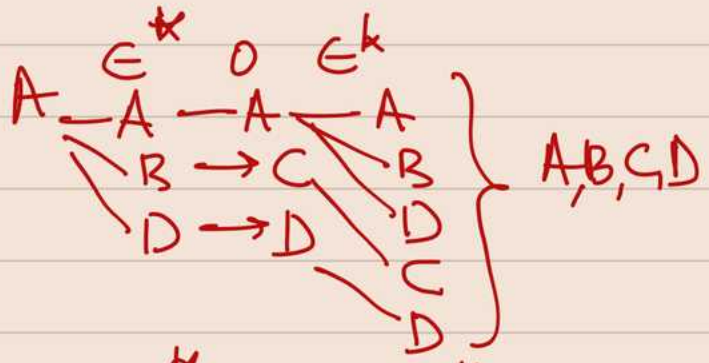
$\in^*(B) = \{B, D\}$

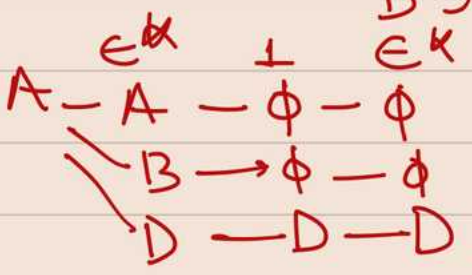
$\in^*(C) = \{C\}$

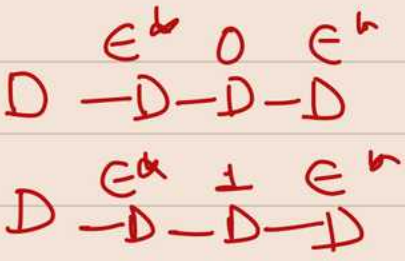
$\in^*(D) = \{D\}$

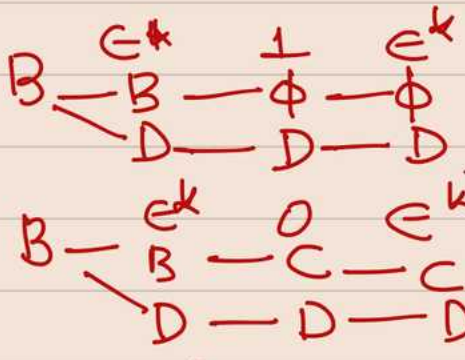


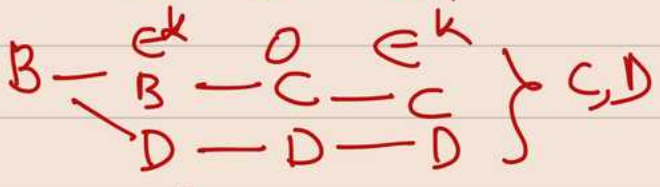
	0	1
A	A, B, C, D	D
B	C, D	ϕ
C	ϕ	B, D
D	D	D

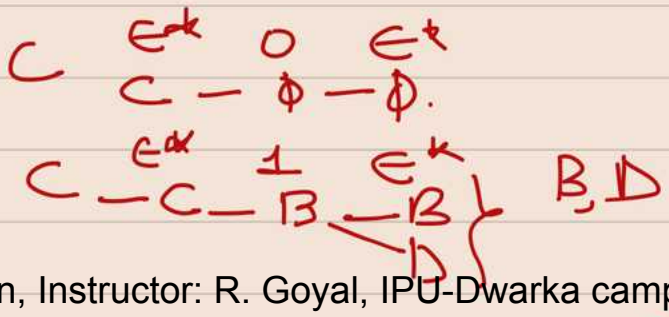












Regular Expression:

"Regular expression (R.E.) is an algebraic way (expression language) to denote regular language"

(1) $\forall a \in \Sigma$, a is a R.E. $\Rightarrow L(a) = \{a\}$

$\in$ is a R.E $\Rightarrow L(\in) = \{\in\}$

ϕ is a R.E $\Rightarrow L(\phi) = \phi$

→ Primitive regular expressions.

(2) If r_1 and r_2 are R.E, then so are $r_1 + r_2$, $r_1 \cdot r_2$, r_1^* and (r_1)

operators that combine R.E's

Disambiguate notation

$\Rightarrow L(r_1 + r_2) = L(r_1) \cup L(r_2)$,

$L(r_1 \cdot r_2) = L(r_1) \cdot L(r_2)$

$L(r_1^*) = (L(r_1))^*$.

(3) A string s is a regular expression if and only if it can be obtained by applying operation in (2) to the primitive regular expression in (1)

Ex: $\Sigma = \{a, b\}$

$(a+\epsilon)\cdot(b\cdot a+\phi^*)$ is a valid regular expression,

but $(a \cap \epsilon) - (b^R \cdot \phi)$ is NOT a valid regular expression.

order of Precedence (excluding ())

Kleene's closure
Positive closure
Concatenation
Union.

Examples: let $\Sigma = \{a, b\}$.

(1) starts with ab — $ab(a+b)^*$

(2) Ending with ab. — $(a+b)^*ab$

(3) Contains a substring aab — $(a+b)^*aab(a+b)^*$

(4) start and end with a — $a + a(a+b)^*a$

(5) start and end with the same symbol.

$a(a+b)^*a + b(a+b)^*b + a + b + \epsilon$

(6) start and end with different symbol

$a(a+b)^*b + b(a+b)^*a$

(7) $|w| = 3$ $\quad (a+b)(a+b)(a+b)$

(8) $|w| \geq 3$ $\quad (a+b)(a+b)(a+b)(a+b)^*$

(9) $|w| \leq 3$ $\quad (a+b+\epsilon)(a+b+\epsilon)(a+b+\epsilon)$

(10) $|w|_a = 2$ $\quad b^*ab^*ab^*$.

(11) $|w| = 0 \pmod 3$ $\quad ((a+b)(a+b)(a+b))^*$

Important identities for regular expressions:

I_1: $\phi + R = R$ $\quad$ I_2: $\phi R = R\phi = \phi$

I_3: $\Lambda R = R\Lambda = R$ $\quad$ I_4: $\Lambda^* = \Lambda$, $\phi^* = \Lambda$

I_5: $R + R = R$ $\quad$ I_6: $R^*R^* = R^*$.

I_7: $RR^* = R^*R$ $\quad$ I_8: $(R^*)^* = R^*$.

I_9: $\Lambda + RR^* = R^*$ $\quad$ I_{10}: $(PQ)^*P = P(QP)^*$.

I_{11}: $(P+Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$

let P and Q be two regular expressions over Σ, if P does not contain $\in$, then the following equation in R

$$R = Q + RP$$

has a unique solution $R = QP^*$

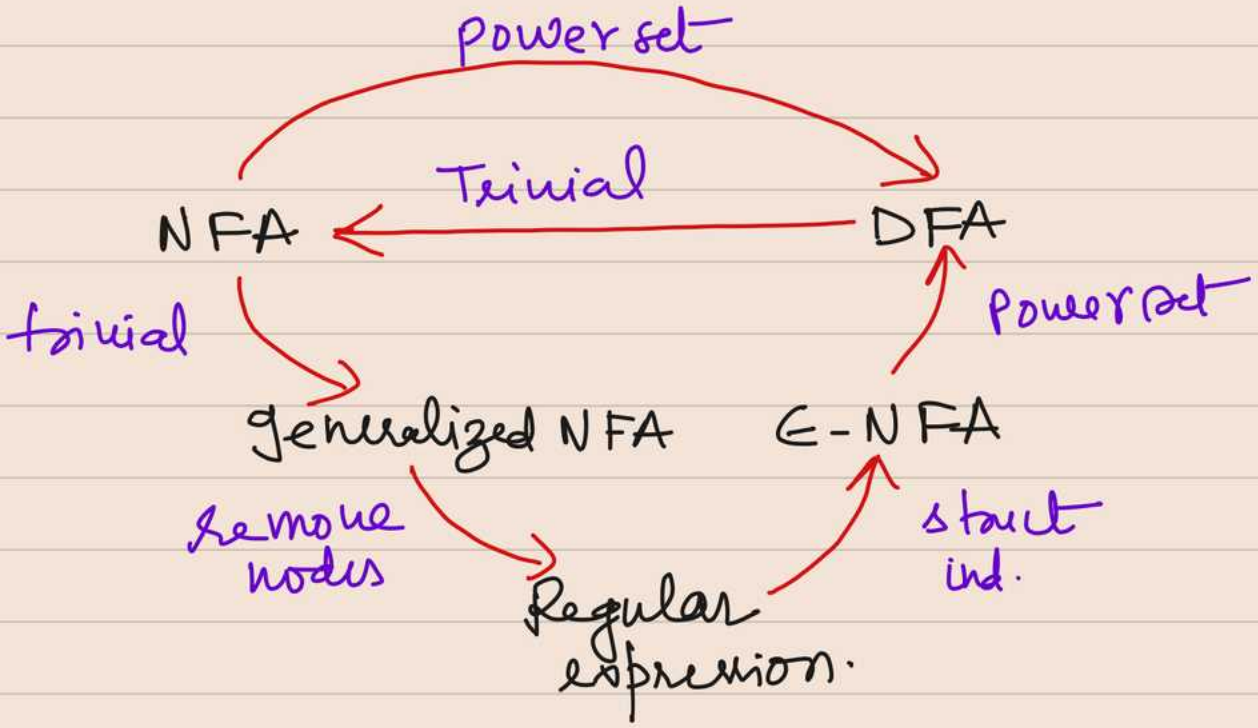
Note:

$R = PR + Q$ and $R = P^*Q$ } Right

$R = RP + Q$ and $R = QP^*$ } left

Kleene's theorem:

"the set of regular languages, the set of NFA-recognizable languages and the set of DFA-recognizable language are all same"

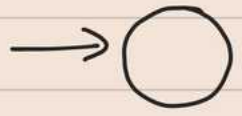


Converting regular expressions to DFA :

Note: To convert a R.E to an NFA, first convert it to an ϵ-NFA, then convert that to a DFA.

R.E

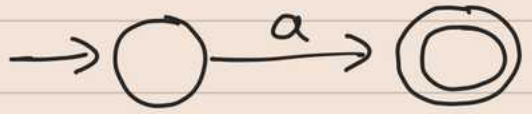
$r = \phi$



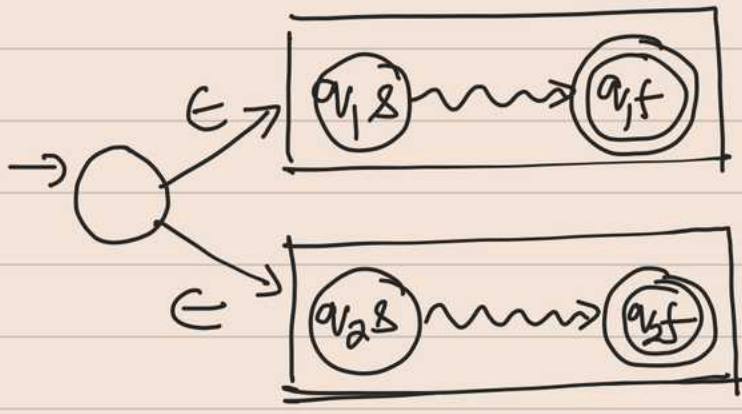
$r = \epsilon$



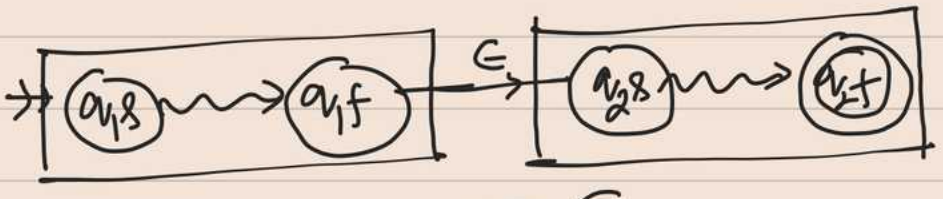
$r = a$



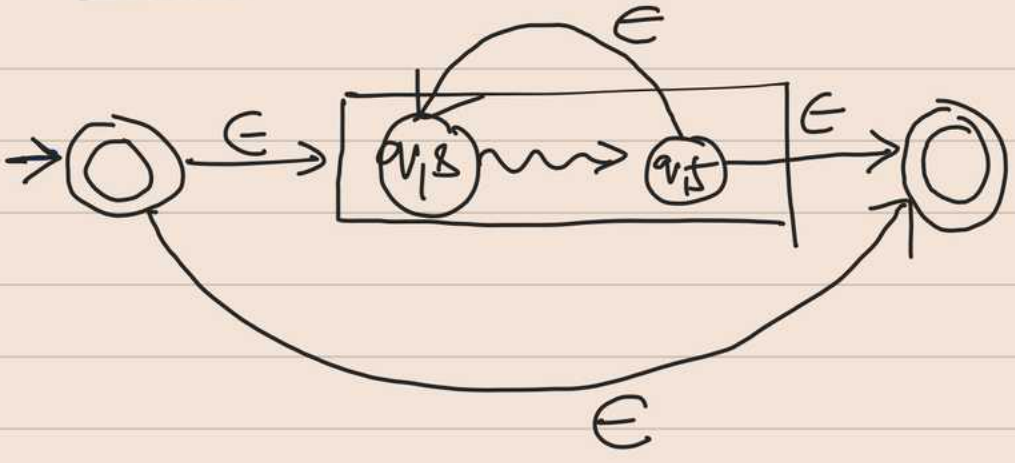
$r = r_1 | r_2$



$r = r_1 \cdot r_2$



$r = r_1^*$

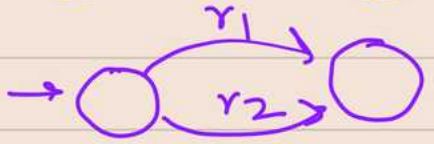


A simplified view: (useful when converting R-E: to DFA)

$r = r_1 . r_2$



$r = r_1 + r_2$



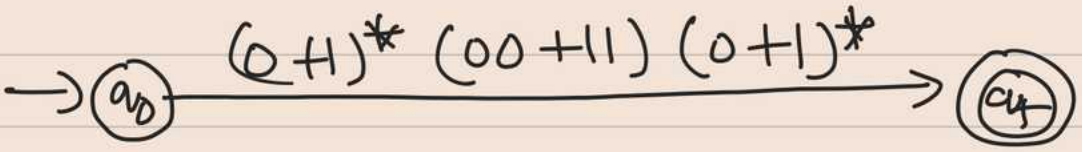
$r = r_1^*$

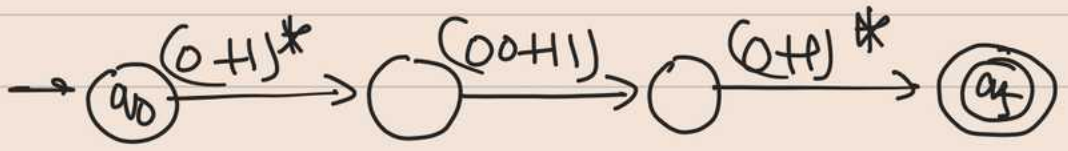


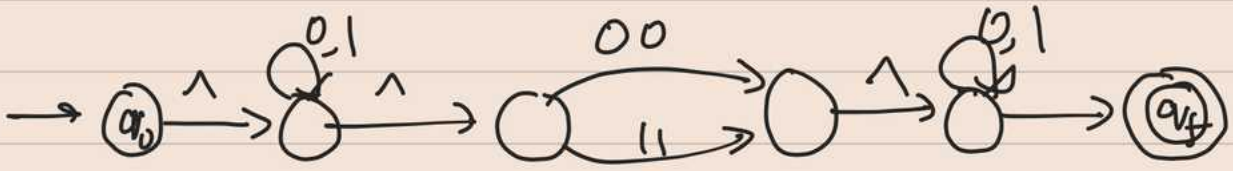
→ Not precise, but useful for solving problems

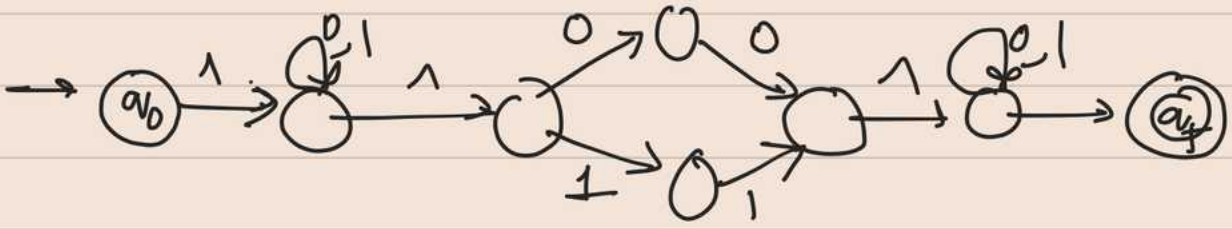
Ex: Construct a Finite automata equivalent to the regular expression

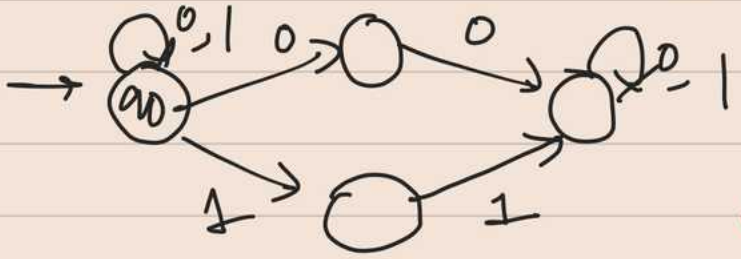
$$(0+1)^*(00+11)(0+1)^*$$





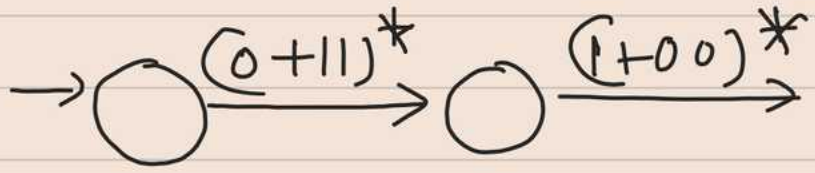


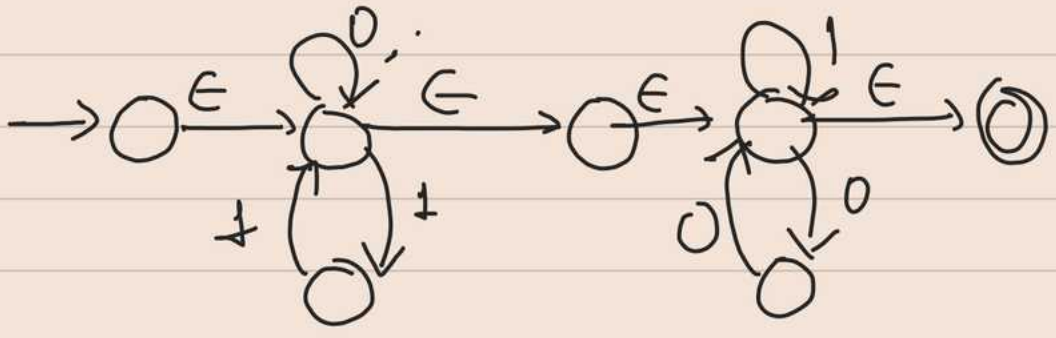




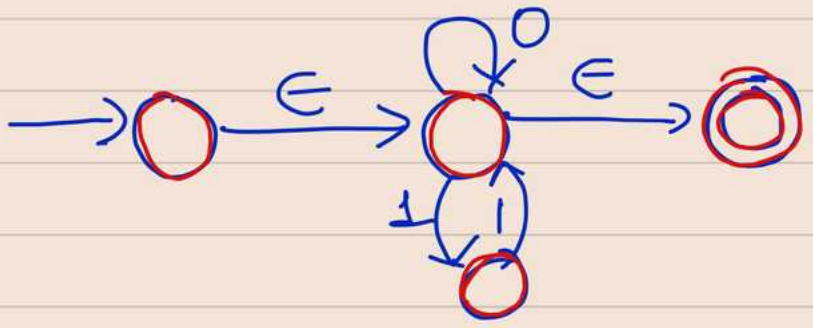
} this is NFA, convert this into a DFA

Ex: $(0+11)^*(1+00)^*$



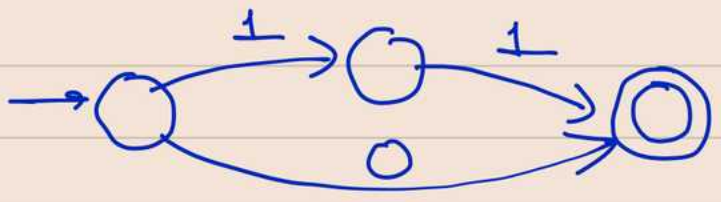


Ex FA for $(11+0)^*$

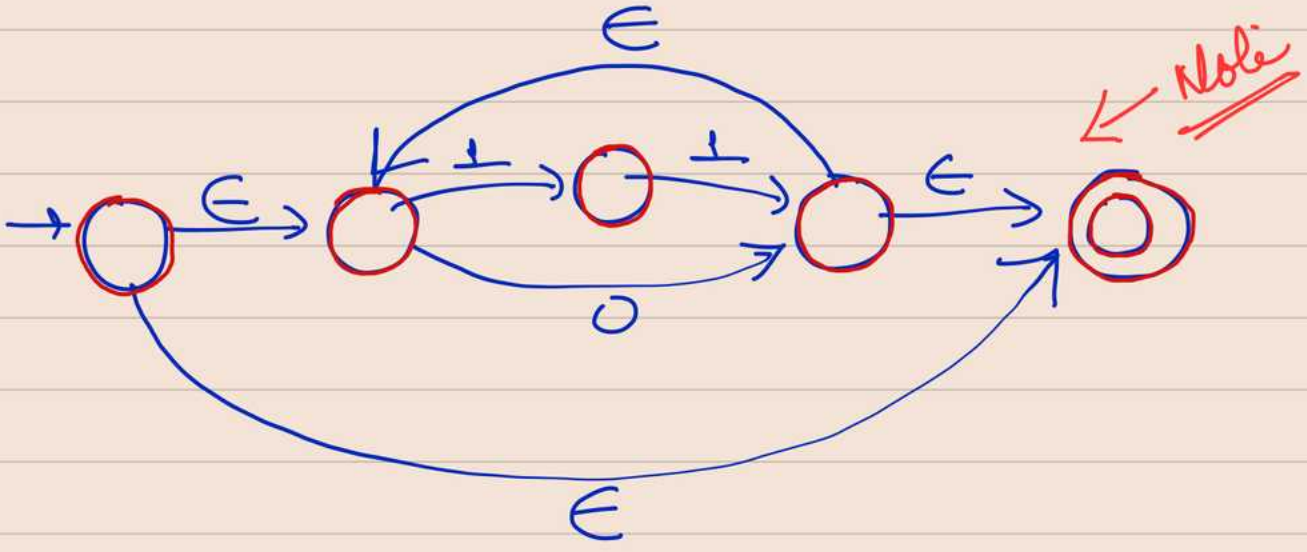


Note

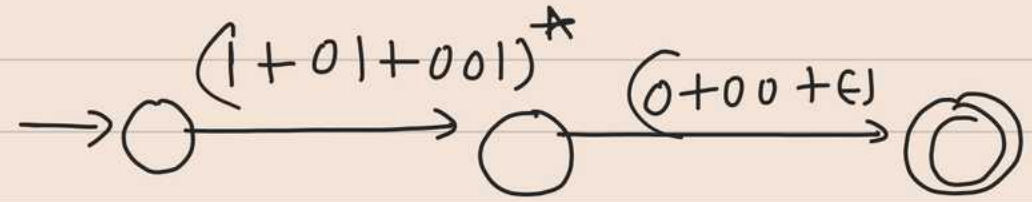
OR FA for $11+0$ is

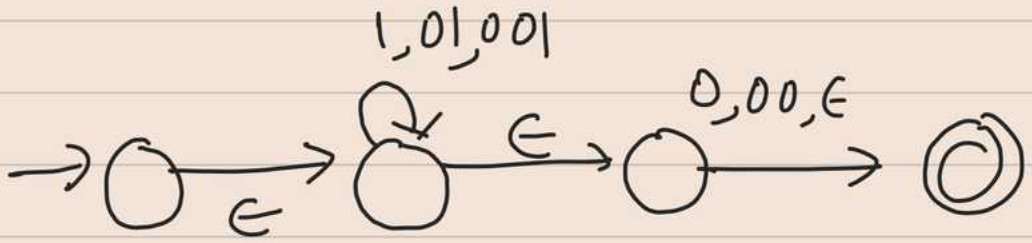


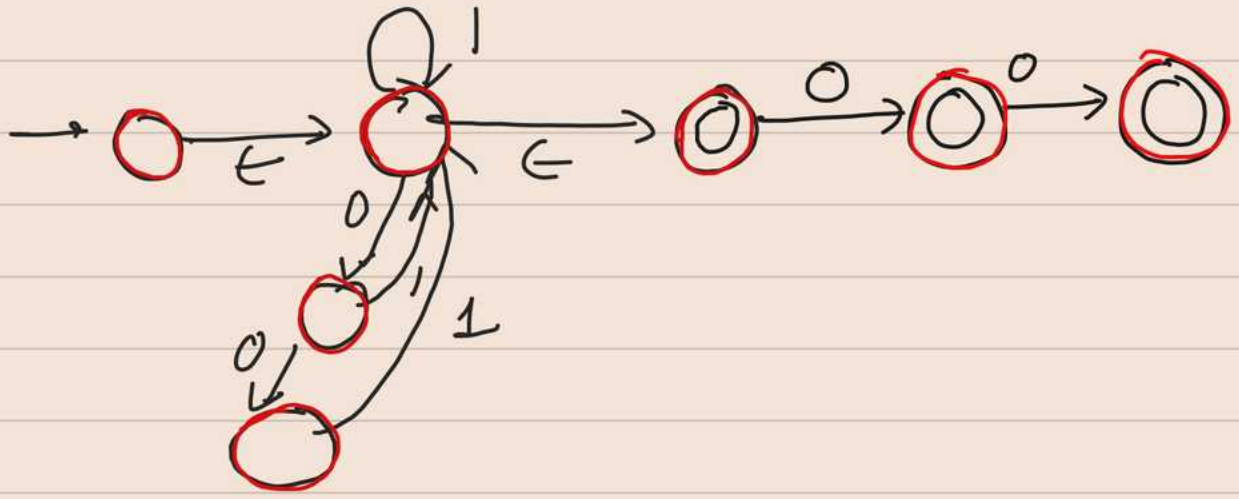
$\Rightarrow$ FA for $(11+0)^*$ would be



$r = (1+01+001)^* . (0+00+\epsilon)$





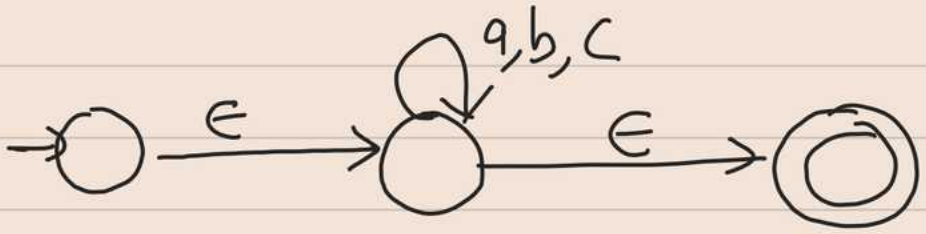


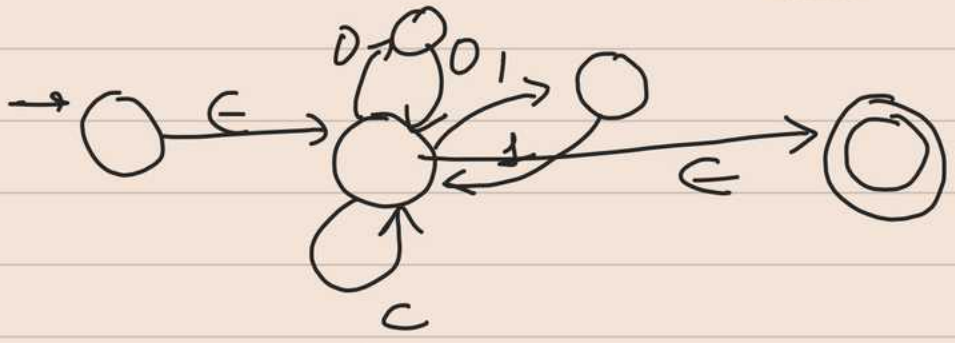
Ex

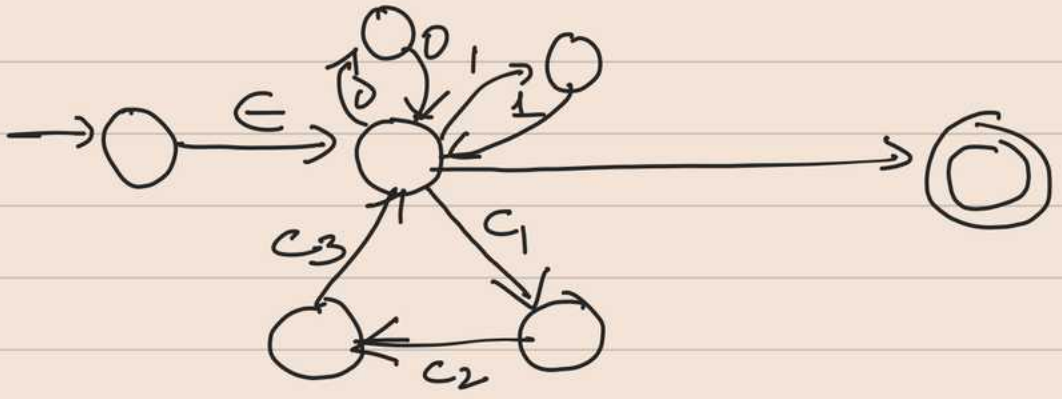
$$\left[\underbrace{00}_{a} + \underbrace{11}_{b} + \underbrace{(01+10)}_{c_1}\underbrace{(00+11)^*}_{c_2}\underbrace{(01+10)}_{c_3}\right]^*$$

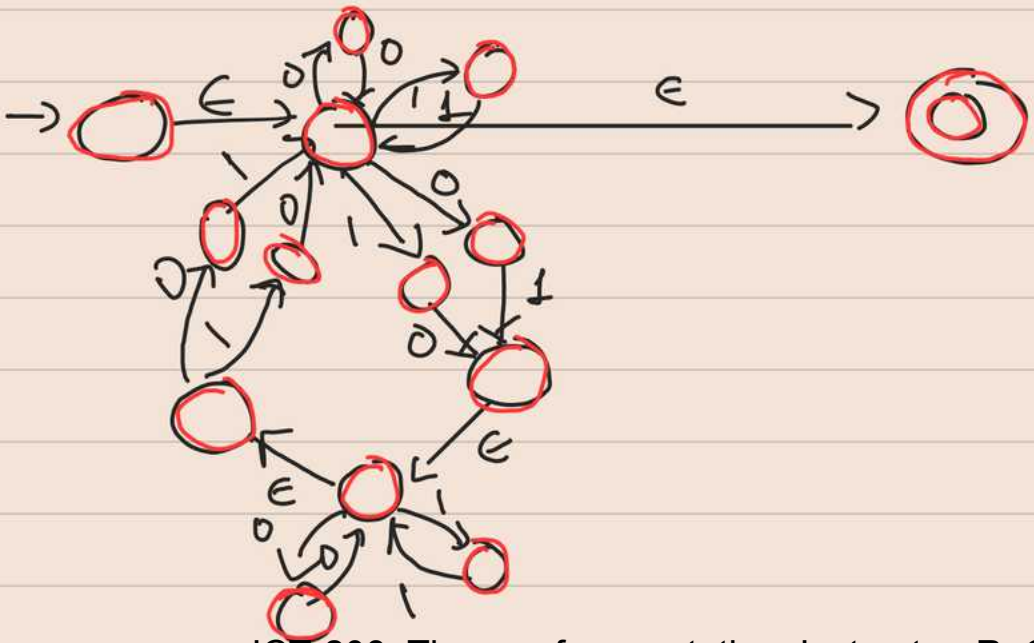
γ











Converting NFA to R.E.:

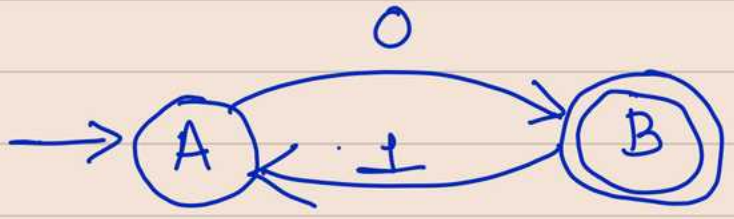
→ Two Methods
① State elimination method
② By solving language equations

State elimination Method:

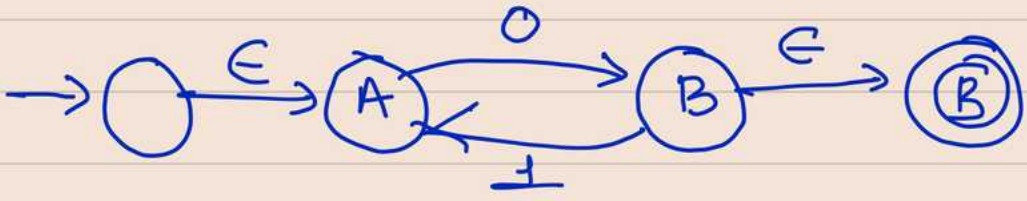
Case-1 → If incoming edges (transitions) towards initial state
Then use ∈-move, and create a new state

Case-2 → If outgoing edges (transitions) from final state
Then use ∈-move and create a new final state.

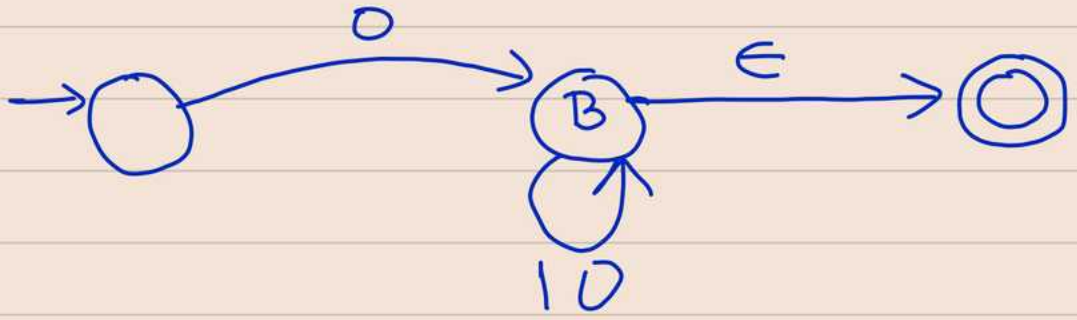
Ex:



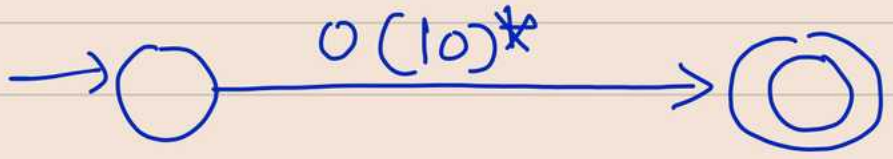
} Case 1 and 2 both



Let us eliminate 'A' first, Ask yourself where can you move by using ONLY. State 'A' as an intermediate?

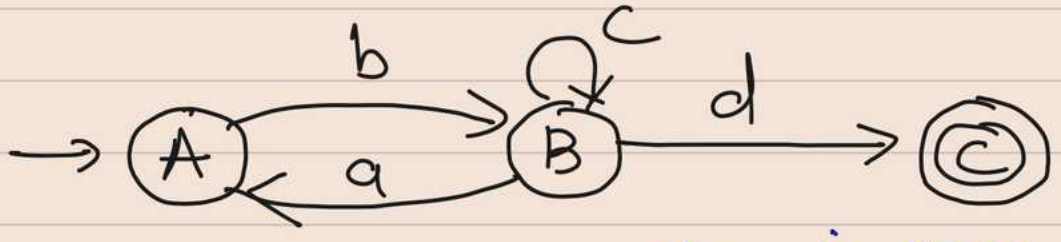


Now Eliminate B

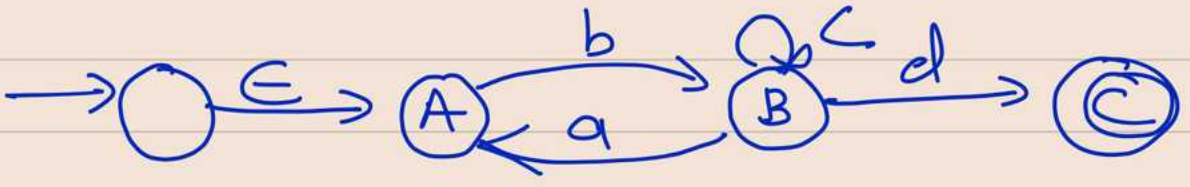


0(10)* is the equivalent R.E of the given NFA.

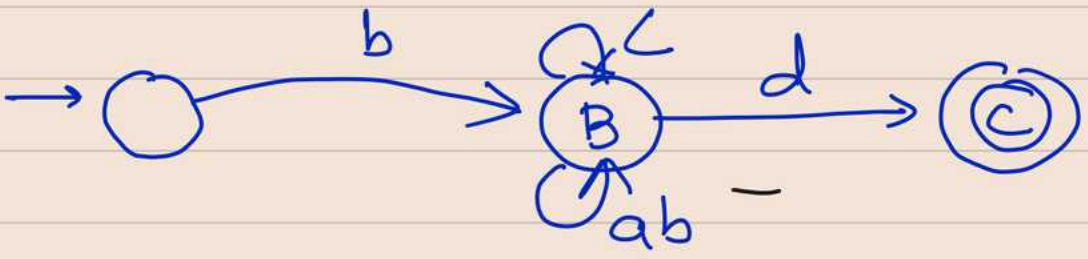
Ex:

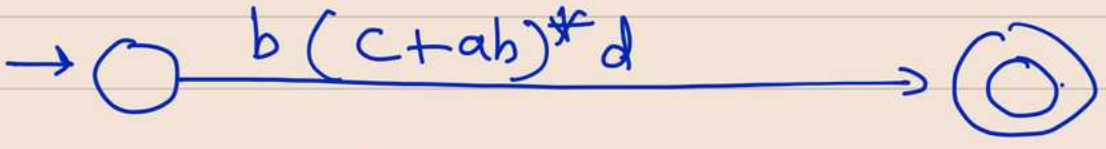


this is case-1 only



Remove state A

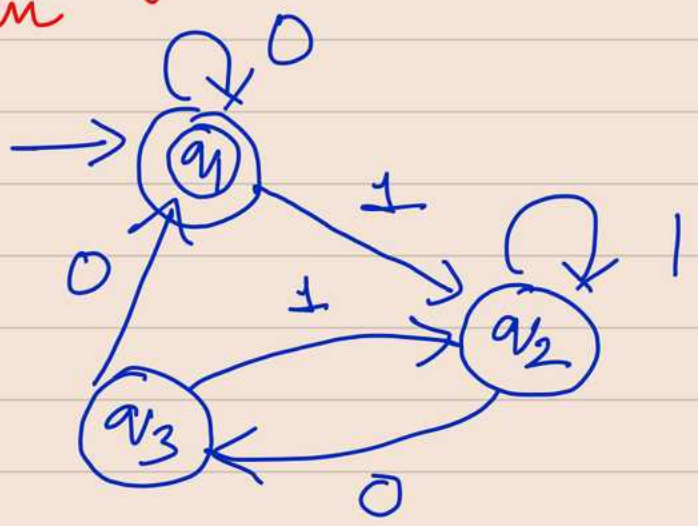




⇒ b(c+ab)*d is the equivalent Regular expression.

Exercise

Construct a regular expression corresponding to the following state diagram



Method – II (By solving language equations)

Assumptions

→ Does not have $\in$-moves

→ Must have only one initial state

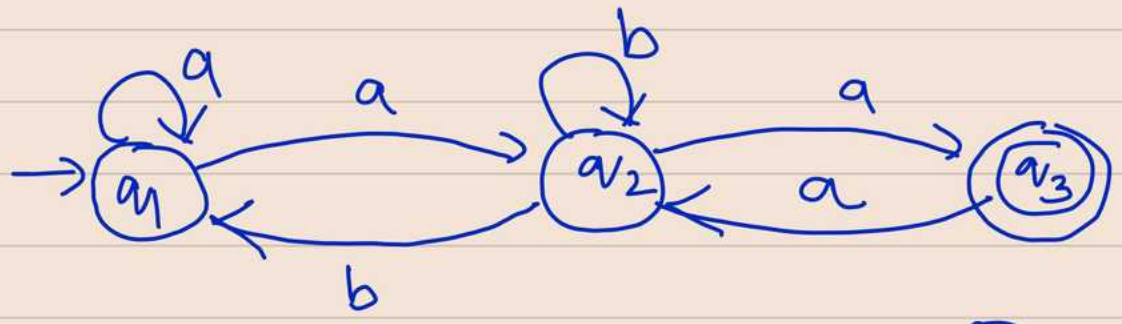
(1) Here, form the state equations like

$$q_1 = q_1 w_{11} + q_2 w_{21} + \cdots + q_n w_{n1} + \in$$

($\in$ if q_1 is an initial)

where w_{ij} = regular expression representing the set of labels of edges from q_i to q_j

(2) Form state equations for other states as well. and then solve these equations for final state (or final states)



$$q_1 = q_1 a + q_2 b + \in \quad \text{— (1)}$$

$$q_2 = q_2 b + q_1 a + q_3 a \quad \text{— (2)}$$

$$q_3 = q_2 a. \quad \text{— (3)}$$

use eqn (3)

$$q_2 = q_2 b + q_1 a + q_2 a a$$
$$= q_1 a + q_2 (b + aa)$$

using Arden's Rule

$$q_2 = q_1 a (b+aa)^*$$

put this value in eqn - (1)

$$q_1 = q_1 a + q_1 a (b+aa)^* + \in$$
$$q_1 = q_1 (a + a(b+aa)^*) + \in$$

$\Rightarrow$ using Arden's Rule

$$q_1 = (a + a(b+aa)^*)^*$$
$$q_2 = (a + a(b+aa)^*)^* a (b+aa)^*$$

$$\boxed{q_3 = (a + a(b+aa)^*)^* a (b+aa)^* a.}$$

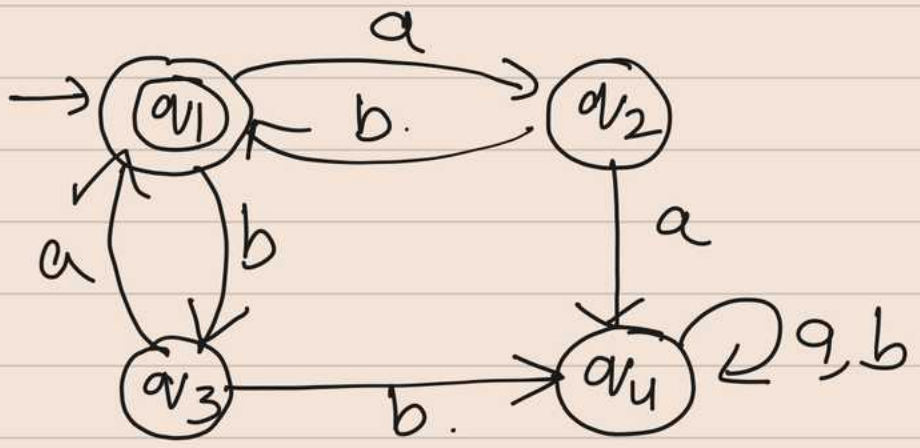
Final equivalent R.E.

Qn: What if you have multiple final states?

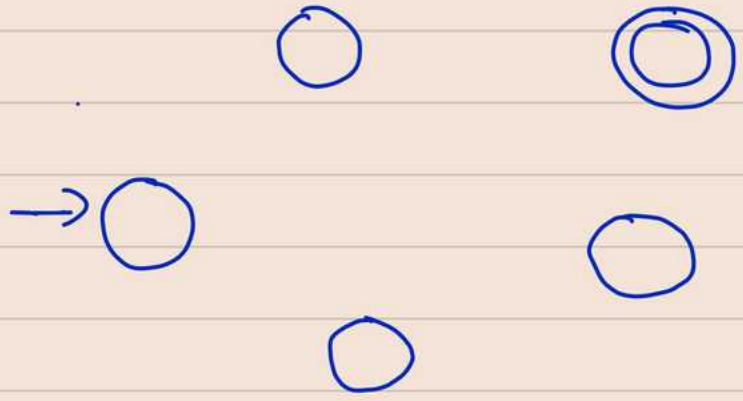
[Hint: use union of individual solutions]

Exercise:

Find the R-E for the following FA



Intution of Pumping Lemma :



FA (having 5 states and many transitions (Not-shown))

Q1: How can we generate a long string?

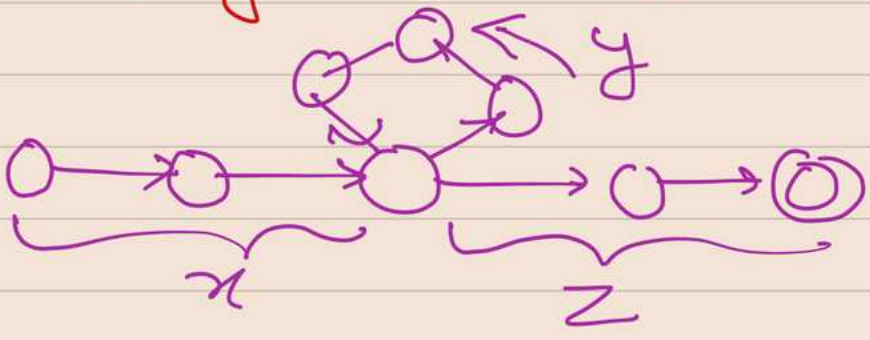
Ans: Take a cyclic path

Q2: How long can the string be without containing a cycle in the figure above

Ans: 4

Note: ① if you can go around the cycle once, then you can go around it multiple times, and that string must also be in the same language.

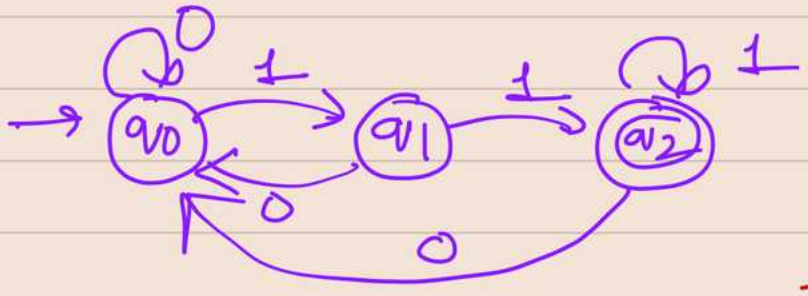
② You can also skip the cycle and that string must also be in the same language



Remember !!!

"...If a language is **Regular** then every **long string** in the language will have a **section** that can be **repeated or pumped** any **number of times** and **still be in the language**.." "

Ex $M = \{$ all strings that end in 11 $\}$



All strings in the language can be repeated (pumped). If they are at least as long as a certain length.

↳ pumping length

Say $p = 3$ in this example



However in 01111 ↳ This can not be pumped because it's length is ≥ 3

Pumping lemma for regular languages

If L is a regular language, then there exists a constant p such that for every string $w \in L$, such that $|w| \geq p$, there exists a division of w in strings x, y, and z, such that $w = xyz$ $\because |y| > 0$, $|xy| \leq p$ and for all $i \geq 0$ we have that $xy^iz \in L$.

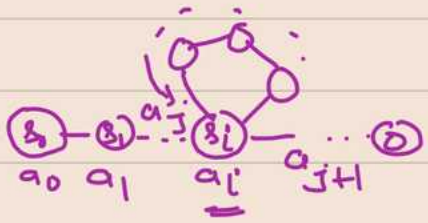
Proof:

Let M be the DFA accepting L and p be the set of states in M.

Let $w = (a_1 a_2 \ldots a_k) \in L$ be any string of length $\geq p$.

Let $s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \cdots \xrightarrow{a_k} s_k$ be the run of w on M

Let i be the index of first state that the run revisits and let j be the index of second occurrence of that state, i.e, $s_i = s_j$



Let $x = a_1 a_2 \ldots a_i$
$y = a_{i+1} \ldots a_j$
and $z = a_{j+1} \ldots a_k$

Notice that $|y| > 0$ and $|xy| \leq p$
also notice that $\forall i \geq 0$, $xy^iz \in L$

$a_1 a_2 \ldots a_i \; \underline{a_{i+1} \ldots a_j} \; \underline{a_{i+1} \ldots a_j} \; a_{j+1} \ldots a_k$

Summary of the important results of the pumping lemma:

this can also be used.

Theorem: (Pumping lemma for regular languages)

$L \in \Sigma^*$ is a regular language

$\Rightarrow$

There exists $p \geq 1$ such that

for all strings $w \in L$ with $|w| \geq p$ we have that

there exists $x, y, z \in \Sigma^*$ with $w = xyz$, $|y| > 0$, $|xy| \leq p$ such that

for all $i \geq 0$, we have that

$xy^iz \in L$

Now Contrapositive of Pumping lemma is Pumping Lemma (contrapositive)

for all $p \geq 1$ we have that

There exists a string $w \in L$ with $|w| \geq p$ such that

for all $x, y, z \in \Sigma^*$ with $w = xyz$, $|y| > 0$, $|xy| \leq p$

we have that

there exists $i \geq 0$ such that

$xy^iz \notin L$ ← Note

$\Rightarrow$

$L \in \Sigma^*$ is not a regular language

A SMALL NOTE !!

$X \to Y$

"If X then Y" is equivalent to

(A) "If Y then X"

$Y \to X$ (Converse)

NO

(B) "If not X then not Y"

$\neg X \to \neg Y$ (inverse)

NO

(C) "If not Y then not X"

$\neg Y \to \neg X$

(contrapositive)

YES

A detailed clarification of contrapositive of pumping lemma (optional to study):

L is regular $\Rightarrow$

$$\exists p \forall w \in L \Big(|w| \geq p \Rightarrow \exists x\, y\, z \big(w = xyz \wedge y \neq \in \wedge |xy| \leq p \wedge (\forall i \;\; xy^i z \in L) \big) \Big)$$

contrapositive

(Note: $A \Rightarrow B$, contrapositive $\neg B \Rightarrow \neg A$)

further $C \Rightarrow D = \neg C \vee D$

$\therefore \overline{C \Rightarrow D} = C \wedge \neg D$

Also Note:

$p \wedge q \wedge r \wedge s$

Take negation $\neg p \vee \neg q \vee \neg r \vee \neg s$

which is same as $p \wedge q \wedge r \Rightarrow \neg s$

Note

$$\forall p \exists w \in L \Big(|w| \geq p \wedge \forall xyz \big[(w = xyz \wedge y \neq \in \wedge |xy| \leq p) \Leftrightarrow \exists i \; xy^i z \notin L \big] \Big)$$

Steps to apply (How to show that a language is non-regular; use contrapositive of pumping lemma):

1. Let p be an arbitrary number (pumping length)
2. (cleverly) find a representative string w of L of size $\geq p$
3. Try out all ways to break the string into xyz triplet satisfying that $|y| > 0$ and $|xy| \leq n$. If the step 3 was clever enough, there will be finitely many cases to consider.
4. For every triplet show that for some i the string xy^iz is not in L, and hence it yields contradiction with pumping lemma.

Note: Pumping lemma is necessary but not sufficient condition for regularity

Ex prove that the language $L = \{0^n 1^n\}$ is not regular

Soln

(1) let p be an arbitrary number (pumping length)

(2) consider $w = 0^p 1^p \in L$ and $|0^p 1^p| \geq p$

(3) only way to break this string in xyz triplets such that $|xy| \leq p$ and $y \neq \epsilon$ is to choose $y = 0^k$ for some $1 \leq k \leq p$.

(4) for each such triplet, there exists an i (say $i = 2$) such that $xy^i z \notin L$

(5) Hence L is non-regular

Note: In classroom, we have also considered other possibilities, like y contains only

(1) '0's, or only '1's (2) or 0's 1's (3) all lead to contradiction. However, in light of the constraint $|xy| \leq p$, y can only contain '0's', which will lead to disbalanced strings if pumped more than once.

Ex:

Prove that the language

$$M = \{ 1^{n^2} : n \geq 0 \}$$

is not regular

Soln:

(1) Let p be the pumping length

(2) Let $w = 1^{p^2}$. Notice that $|1^{p^2}| \geq p$

(3) Only way to break this string in xyz triplet such that $|xy| \leq p$ and $y \neq \in$ is to choose $y = 1^k$ for some $1 \leq k \leq p$

(4) Now consider

$$1^l 1^k 1^k 1^{p^2 - l - k} \quad \text{(pumping twice)}$$

$l + k \leq p$,

pumping twice ($i = 2$)

$w = 1^{p^2 + k}$ since $0 < k \leq p$

because difference in between p^2 and the next square is always greater than p of natural numbers.

Ex:

Prove that the language

$L = \{ 0^i 1^j : i > j \}$ is NOT regular

Soln:

(1) Let p = pumping length.

(2) Let $w = 0^{p+1} 1^p$. Notice $|0^{p+1} 1^p| \geq p$

(3) Only way to break this string in xyz triplet such that $|xy| \leq p$ and $|y| \neq \in$ is to choose $y = 0^k$ for some $1 \leq k \leq p$.

(4) Choose $i = 0$ (since all pumping-ups are in the language so go for pump-down)

(5) $xy^i z \notin L$ for $i = 0$

(6) Hence L is not regular.

use closure properties to prove non-regularity :

Ex: prove that language

$$EQUAL = \{ w \mid w \in \{a,b\}^* \therefore n_a(w) = n_b(w) \}$$

($n_a(w)$: No. of a's, $n_b(w)$: No. of b's)

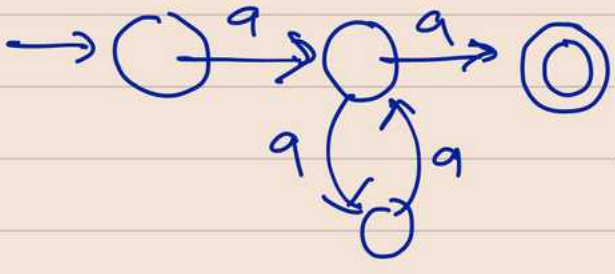
Soln:

$$EQUAL \cap a^* b^* = a^n b^n$$

- $EQUAL$: since intersection is NOT regular $\Rightarrow$ EQUAL can not be regular
- $a^* b^*$: Regular
- $a^n b^n$: NOT-regular

Ex: Is $L = \{ a^{2n} \mid n \geq 1 \}$ regular

$$a^{2n} = a (a^2)^n a$$



← FA can be constructed, hence Regular

Note: following are always regular

1. A finite language
2. A union of a finite number of regular languages
3. A union of a countable number of regular languages
4. A co-finite language (a set is co-finite if it's complement is finite)
5. The reversal of a regular language

Note:
1. Finite → Regular → Reg
2. Infinite → ordering or Counting → NOT regular
   - ↳ May or may not be regular.

Ex:
1. $L=\{a^m b^n \mid m,n \geq 0\}$ ← Regular
2. $L=\{a^n b^n \mid n \geq 0\}$ ← Non-regular
3. $L=\{a^n b^n \mid 1 \leq n \leq 10\}$ ← Regular
4. $L=\{a^n b^n \mid 1 \leq n \leq 2^{|ip|}\}$ ← Regular
5. $L=\{a^m b^n \mid m > n, m \geq 0\}$ ← NOT-regular
6. $L=\{a^m b^n \mid m,n \geq 0, m \neq n\}$ ← NOT-regular
7. $L=\{w c w^R \mid w \in \Sigma^*\}$ ← NOT-regular
8. $L=\{w c w \mid c, w \in \Sigma^*\}$ ← Regular

① Two strings are related (or x and y are L-equivalent; written as $x \sim_L y$) OR xRy

$$R \subseteq \Sigma^* \times \Sigma^*$$

x ↑ ↖ y

Note: This relation is an "equivalence relation"

xRy if $\forall z \in \Sigma^*$

$$xz \in L \iff yz \in L$$

(Neither is in L or both are in L)

② [Concept of Separating extension]

x and y have a separating (distinguishing) extension $z \in \Sigma^*$ if $xz \in L \iff yz \notin L$ (exactly one of xz or yz is in L).

We say x and y are L-separable.

Example of Separating extension:

$$B = \{0^n 1^n \mid n \in \mathbb{N}\} = \{\epsilon, 01, 0011, 000111, \cdots\}$$

let $x = 00$ $y = 000$ $x, y \notin B$

do they have a separating extension?

Let $z = 11$

$\therefore xz = 0011 \in B$, $yz = 00011 \notin B$ so z is a separating extension so $x \not\sim_B y$.

Let $x = 010$ and $y = 00100$

Note: $x, y \notin B$, no extension of either is in B

(2)

L is regular if and only if

$\forall x, y \in \Sigma^*, \forall z \in \Sigma^*$
$xz \in L \iff yz \in L$

If this holds then xRy → equivalence relation

→ Number of equivalence classes defined by R are finite.

Applicability: If there are finitely many equivalence classes ⇒ L is regular (To prove L is regular)

No. of equivalence classes are infinite ⇒ L is Not regular (To prove L is NOT regular)

Myhill-Nerode Theorem: A language L is regular if and only if Relation $\sim_L$ defines a finite number of equivalence classes.

Note: The number of equivalence classes of R equals to the number of states in the smallest DFA that decides L.

Ex: ①

Prove $L = \{ 0^n 1^n \mid n \in \mathbb{N} \}$ is not regular.

By the Myhill-Nerode theorem, it suffices to find an [infinite set S of pairwise L-separable strings.] ← Note:

$S = \{ 0^n \mid n \in \mathbb{N} \} = \{ \epsilon, 0, 00, 000, \cdots \}$

Each pair $x = 0^i$ $y = 0^k \in S$ with $i < k$ has a separating extension 1^i since $0^i 1^i \in L$ but $0^k 1^i \notin L$.

Ex: ②

Prove $L = \{ w \in \{0,1\}^* \mid \#w_0 = \#w_1 \}$ is not regular.

Hint: Above proof can work.

Ex: ③

Prove $L = \{ ww \mid w \in \{0,1\}^* \}$ is not regular.

By Myhill-Nerode, it suffices to show an infinite set S of pairwise L-separable strings

Let $S = \{ 0^n 1^n \mid n \in \mathbb{N} \} = \{ \epsilon, 01, 0011, 000111, \cdots \}$

~~For~~ each pair $x = 0^i 1^i$, $y = 0^k 1^k$ with $i < k$

has separating extension, $z = 0^i 1^i$

$\Rightarrow$ $xz \in L$ but $yz \notin L$

DFA Minimization

- Any DFA defines one language, For a particular language there are many DFA's that accepts it.
- → For simplicity, a DFA with the fewest number of states may be preferred.
- → A DFA is minimal if it satisfies two properties:

(a) Every state is reachable:

$$\forall q \in Q, \exists w \in \Sigma^* \ni \delta^*(q_0, w) = q$$

(b) Every pair of states is distinguishable:

$$\forall q, r \in Q \ni q \neq r$$

implies there exists $w \in \Sigma^*$, such that $\delta^*(q, w) \in F$ iff $\delta^*(r, w) \notin F$

Definitions:

(1) A state $p \in Q$ of a DFA $(Q, \Sigma, \delta, q_0, F)$ is said to be accessible or reachable if for some $w \in \Sigma^*$ it's the case that $\delta^*(q_0, w) = p$

(2) A state $p \in Q$ of a DFA $(Q, \Sigma, \delta, q_0, F)$ is said to be inaccessible if for all $w \in \Sigma^*$ it's the case that $\delta^*(q_0, w) \neq p$

(3) The states p and q of a DFA $(Q, \Sigma, \delta, q_0, F)$ are said to be indistinguishable/equivalent written as $p \approx q$ if for all $x \in \Sigma^*$.

$\delta^*(p,x) \in F \Leftrightarrow \delta^*(q,x) \in F$

$\delta^*(p,x) \in NF \Leftrightarrow \delta^*(q,x) \in NF$

→ both final or both Non-final for all $x \in \Sigma^*$

Note: As it's difficult to construct $\delta^*(p,x)$ and $\delta^*(q,x)$ $\forall x \in \Sigma^*$, because there are infinite string in Σ^*, we consider k-equivalent

(4) Two states p and q are k-equivalent ($k \geq 0$) if both $\delta^*(p,x)$ and $\delta^*(q,x)$ are final states or both non-final for all strings x of length k or less.

Note: if two states are not equivalent then they are distinguishable.

(4) The states p and q of a DFA $(Q, \Sigma, \delta, q_0, F$ are said to be distinguishable, written $p \not\approx q$, if for some $x \in \Sigma^*$, one of these equivalent statements hold

$\delta^*(p,x) \in F \Leftrightarrow \delta^*(q,x) \notin F$

$\delta^*(p,x) \notin F \Leftrightarrow \delta^*(q,x) \in F$

Note: (1) if p and q are K-equivalent for all $K \geq 0$, then they are equivalent

(2) if p and q are $(K+1)$-equivalent, then they are K-equivalent.

Procedure to construct Minimized DFA:

Step 1: Construct π_0

By definition of 0-equivalence,

$\pi_0 = \{ Q_1^0, Q_2^0 \}$

Q_1^0 → set of all final states

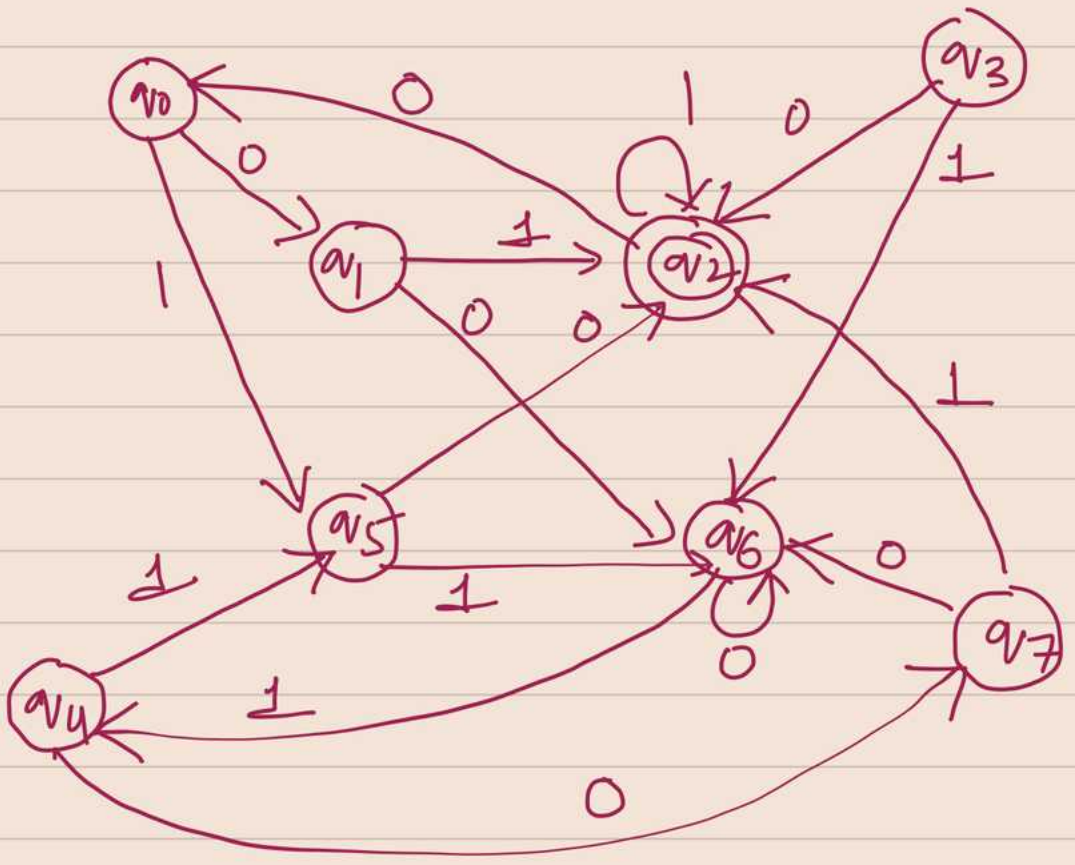
Q_2^0 → set of all non-final states

Step 2: Construct π_{K+1} from π_K

Step 3: Construct π_n for $n = 1, 2, \ldots$ until $\pi_n = \pi_{n+1}$

Step 4: construct minimum DFA by combining equivalent states.

Ex:



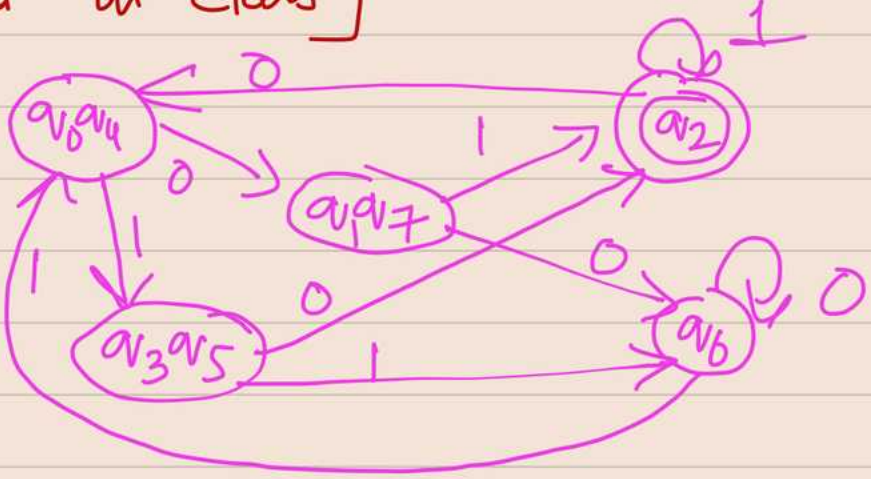
$\pi_0 = \{\{q_2\}\{q_0, q_1, q_3, q_4, q_5, q_6, q_7\}\}$

$\pi_1 = \{\{q_2\}\{q_0, q_4, q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\}$

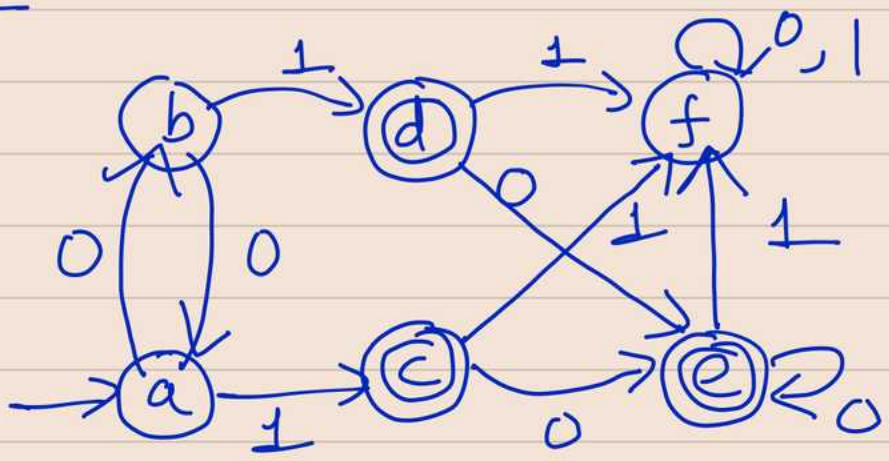
$\pi_2 = \{\{q_2\}, \{q_0, q_4\}, \{q_1, q_7\}, \{q_3, q_5\}\{q_6\}\}$

$\pi_3 = \pi_2$

[Detailed solution to be discussed on "Board" in class]



Ex



Minimized DFA ↴



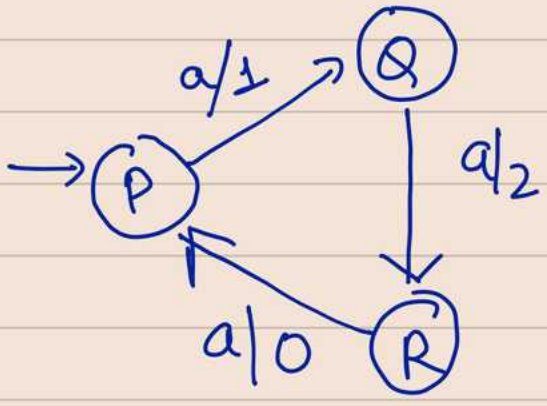
Finite Automata with outputs

Mealy Machine:

It's a six-tuple $(Q, \Sigma, \delta, q_0, \Delta, \lambda)$

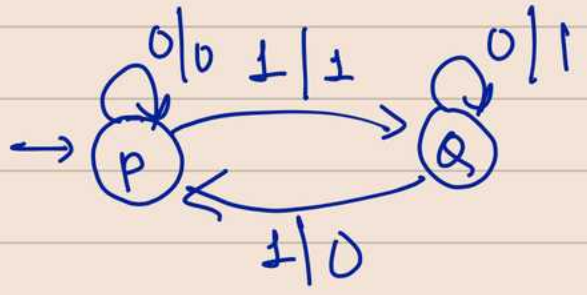
where Q = finite set of states
Σ = alphabet (input)
Δ = output alphabet
δ = transition function
$\delta: Q \times \Sigma \rightarrow Q$
λ = output function
$\lambda: Q \times \Sigma \rightarrow \Delta$
q_0 = initial state

Example : Design a Mealy m/c, whose output tells the number of input symbols modulo 3



Example : Design a Mealy m/c that produces an output 1, whenever the parity of input sequence is odd, and an output 0, whenever the parity of input sequence is even

No. of 1's



Example : Design a Mealy m/c where the output sequence is to be a replica (copy) of the input sequence delayed by two time units

$$z(t) = s(t-2) \quad t > 2$$

$s(t)$

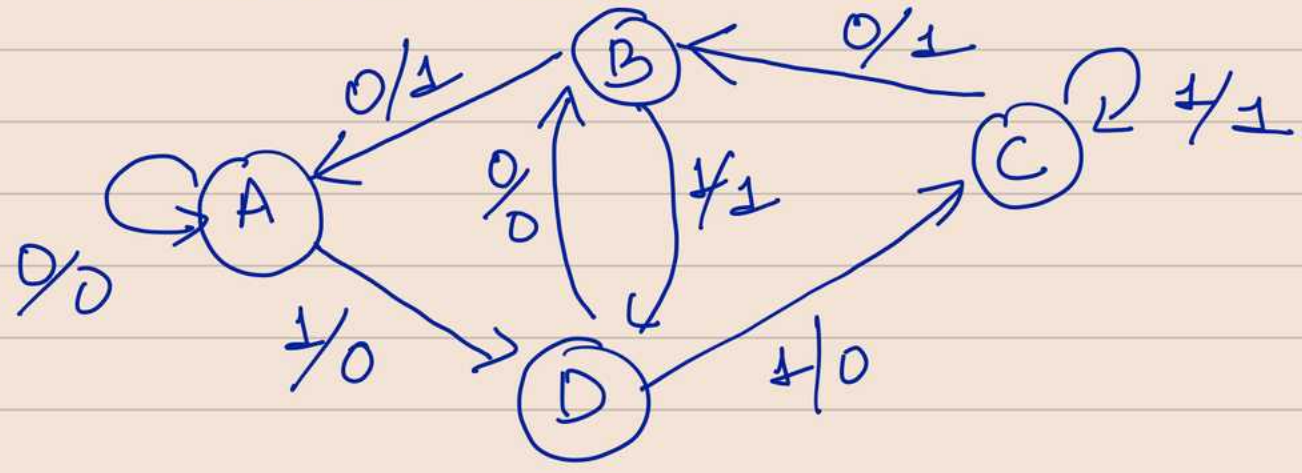
input is 0 0 0 1 1 0 1 0 0

output is 0 0 0 0 0 1 1 0 1

$z(t)$

$z(1)$ and $z(2)$ are don't care

$s(t-1)$	$s(t-2)$	
0	0	→ state A
0	1	→ state B
1	1	→ state C
1	0	→ state D



Note: an n-unit delay would require 2^n states.

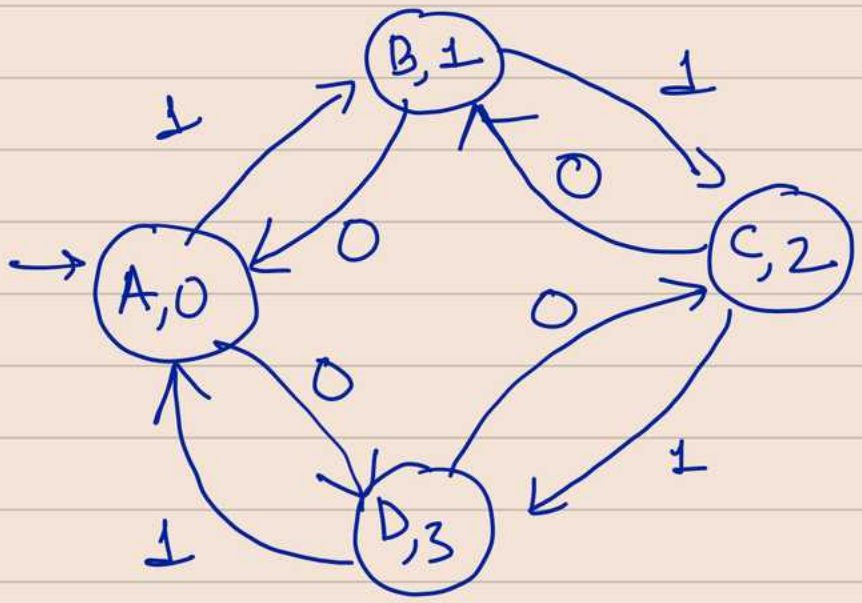
Moore machine:

It's a six-tuple $(Q, \Sigma, \delta, q_0, \Delta, \lambda)$

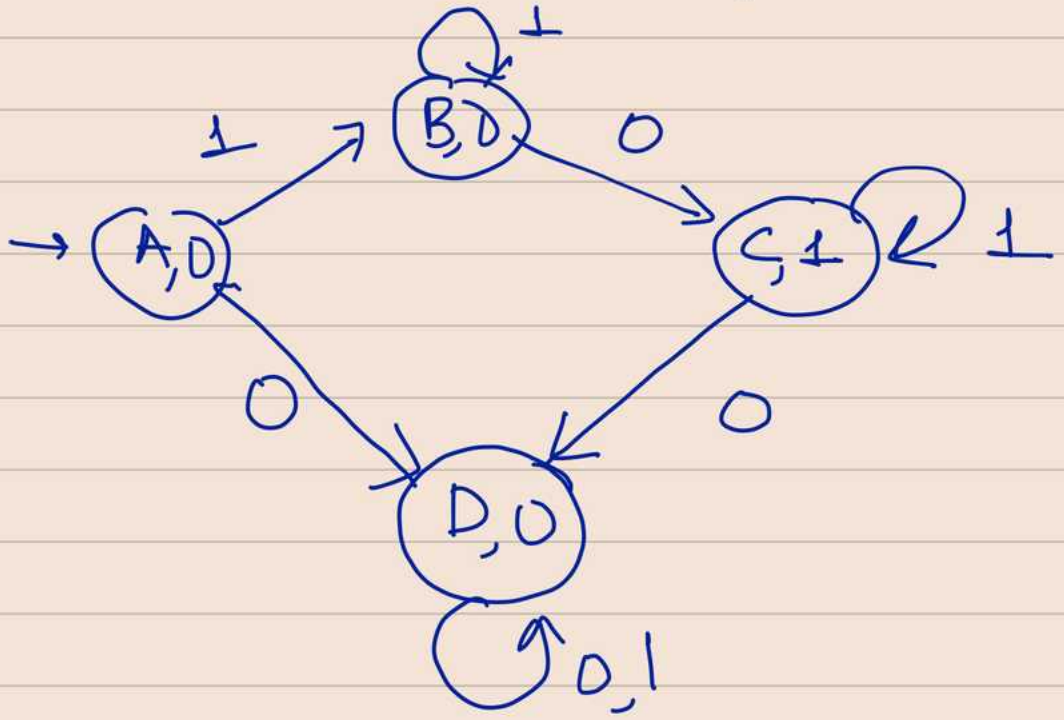
where Q = finite set of states
Σ = alphabet (input)
Δ = output alphabet
δ = transition function
$\delta: Q \times \Sigma \rightarrow Q$
λ = output function
$\lambda: Q \rightarrow \Delta$
q_0 = initial state.

Example: Design a Moore m/c that acts as a Modulo 4 up-down Counter that

$$z(t) = [N_1(w) - N_0(w)] \bmod 4$$

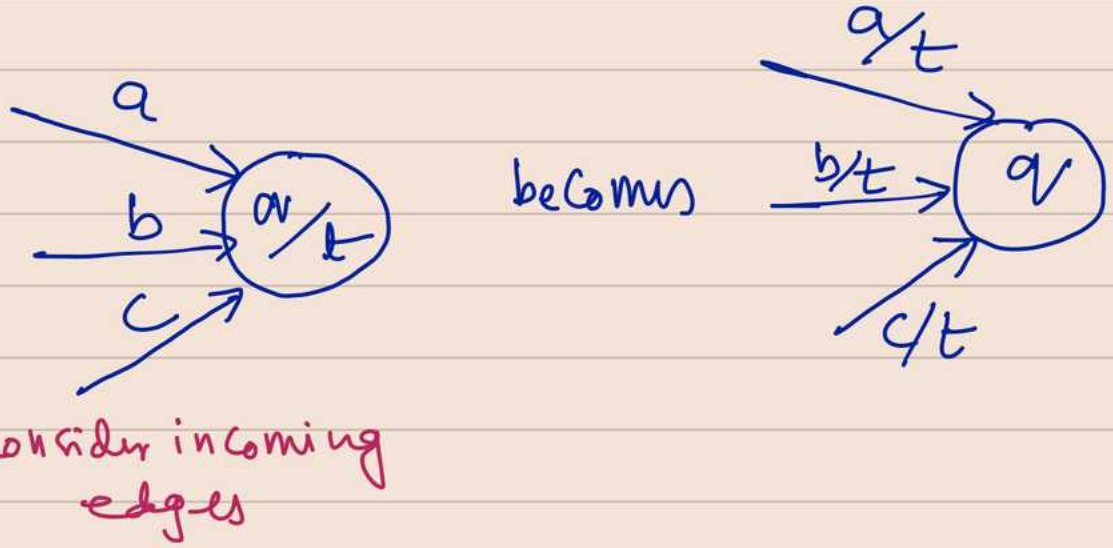


Example: Design a Moore m/c that acts as a language recognizer and accept a binary string iff it begins with a 1 and contains exactly one 0 $(1 1^* 0 1^*)$

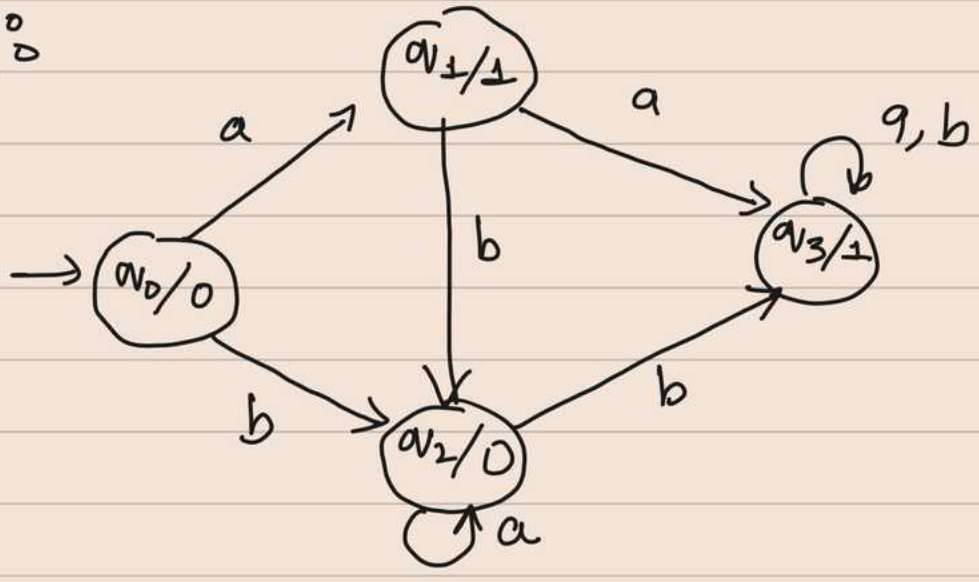


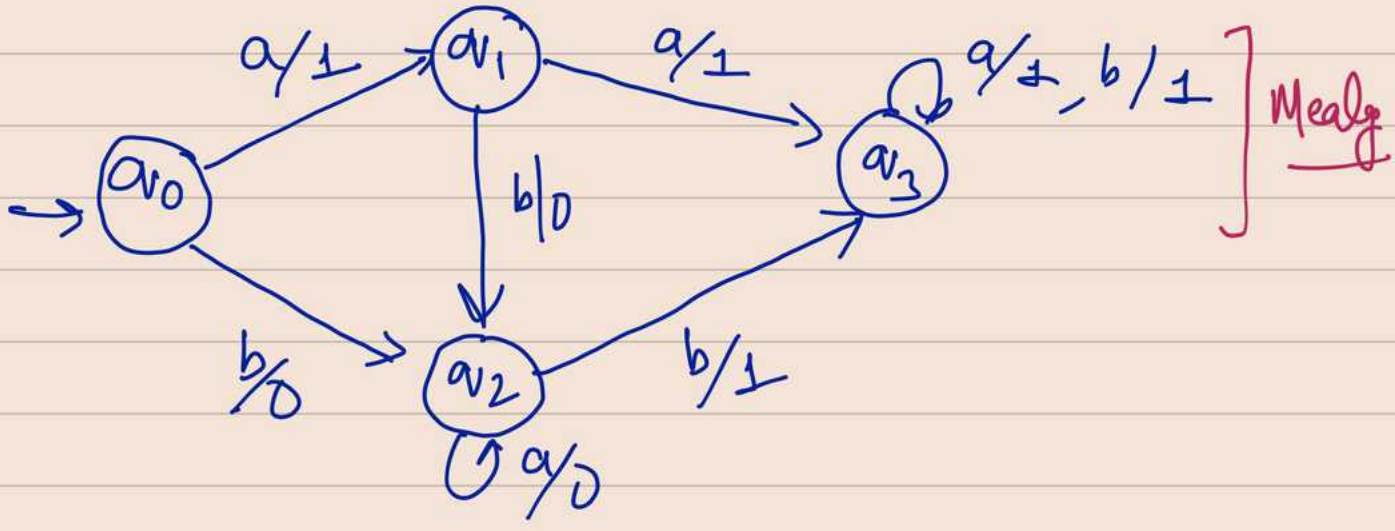
Moore = Mealy :

if M_0 is a Moore m/c, then there is a Mealy m/c M_e that is equivalent to it



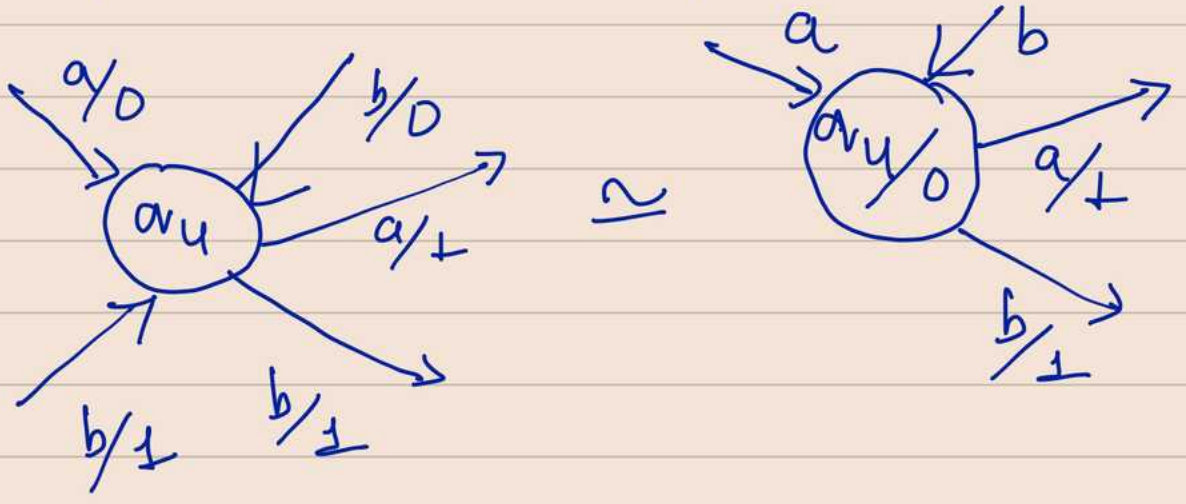
Example :



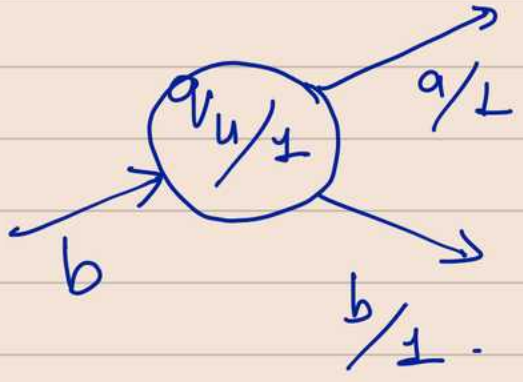


Mealy = Moore

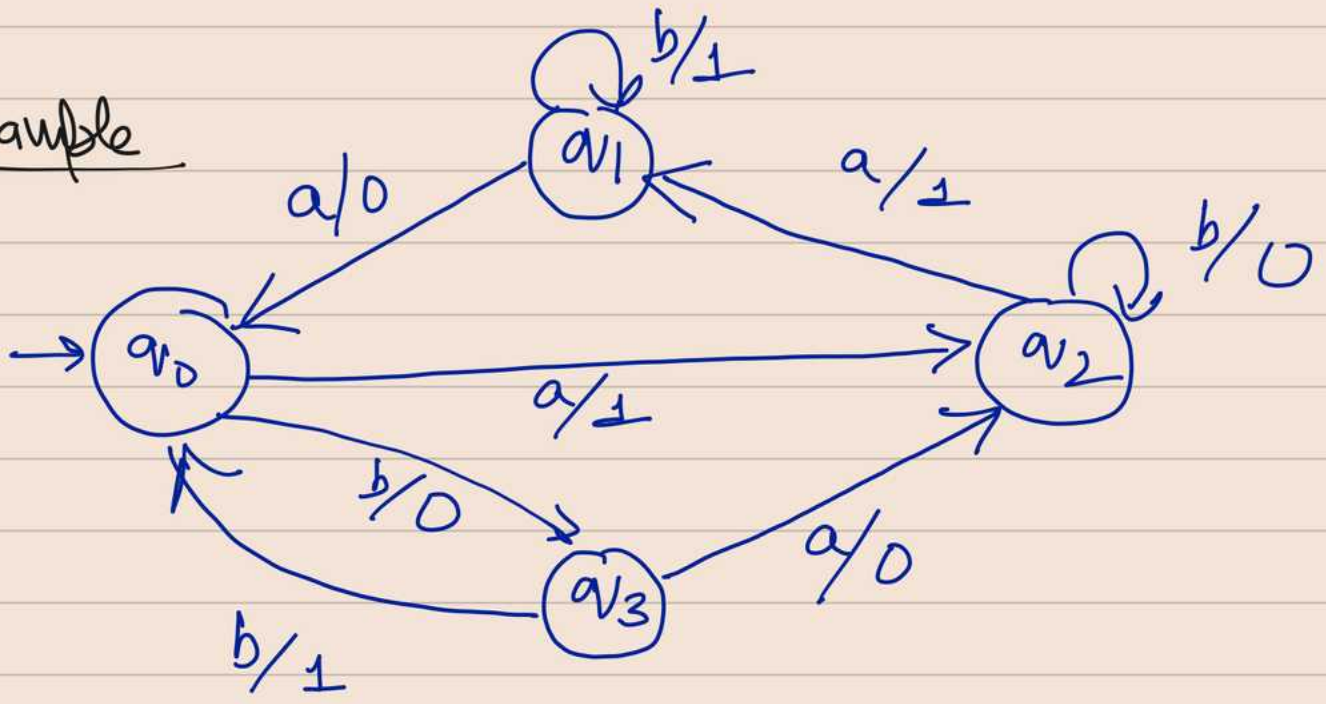
For every Mealy m/c M_e, there is an Moore m/c M_o that is equivalent to it



Two distinct o/p's on incoming edges towards state q_4 so have two copies of q_4 and copy outgoing edges.

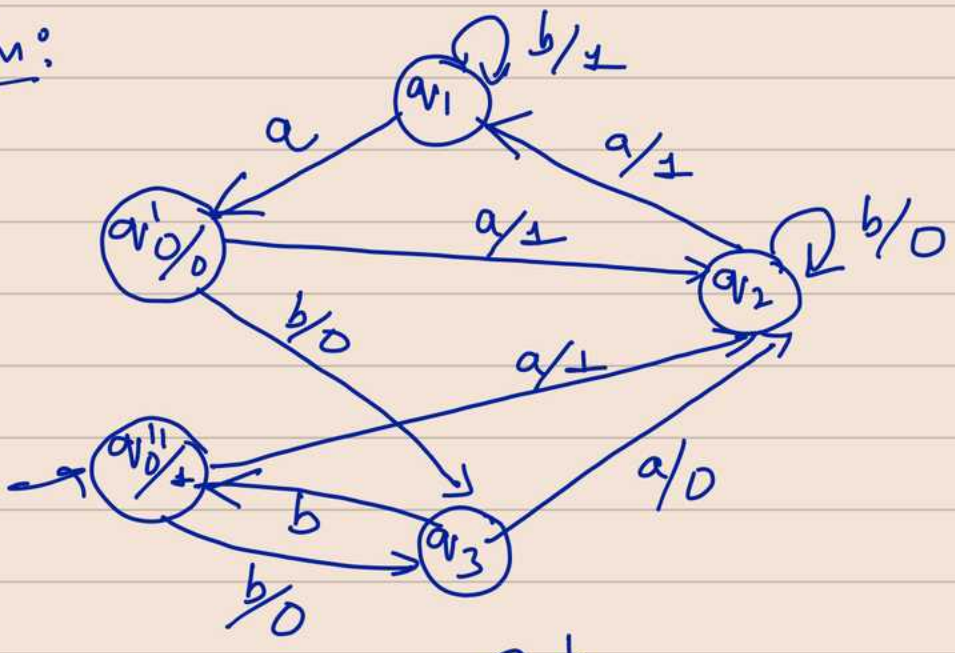


Example

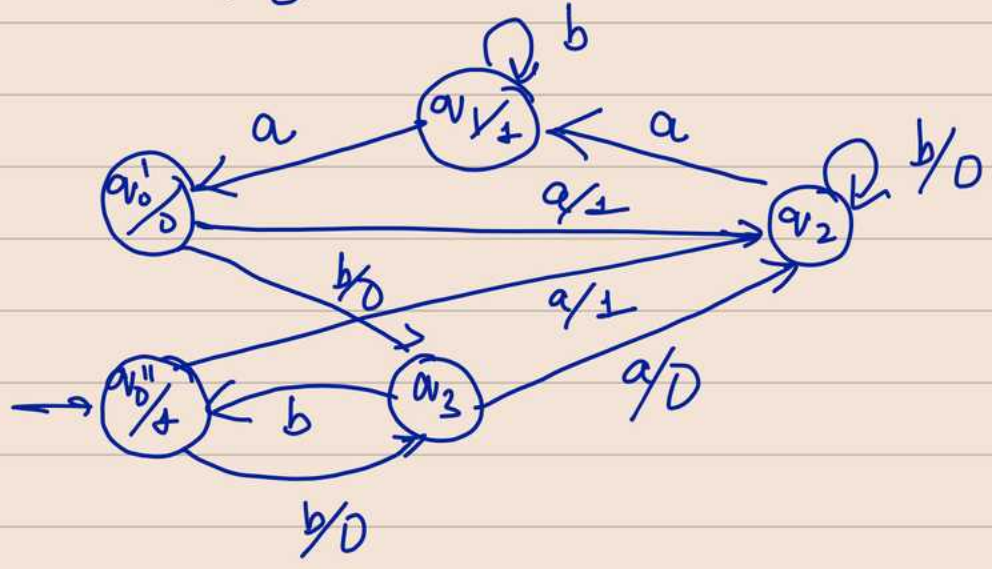


Soln:

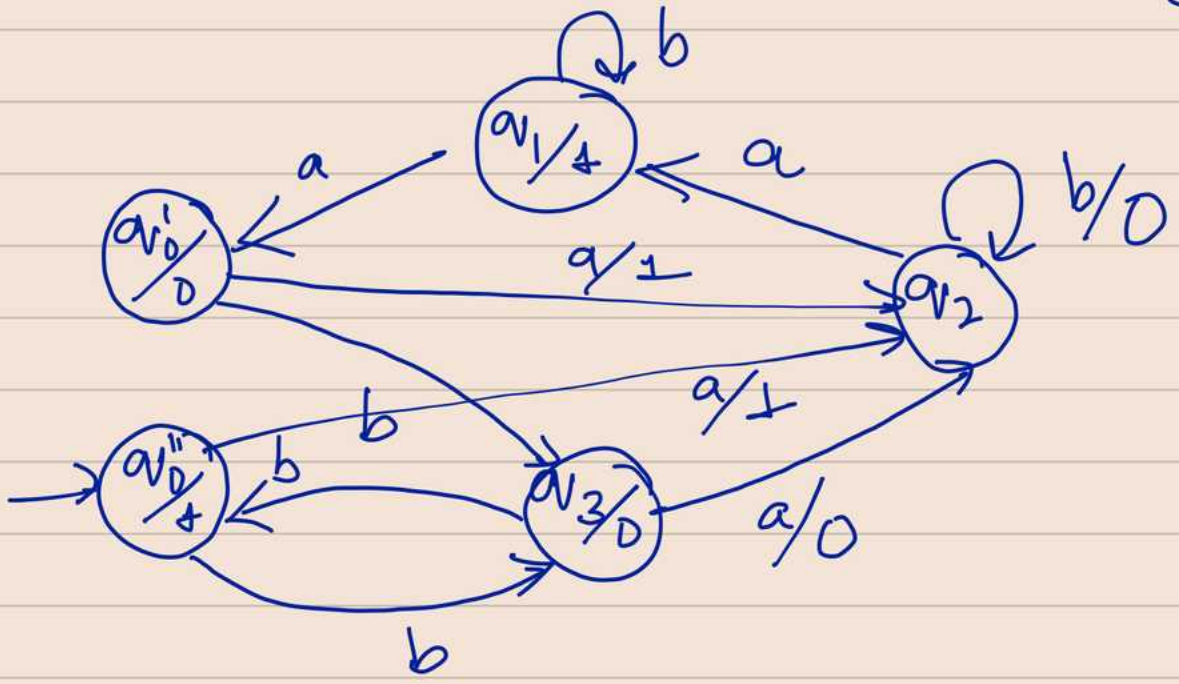
Consider q_0

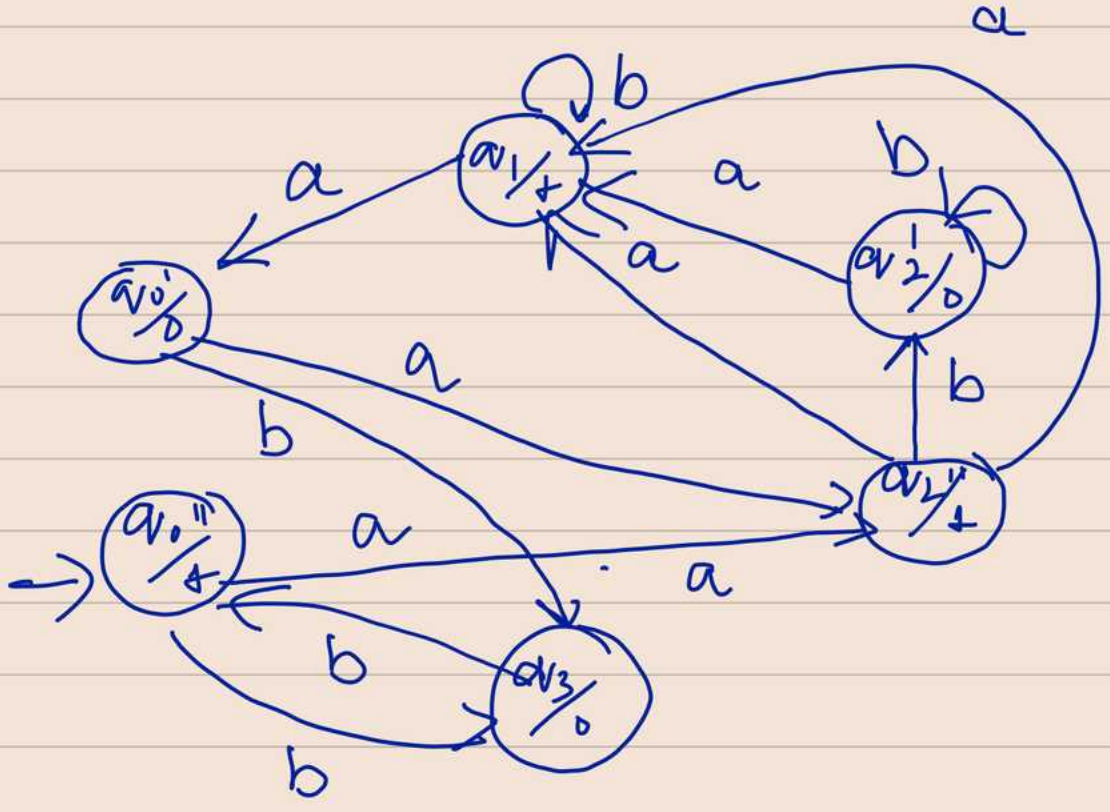


Consider q_1



Consider q_3





Consider q_2

Closure properties :

(I) Set theoretic properties

Thm : The class of regular languages is closed with respect to complement

Proof :

Let L_1 be a regular language accepted by the DFA, $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$. construct the DFA $M' = (Q, \Sigma, \delta, q_0, Q - F_1)$ That is interchanging the roles of final and nonfinal states of M_1

We claim that $L(M') = L_1^c$ so that L_1^c is also regular.

(complement of L)

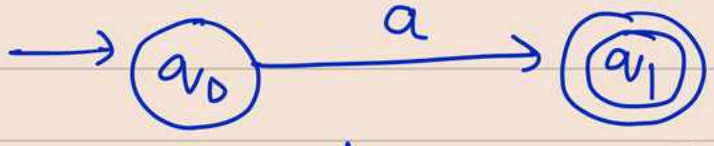
For $x \in \Sigma^*$

(extended transition function)

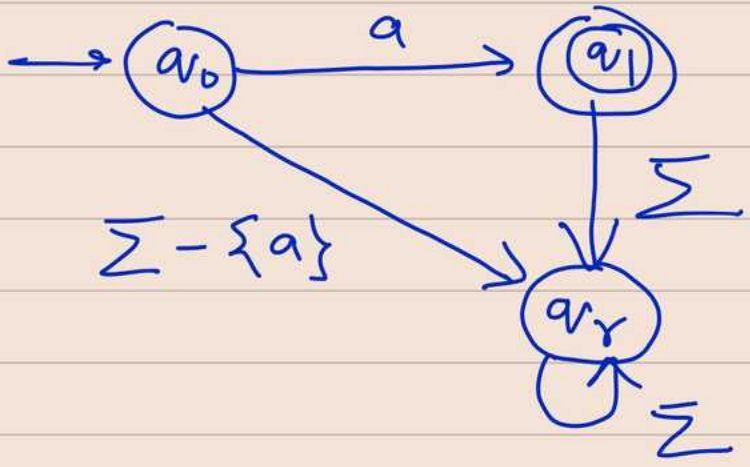
$$x \in L_1^c \iff x \notin L_1$$
$$\iff \hat{\delta}(q_0, x) \notin F_1$$
$$\iff \hat{\delta}(q_0, x) \in Q - F_1$$
$$\iff x \in L(M')$$

Note: It's important that the transition function is a total function; that is it's fully defined.

A partial transition function does not guarantee that this construct will accept Σ^*. like.



However, if we fully define an equivalent machine, then the construction in the proof guarantees the complement machine accepts $\Sigma^* \setminus \{a\}$



} Importance of dead state

Thm: Regular languages are closed under Union.

Note

Proof: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be the FA for languages L_1 and L_2 respectively.

Now construct $M = (Q, \Sigma, \delta, q_0, F)$ to accept $L_1 \cup L_2$, where;

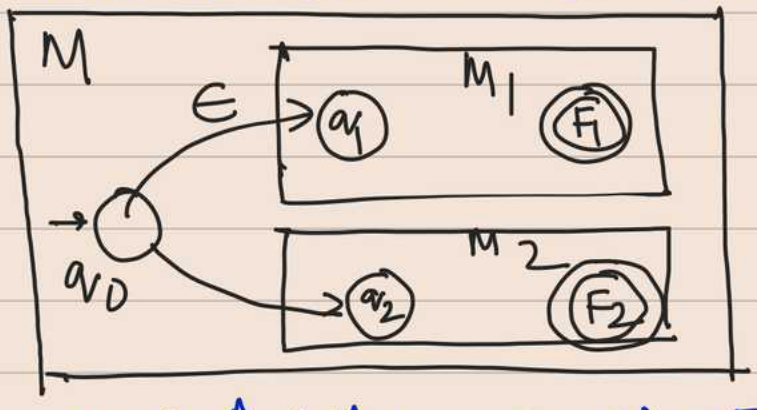
Note

$Q = Q_1 \cup Q_2 \cup \{q_0\}$ with a new state q_0

$F = F_1 \cup F_2$

$\delta : Q \times (\Sigma \cup \{\in\}) \longrightarrow P(Q)$ defined by

$$\delta(q,a) = \begin{cases} \delta_1(q,a) \text{ if } q \in Q_1, a \in \Sigma \cup \{\in\} \\ \delta_2(q,a) \text{ if } q \in Q_2, a \in \Sigma \cup \{\in\} \\ \{q_1, q_2\} \text{ if } q = q_0, a = \in \end{cases}$$



for $x \in \Sigma^*$

$x \in L(M)$

$\Leftrightarrow (\hat{\delta}(q_0, x) \cap F) \neq \phi$

$\Leftrightarrow (\hat{\delta}(q_0, \in x) \cap F) \neq \phi$

$\Leftrightarrow (\hat{\delta}(\hat{\delta}(q_0, \in), x)) \cap F) \neq \phi$

$\Leftrightarrow (\hat{\delta}(\{q_1, q_2\}, x) \cap F) \neq \phi$

$\Leftrightarrow ((\hat{\delta}(q_1, x) \cup \hat{\delta}(q_2, x)) \cap F) \neq \phi$

$\Leftrightarrow ((\hat{\delta}(q_1, x) \cap F) \cup (\hat{\delta}(q_2, x) \cap F)) \neq \phi$

First $M \rightarrow$ closure(x) then closure(x) $\rightarrow M$

Proof skeleton

$\Leftrightarrow x \in L(M_1)$ or $x \in L(M_2)$

$\Leftrightarrow x \in L_1 \cup L_2$.

Thm: Regular languages are closed under concatenation.

Proof:

Note

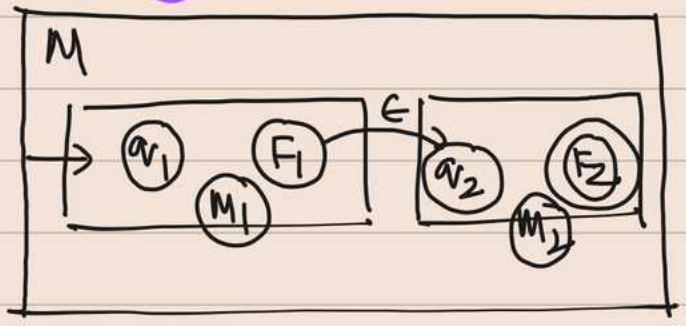
Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be FA accepting L_1 and L_2, respectively.

Note

Construct $M = (Q, \Sigma, \delta, q_0, F)$ to accept L_1^* where

$Q = Q_1 \cup Q_2$

$$\delta(q,a) = \begin{cases} \delta_1(q,a), \text{ if } q \in Q_1, a \in \Sigma \cup \{\epsilon\} \\ \delta_2(q,a), \text{ if } q \in Q_2, a \in \Sigma \cup \{\epsilon\} \\ \{q_2\}, \text{ if } q \in F_1, a = \epsilon \end{cases}$$



We claim that $L(M) = L(M_1)L(M_2)$

suppose

$x = a_1 a_2 \cdots a_n \in L(M)$

$\Rightarrow \hat{\delta}(q_1, x) \cap F_2 \neq \phi$

Note: It's clear from the construction of M that the only way to reach from q_1 to any state of F_2 is via q_2. Further we have only $\in$-transitions from the states of F_1 to q_2.

$\Rightarrow$ While traversing through x from q_1 to some state of F_2, $\exists\ p \in F_1$ and and a number $k \le n$ such that

$$p \in \hat{\delta}(q_1, q_1 q_2, \ldots, q_k) \text{ and } \hat{\delta}(q_2, q_{k+1} q_{k+2} \ldots q_n) \cap F_2 \neq \phi$$

Then

$$x_1 = q_1 q_2 \ldots q_k \in L_1$$

and

$$x_2 = q_{k+1} q_{k+2} \ldots q_n \in L_2$$

So that $x_1 x_2 \in L_1 L_2$

Conversely, suppose $x \in L_1 L_2$ ← Note

Then $x = x_1 x_2$ for some $x_1 \in L_1$

and some $x_2 \in L_2$

$$\Rightarrow \hat{\delta}(q_1, x_1) \cap F_1 \neq \phi \text{ and } \hat{\delta}_2(q_2, x_2) \cap F_2 \neq \phi$$

Now,

$$\begin{aligned}(q_1, x) &= (q_1, x_1 x_2) \\ &\vdash^{*} (p, x_2) \text{ for some } p \in F_1 \\ &= (p, \in x_2) \\ &\vdash (q_2, x_2) \quad \because \delta(p, \in) = \{q_2\} \\ &\vdash^{*} (p', \in) \text{ for some } p' \in F_2\end{aligned}$$

Hence, $\hat{\delta}(q_1, x) \cap F_2 \neq \phi$ so that $x \in L$

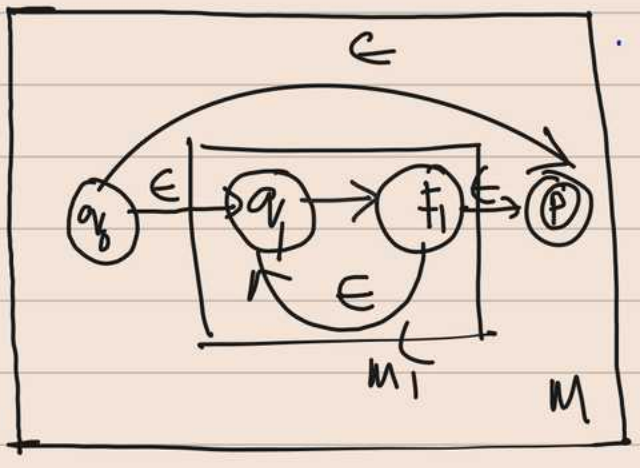
Thm: Regular languages are closed under Kleene's closure.

Proof: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ be a FA accepting L_1. ← Note

Now construct $M = (Q, \Sigma, \delta, q_0, F)$ accepting L_1^* such that] Note

$Q = Q_1 \cup \{q_0, p\}$ with new states q_0 and p, $F = \{p\}$ and define δ by

$$\delta(q,a) = \begin{cases} \{q_1, p\} & \text{if } q \in F_1 \cup \{q_0\} \text{ and } a = \epsilon \\ \delta_1(q,a) & \text{if } q \in Q_1, \text{ and } a \in \Sigma \cup \{\epsilon\} \end{cases}$$



We prove that $L = L_1^*$ for some $x \in \Sigma^*$

$x \in L$

$\Rightarrow \hat{\delta}(q_0, x) = \{p\}$

$\Rightarrow$ either $x = \epsilon$ or $\hat{\delta}(q_0, x) \cap F_1 \neq \phi$

if $x = \epsilon$ then trivially $x \in L_1^*$,

otherwise, $\exists\ p_1, p_2, \ldots, p_k \in F_1$ and $x_1, x_2, \ldots, x_k$ such that

$p_1 \in \delta^*(q_1, x_1), p_2 \in \delta(q_1, x_2), \ldots, p_k \in$ $\quad , x_k)$

with $x = x_1 x_2 \ldots x_k$

Thus $\forall\ 1 \le i \le k$, $x_i \in L_1$ so that $x \in L_1^*$.

Note

Conversely, suppose $x \in L_1^*$. then $x = x_1 x_2, \ldots, x_l$ with $x_i \in L_1$ $\forall i$ and for some $l \ge 0$.

if $l = 0$, $x = \epsilon$, and clearly $x \in L(M)$

Otherwise, we have

$\hat{\delta}(q_1, x_i) \cap F_1 \neq \phi$ for $1 \le i \le l$

Now, consider the following computation of M on x

$$
\begin{aligned}
(q_0, x) &= (q_0, \epsilon x) \\
&\vdash (q_1, x) \\
&= (q_1, x_1 x_2 x_3 \cdots x_l) \\
&\overset{*}{\vdash} (p_1', x_2 x_3 \cdots x_l) \text{ for some } p_1' \in F_1 \\
&= (p_1', \epsilon x_2 x_3 \cdots x_l) \\
&\vdash (q_1, x_2 x_3 \cdots x_l) \text{ since } q_1 \in \delta(p_1', \epsilon) \\
&\vdots \\
&\overset{*}{\vdash} (p_l', \epsilon) \text{ for some } p_l' \in F_1 \\
&\vdash (p, \epsilon) \text{ since } p \in \delta(p_l', \epsilon)
\end{aligned}
$$

As $\hat{\delta}(q_0, x) \cap F \neq \phi$, we have $x \in L_1^*$.

Corollary:

The class of regular languages is closed with respect to intersection.

Proof:

if L_1 and L_2 are regular, then L_1^c and L_2^c are also regular.

Then their union is also regular

$\Rightarrow L_1^c \cup L_2^c$ is regular

$\Rightarrow (L_1^c \cup L_2^c)^c$ is also regular

Use De Morgan.

$\Rightarrow L_1 \cap L_2 = (L_1^c \cup L_2^c)^c$

so that $L_1 \cap L_2$ is also regular.

Alternative Proof: (Proof by construction)

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and M_2

$= (Q_2, \Sigma, \delta_2, q_2, F_2)$ be two DFA.

accepting L_1 and L_2, respectively.

Set the DFA $M = (Q_1 \times Q_2, \Sigma, \delta, (q_1 q_2), F_1 \times F_2)$

where δ is defined point-wise by

$$\delta((p,q),a) = (\delta_1(p,a), \delta_2(q,a))$$

$\forall\ (p,q) \in Q_1 \times Q_2$ and $a \in \Sigma$.

We claim that $L(M) = L_1 \cap L_2$.

Using induction on $|x|$, first observe that

$\hat{\delta}((p,q),x) = (\hat{\delta}_1(p,x), \hat{\delta}_2(q,x))$, for all $x \in \Sigma^*$.

Now it clearly follows that

$$x \in L(M) \iff \hat{\delta}((q_1,q_2),x) \in F_1 \times F_2$$

$$\iff (\hat{\delta}_1(q_1,x), \hat{\delta}_2(q_2,x)) \in F_1 \times F_2$$

$$\iff \hat{\delta}_1(q_1,x) \in F_1 \text{ and } \hat{\delta}_2(q_2,x) \in F_2$$

$$\iff x \in L_1 \text{ and } x \in L_2$$

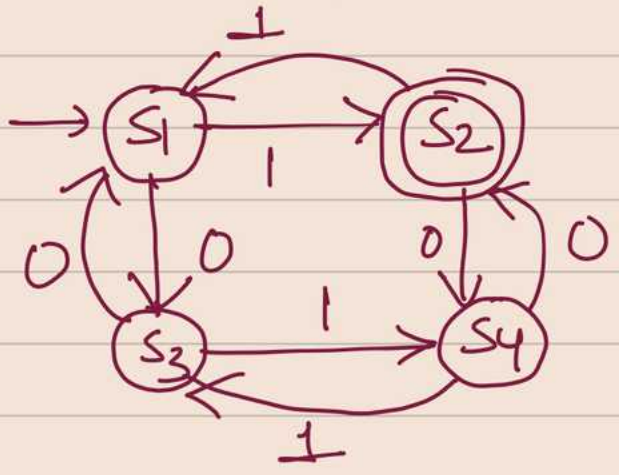
$$\iff x \in L_1 \cap L_2$$

Ex (Try yourself):

Using the construction given in the proof, Design a DFA that accepts the language

$L = \{x \in \{0+1\}^* \mid |x|_0 \text{ is even and } |x|_1 \text{ is odd}\}$

Ans:



Corollary:

The class of regular languages is closed under set difference

Proof: Since $L_1 - L_2 = L_1 \cap L_2^C$,

and regular languages are closed w.r.t intersection and complement; regular languages are also closed under set difference.

(II) Other properties:

Lemma:
For every regular language L, there exists a finite automaton M with a single final state such that $L(M) = L$

Proof: Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA accepting L,
construct $M' = (Q \cup \{p\}, \Sigma, \delta', q_0, \{p\})$, where $p \notin Q$ is a new state and δ' is given by

$$\delta'(q,a) = \begin{cases} \delta(q,a), & \text{if } q \in Q, a \in \Sigma \\ p & \text{if } q \in F, a = \epsilon \end{cases}$$

Note: M' is an NFA which is obtained by adding a new final state p to M that is connected from all the final states of M through ϵ-transitions.

Here, p is the only final state in M' and all the final states of M are made non-final states.
It's easy to prove that $L(M) = L(M')$.

Thm: if L is regular, then so is
$$L^R = \{x^R \mid x \in L\}$$

Proof:
Let M be a finite automata with the initial state q_0, and single final state q_f that accepts L.

Construct a finite automata M^R by reversing the arcs of M with the same labels and by interchanging the roles of initial and final states.

If $x \in \Sigma^*$ is accepted by M, there is a path from q_0 to q_f labelled x in M.
$\Rightarrow$ There will be a path from q_f to q_0 in M^R labelled x^R so that $x^R \in L(M^R)$.

Note: ① $E = F + G$ Then $E^R = F^R + G^R$

↳ the reversal of the union of two languages is obtained by computing the reversal of the two languages and taking union of these languages

② $E = F.G$ then $E^R = G^R . F^R$

↳ if $w \in L(E)$ is the concatenation of $w_1 \in L(F)$ and $w_2 \in L(G)$, then $w^R = w_2^R . w_1^R$

③ $E = F^*$ then $E^R = (F^R)^*$

Ex let $E = 01^* + 10^*$

$$E^R = (01^* + 10^*)^R$$
$$= (01^*)^R + (10^*)^R$$
$$= (1^*)^R 0^R + (0^*)^R (1)^R$$
$$= (1^R)^* 0 + (0^R)^* 1$$
$$= 1^* 0 + 0^* 1$$

	(intersection) $\cap$	(union) $\cup$	(complement) $\neg$	(concatenation) $\cdot$	(Kleene) $*$
DFA	Easy (Product automata)	Easy (Product automata)	Easy	Hard	Hard.
NFA	Doable	Easy	Hard	Easy	Easy.
Regular expression	Hard	Easy	Hard	Easy	Easy

→ Regular languages.

Some interesting observations:

① if $L_1 \cup L_2$ is regular then neither L_1 nor L_2 needs necessarily be regular

ex: let $L_1 = \{a^m b^n \mid m = n\}$ ← both non-regular

and $L_2 = \{a^m b^n \mid m \neq n\}$

$\Rightarrow L_1 \cup L_2 = \Sigma^*$ ← Regular

② if $L_1 \cap L_2$ is regular then neither L_1 nor L_2 needs necessarily be regular

ex: let $L_1 = \phi$ ← Regular

$L_2 = a^n b^n$ ← Non-regular

$L_1 \cap L_2 = \phi$ ← Regular

OR $L_1 = a^n b^n$ ← Non-regular

$L_2 = b^n a^n$ ← Non-regular

$L_1 \cap L_2 = \phi$ ← Regular

(3) if L is regular then its complement is regular and L is non regular then its complement would also be non-regular and vice-versa.

(4) if $L_1 \cdot L_2$ is regular then both L_1 nor L_2 needs necessarily be regular

ex: $L_1 = a^n b^n$ $L_2 = \phi$

$(a^n b^n) \cdot \phi = \phi$ → regular

$(a^n b^n)$ → Non-regular

ϕ → regular

Decision properties of the Regular languages:

(1) Membership question:

Does a string w belong to L?

OR

is $w \in L$?

Proof: (1) construct DFA(L)
(2) Run w on DFA(L)
(3) if DFA(L) accepts w, then $w \in L$, else $w \notin L$

Conclusion: Membership is decidable.

(2) Emptiness question:

is $L = \phi$?

Proof: (1) construct DFA(L)
(2) if there exists no path from start state to final state, $L = \phi$ else $L \neq \phi$

Note: may conduct DFS (Depth first search)

Conclusion: emptiness is decidable.

(3) Equivalence:

is $L_1 = L_2$?

Proof:
(1) Construct DFA(L_1) and DFA(L_2)
(2) Reduce them to their minimal DFAs: MinDFA(L_1) and MinDFA(L_2)
(3) if MinDFA(L_1) = MinDFA(L_2) then $L_1 = L_2$ else $L_1 \neq L_2$

Alternative Proof:

let $L_3 = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$

↳ symmetric difference of L_1 and L_2 (should be regular)

Since emptiness problem is decidable

$\Rightarrow L_3 = \phi$ iff $L_1 = L_2$

Conclusion: Equivalence of regular languages is decidable.

④ Subset:

① is $L_1 \subset L_2$?

Proof:

① if $L_1 - L_2 = \phi$ and $L_2 - L_1 \neq \phi$ then $L_1 \subset L_2$ else $L_1 \not\subset L_2$

② is $L_1 \subseteq L_2$?

Proof: ① if $L_1 \cap \overline{L_2} = \phi$ then $L_1 \subseteq L_2$ else $L_1 \not\subseteq L_2$.

Conclusion: subset property of RL is decidable.

⑤ Infinite:

is L infinite?

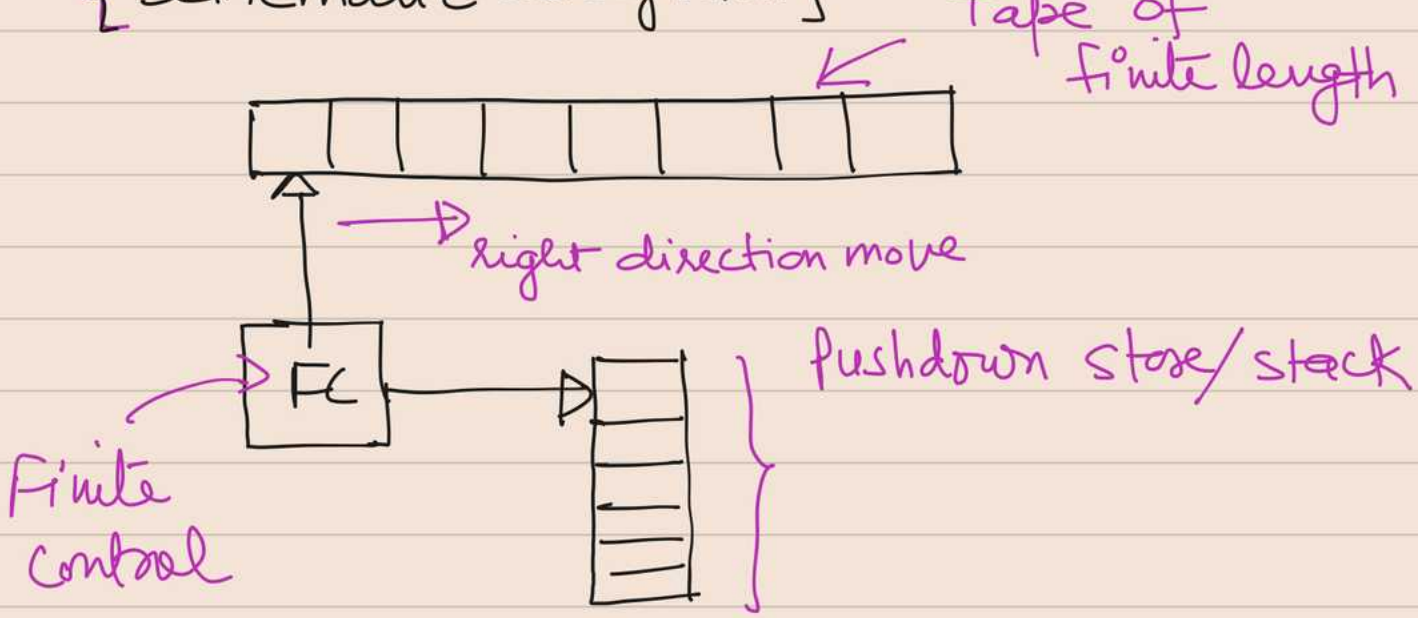
Proof: ① construct DFA(L)

② if DFA(L) has at least one loop, then L is infinite else L is finite.

Conclusion: Infinite property of RL is decidable.

Pushdown Automata :

{Schematic diagram}



{Mathematical description}

Set of PDS symbols

$(Q, \Sigma, \delta, q_0, F, \tau, z_0)$

- Q: N.E finite set of states
- Σ: alphabet
- δ: Transition function
- $q_0 \in Q$: initial state
- $F \subseteq Q$: Final states
- $z_0 \in \tau$: initial symbol in PDS

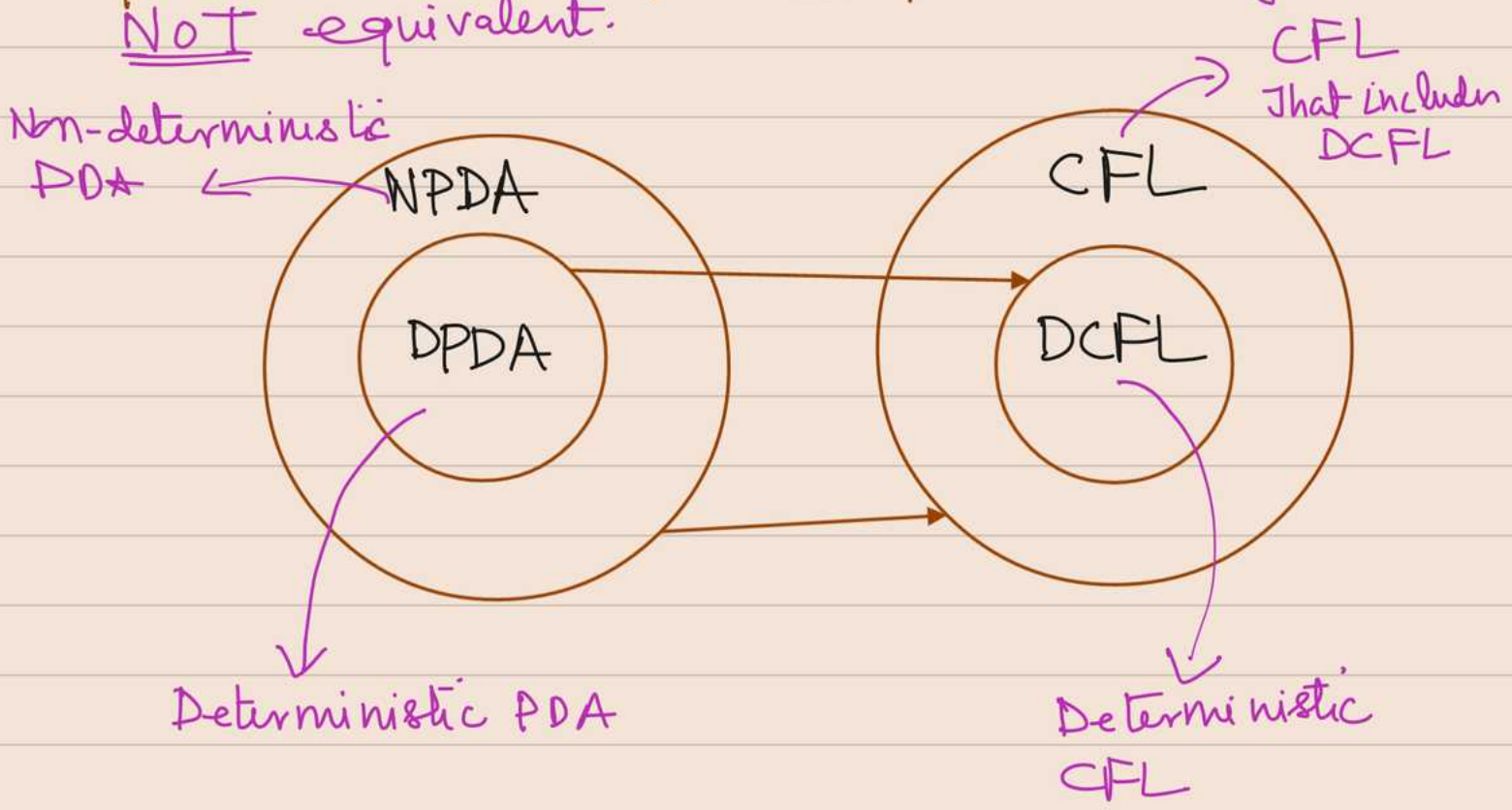
$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \to Q \times \Gamma^*$$

Note:

(1) single top of the stack can be replaced by a complete string (of the symbols taken from τ)

(2) can make a move, even without reading anything from the tape.

Note: Capability of NPDA (Non-deterministic pushdown automata) and DPDA (Deterministic pushdown automata) is different. They are NOT equivalent.



Important terms:

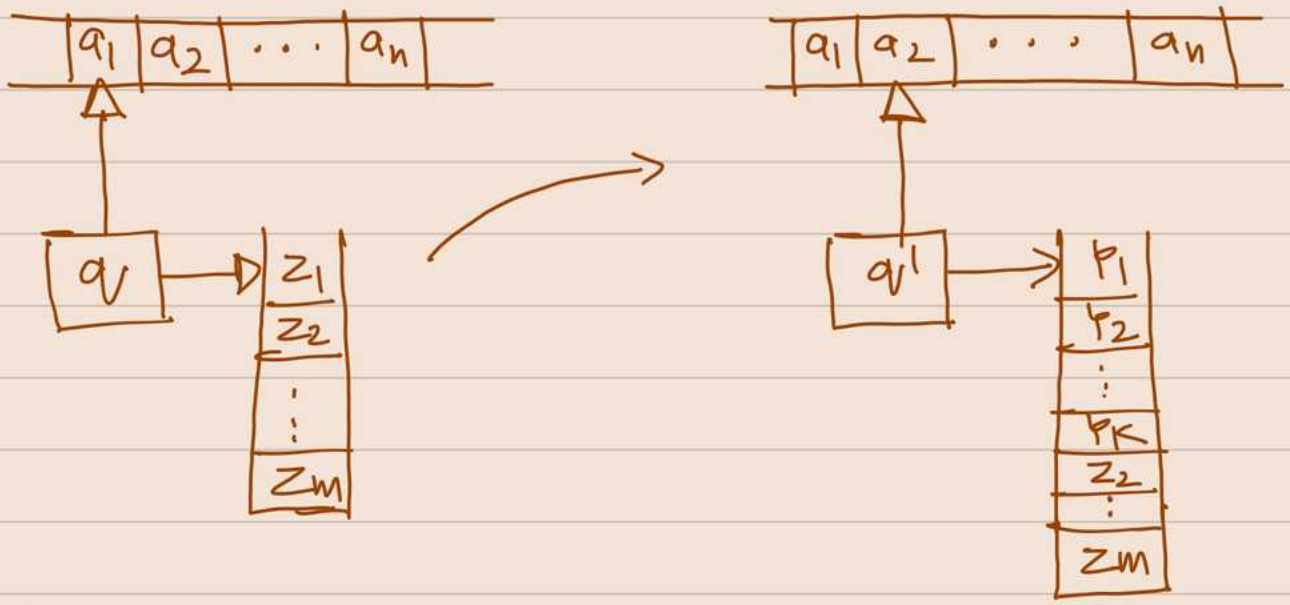
(1) Instantaneous descriptor (ID) –:

(q, x, α)

$q \in Q$ — Current state of PDA

$x \in \Sigma^*$ — remaining string to be scanned.

$\alpha \in \Gamma^*$ — current complete content of stack.



ID_1

$(q, a_1 a_2 \cdots a_n, Z_1 Z_2 \cdots Z_m)$

- q: current state
- $a_1 a_2 \cdots a_n$: Remaining string on tape
- $Z_1 Z_2 \cdots Z_m$: Content of stack

ID_2

$(q', a_2 a_3 \cdots a_n, \gamma_1 \gamma_2 \cdots Z_m)$

This change in ID's (from ID_1 to ID_2) must have occurred due to the transition function.

$$\delta(q, a_1, Z_1) = (q', \gamma_1 \gamma_2 \cdots \gamma_k)$$

Replace Z_1 by $\gamma_1 \gamma_2 \cdots \gamma_k$ but push $\gamma_1 \gamma_2 \cdots \gamma_k$ in the reverse order in stack to keep γ_1 on the top.

Note: We study the behaviour of PDA in terms of the changes in the ID's.

(2) Acceptance of a string : (w)

Case-1 (Acceptance through Final state)

$$(q_0, w, z_0) \vdash^* (q_f, \in, \alpha)$$

Defn: Let $M = (Q, \Sigma, \delta, q_0, F, \Gamma, z_0)$ be a PDA. The set $T(A)$ accepted by PDA through Final state is defined by

$$T(A) = \{ w \in \Sigma^* \mid (q_0, w, z_0) \vdash^* (q_f, \in, \alpha) \text{ for some } q_f \in F \text{ and } \alpha \in \Gamma^* \}$$

Case-2 (Acceptance through null state or empty stack)

$$(q_0, w, z_0) \vdash^* (q, \in, \in)$$

Defn: Let $M = (Q, \Sigma, \delta, q_0, F, \Gamma, z_0)$ be a PDA. The set $N(A)$ accepted by PDA through Null state (or empty stack) is defined by

$$N(A) = \{ w \in \Sigma^* \mid (q_0, w, z_0) \vdash^* (q, \in, \in) \text{ for some } q \in Q \}$$

↑ Non-final

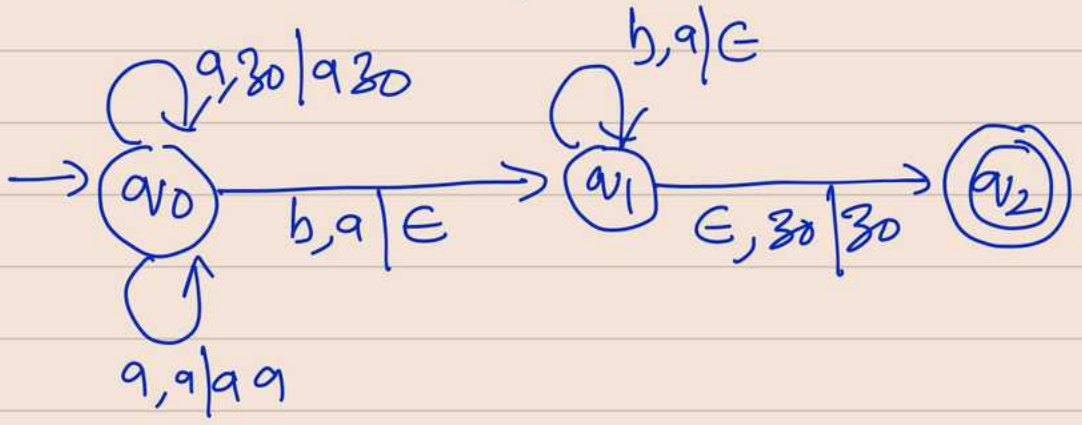
An Important theorem:
if $M = (Q, \Sigma, \delta, q_0, F, \Gamma, z_0)$ accepts L by final state, we can design another PDA M', which will accept L by the empty state and vice-versa

$$L = T(M) = N(M')$$

Design of Pushdown Automata :

" Develop the logic of the solution in terms of PUSH / POP / SKIP operations "

Ex: $L = \{ a^n b^n \mid n \geq 1 \}$

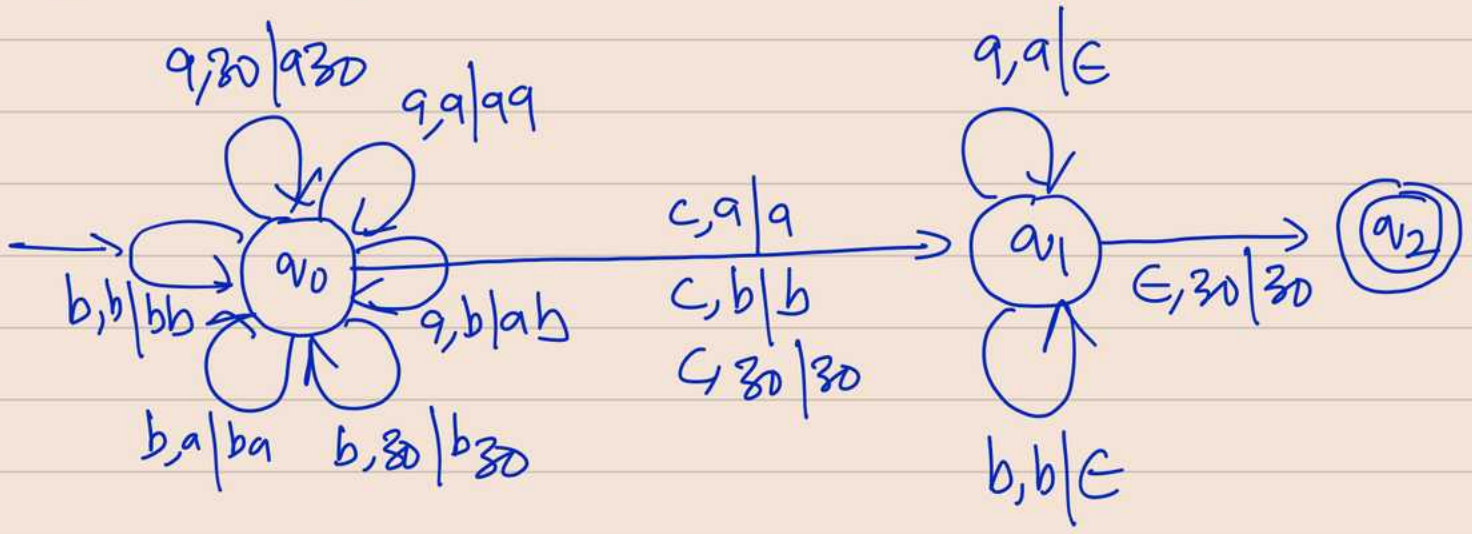


Note: (1) All possible transitions from a state need not to be defined, even for a deterministic pushdown Automata (DPDA).

(2) Its suggested to write transition fn also in the answer, apart from the diagram.

$\delta(q_0, a, Z_0) = (q_0, a Z_0)$
$\delta(q_0, a, a) = (q_0, aa)$
$\delta(q_0, b, a) = (q_1, \in)$
$\delta(q_1, b, a) = (q_1, \in)$
$\delta(q_1, \in, Z_0) = (q_2, Z_0)$

Ex $L = \{ wcw^R \mid w \in \{a,b\}^* \}$



Note: A PDA is deterministic if there is no configuration for which PDA has a choice of more than one move.

In other words, PDA is deterministic, if it satisfies both of the following conditions.

Condition 1:

$\forall q \in Q$, $a \in \Sigma \cup (\epsilon)$, and $X \in \Gamma$

$\delta(q, a, X)$ has at most one element.

Condition 2:

$\forall q \in Q$ and $X \in \Gamma$, if $\delta(q, \epsilon, X) \neq \phi$

then $\delta(q, a, X) = \phi \ \forall a \in \Sigma$

Failing either condition shall make the design "Non-deterministic".

{ Non-deterministic Pushdown automata }

mathematically,

$(Q, \Sigma, \delta, q_0, F, \Gamma, Z_0)$

$\delta : Q \times (\Sigma \cup (\epsilon)) \times \Gamma \to P(Q \times \Gamma^*)$

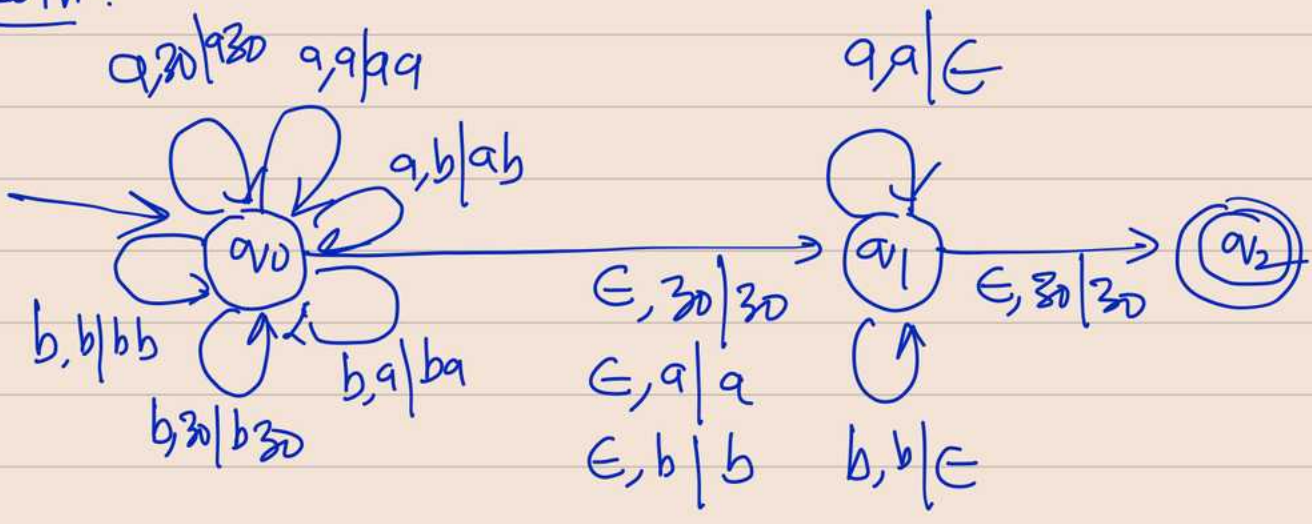
Powerset

OR $\delta : Q \times (\Sigma \cup (\epsilon)) \times \Gamma \to 2^{Q \times \Gamma^*}$

Ex: PDA (NPDA) for even Palindrome. over $\{a, b\}$.

OR $L = \{ xx^R \mid x \in \{a,b\}^* \}$

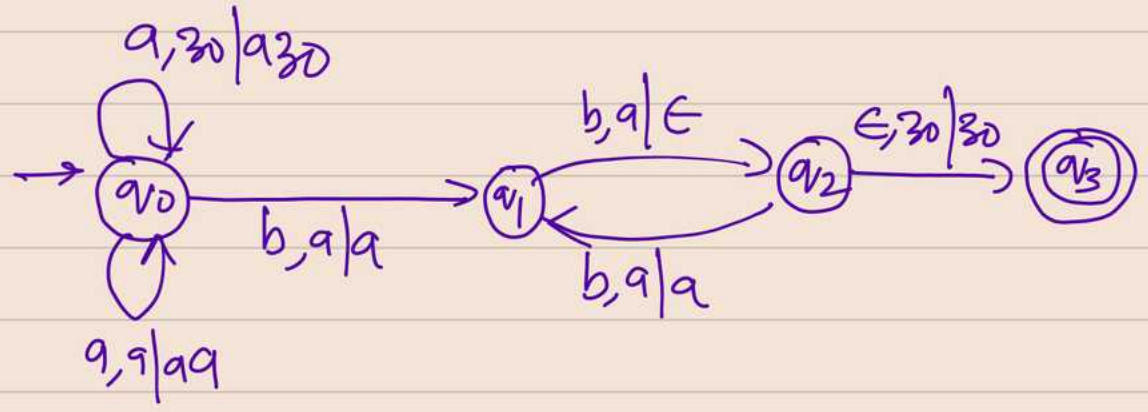
Soln:



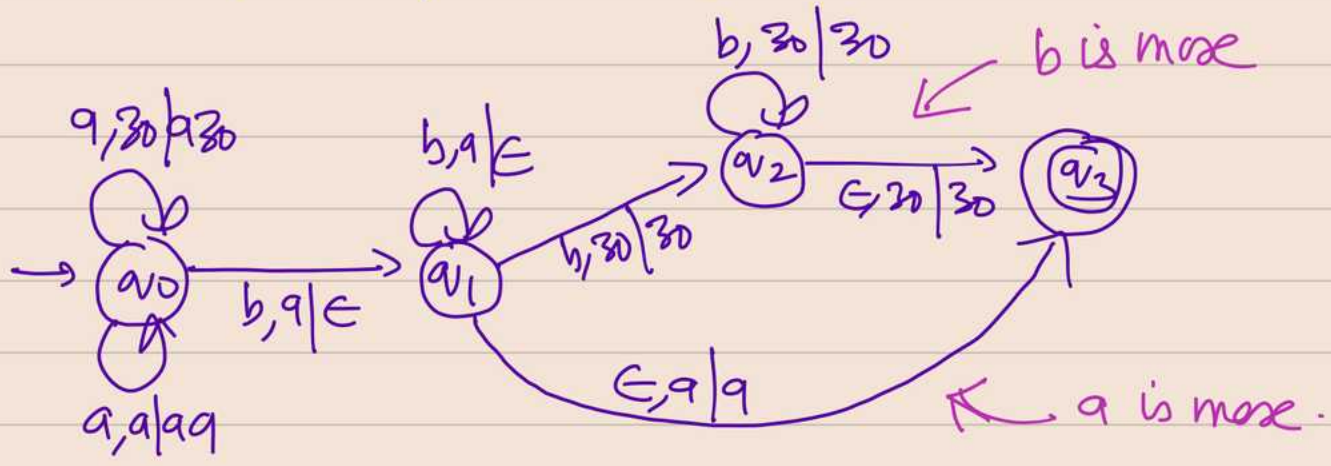
Exercise: Design PDA to accept palindrome (even and odd, both) over $\Sigma = \{a, b\}$.

Solved Examples :

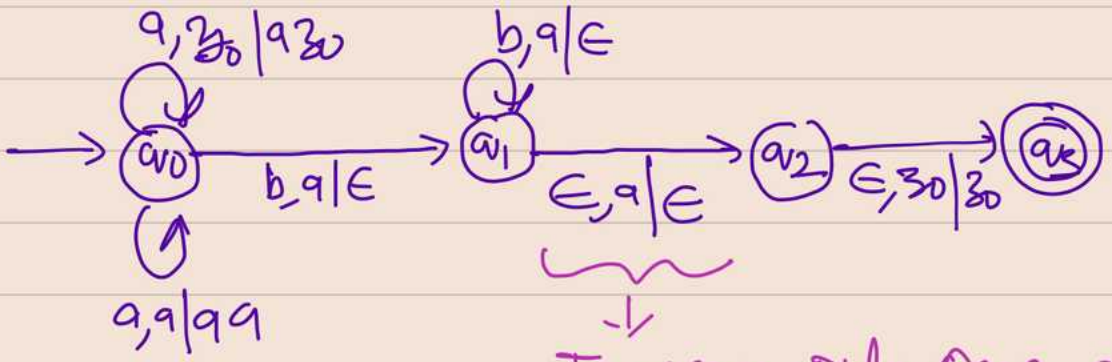
(1) $L = \{ a^n b^{2n} \mid n \geq 1 \}$



(2) $L = \{ a^i b^j \mid i, j \geq 1, i \neq j \}$



(3) $L = \{ a^i b^j \mid i, j \geq 1, i = j + 1 \}$

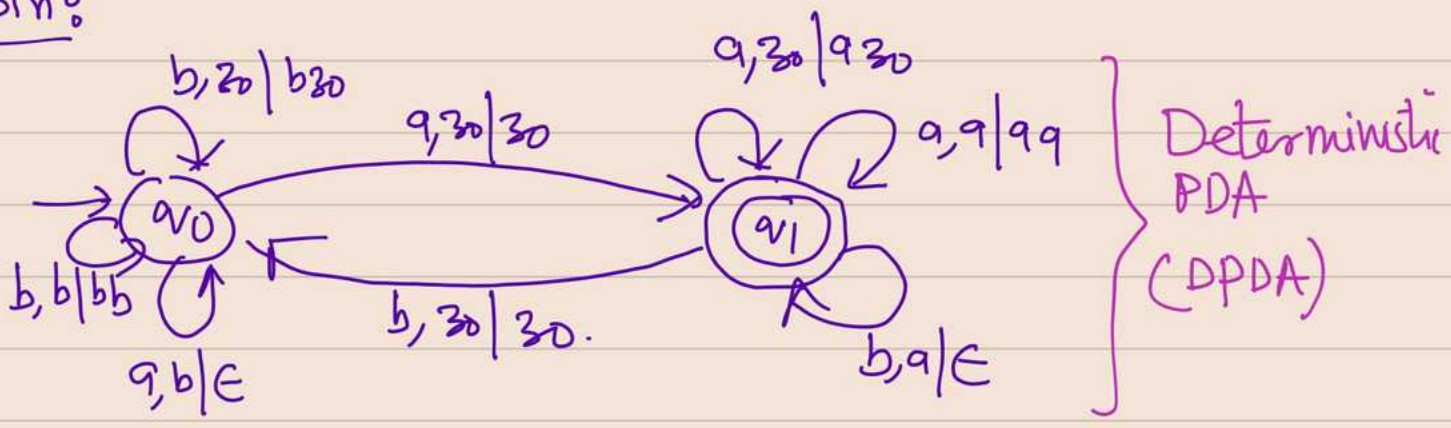


④ $L = \{ x \in \{a,b\}^* \mid n_a(x) > n_b(x) \}$

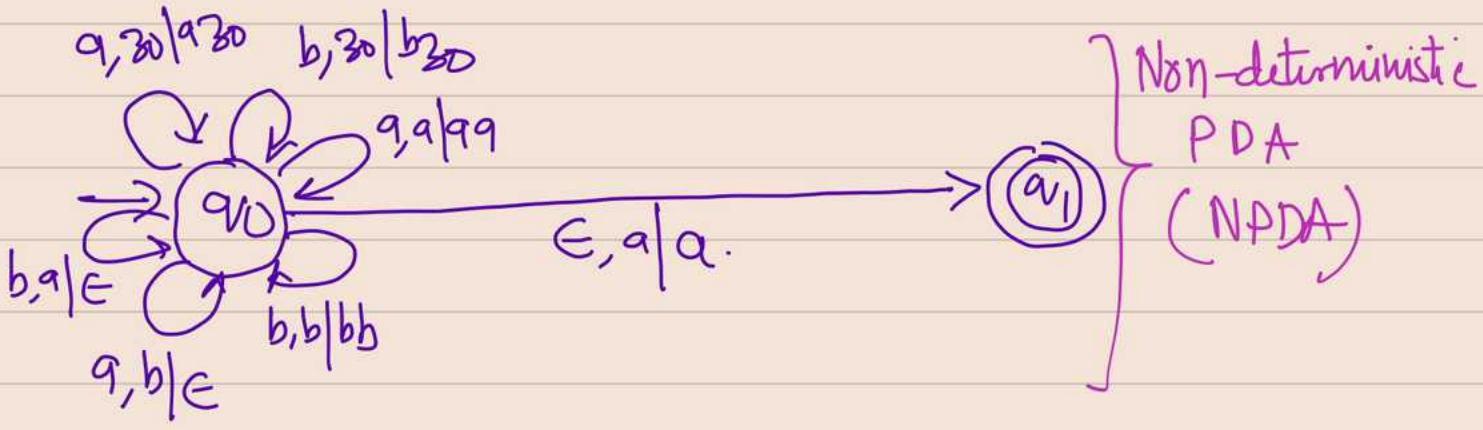
$n_a(x)$ → No. of a's in x

$n_b(x)$ → No. of b's in x

Soln:



OR



[Ref: "Introduction to languages and theory of computation" by John. C. Martin (TMH)]

Equivalence of NFA and DFAs:

① Every DFA is an NFA, so NFAs are at least as powerful as DFAs

② For every NFA N, there exists a DFA M such that

$$L(M) = L(N)$$

Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA

Goal: Construct DFA $M = (Q', \Sigma, \delta', q_0', F')$ recognizing $L(N)$

Intuition: Run all threads of N in parallel, maintaining the set of states where all threads are

Formally: $Q' = P(Q)$

Subset construction method.

Subset Construction (converting NFA → DFA)

input NFA $N = (Q, \Sigma, \delta, q_0, F)$

Output DFA $M = (Q', \Sigma, \delta', q_0', F')$

$Q' = P(Q)$

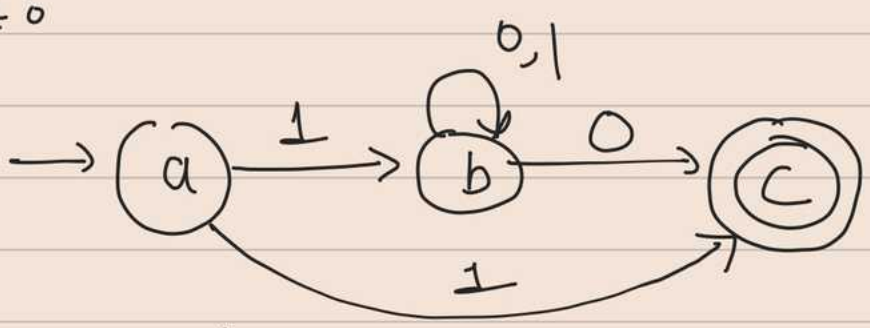
$\delta' = Q' \times \Sigma \to Q'$

$\delta'(S, a) = \bigcup_{r \in S} \delta(r, a)$

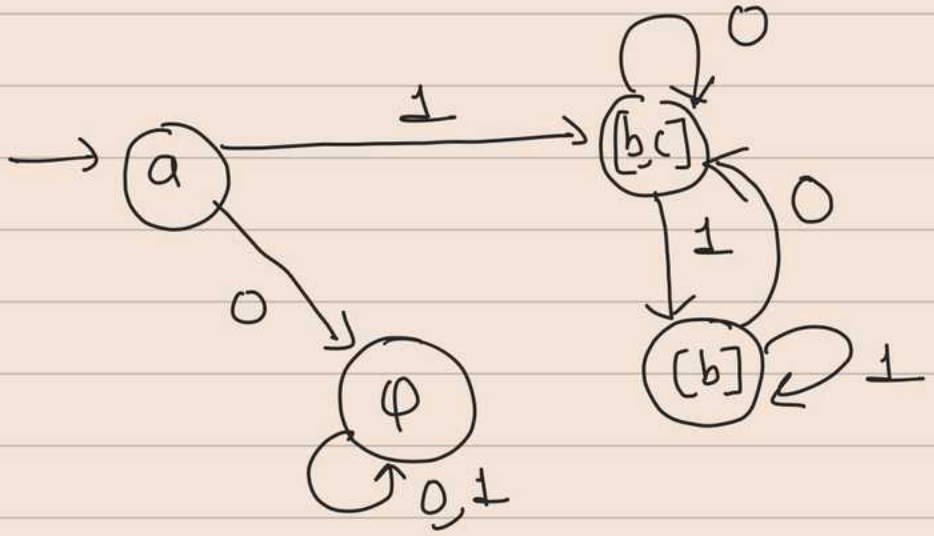
State of DFA $q_0' = \{q_0\}$

$F' = \{R \in Q' \mid R \cap F \neq \phi\}$

Example:



	0	1
→ [a]	φ	[b,c]
[b,c]	[b,c]	[b]
[b]	[b,c]	[b]
φ	φ	φ



Note: N states in NFA may result into 2^N states in DFA. However, in real problems DFA can have fewer states too.

Example 20

NFA

	State	a	b
→	1	2,4	5
	2	4,6	1,3,5
	3	2,6	5
	4	2,8	1,5,7
	5	2,4,6,8	1,3,7,9
	6	2,8	3,5,9
	7	4,8	5
	8	4,6	5,7,9
*	9	6,8	5

NFA has 9 states

Final state

DFA

	a	b
{1}	{2,4}	{5}
{2,4}	{2,4,6,8}	{1,3,5,7}
{5}	{2,4,6,8}	{1,3,7,9}
{2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
{1,3,5,7}	{2,4,6,8}	{1,3,5,7,9}
{1,3,7,9}	{2,4,6,8}	{5}
{1,3,5,7,9}	{2,4,6,8}	{1,3,5,7,9}

DFA has 7 states

Subset Construction (converting ϵ-NFA to DFA)

input NFA $N = (Q, \Sigma, \delta, q_0, F)$

Output DFA $M = (Q', \Sigma, \delta', q_0', F')$

$Q' = P(Q)$

$\delta' = Q' \times \Sigma \to Q'$

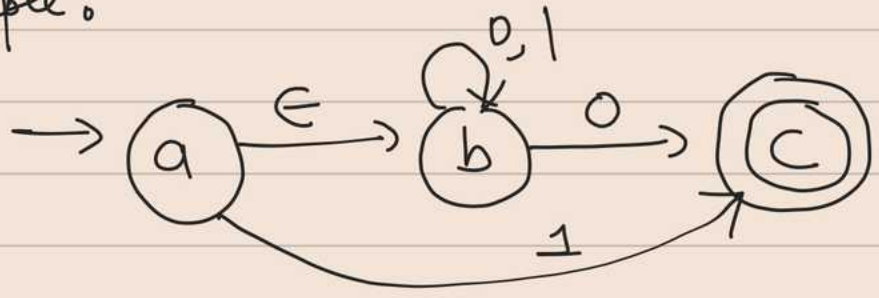
$\delta'(S, a) = \bigcup_{r \in S} \epsilon^*(\delta(r, a))$

State of DFA

$q_0' = \epsilon^*\{q_0\}$

$F' = \{R \in Q' \mid R \cap F \neq \phi\}$

Example:



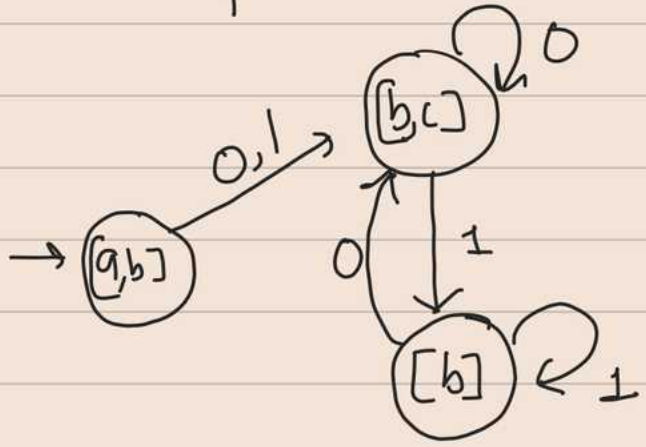
	0	1
→ [a,b]	[b,c]	[b,c]
[b,c]	[b,c]	[b]
[b]	[b,c]	[b]

$\epsilon^*(a) = \{a, b\}$

$\epsilon^*(b) = \{b\}$

$\epsilon^*(c) = \{c\}$

$\epsilon^*(b, c) = \{b, c\}$



Historical Note

Subset Construction introduced in Rabin & Scott's 1959 paper "Finite Automata and their Decision Problems"



1976 ACM Turing Award citation

For their joint paper "Finite Automata and Their Decision Problem," which introduced the idea of nondeterministic machines, which has proved to be an enormously valuable concept. Their (Scott & Rabin) classic paper has been a continuous source of inspiration for subsequent work in this field.



Kleene's theorem :.

Following three statements are equivalent

(i) A language is regular (i.e, represented by a regular expression)

(ii) A language is accepted by a NFA

(iii) A language is accepted by a FA

Note: Point No (ii) may also be stated using TG (Transition graph) instead of NFA

Parse tree:

A parse tree (or a derivation tree) is a tree that represents how a string is derived from the start symbol using the productions of the grammar. A parse tree satisfies:

(1) the Root is labeled by the start symbol S

(2) Each internal node is labeled by a Nonterminal

(3) if a node labeled A has children

$$X_1, X_2, \ldots, X_k$$

Then the grammar contains the rule

$$A \to X_1 X_2 \cdots X_k$$

(4) Each leaf is labeled by a terminal or $\in$

(For final parse tree)

(5) the string generated by the tree (the yield) is obtained by reading the leaf labels from left to right

Leftmost derivation:

A derivation $S \overset{*}{\Rightarrow} w$ is called a leftmost derivation and write as $S \overset{*}{\underset{lm}{\Rightarrow}} w$, if w is obtained from S by a sequence of steps at each of which apply a rule to the leftmost nonterminal available.

Ex $S \Rightarrow SbS \Rightarrow abS \Rightarrow abScS \Rightarrow abacS \Rightarrow abaca$

Note: Each parse tree uniquely corresponds exactly one leftmost derivation.

Rightmost derivation:

A derivation $S \overset{*}{\Rightarrow} w$ is called a leftmost derivation and write as $S \overset{*}{\underset{lm}{\Rightarrow}} w$, if w is obtained from S by a sequence of steps at each of which apply a rule to the rightmost nonterminal available.

Ambiguity in CFG :

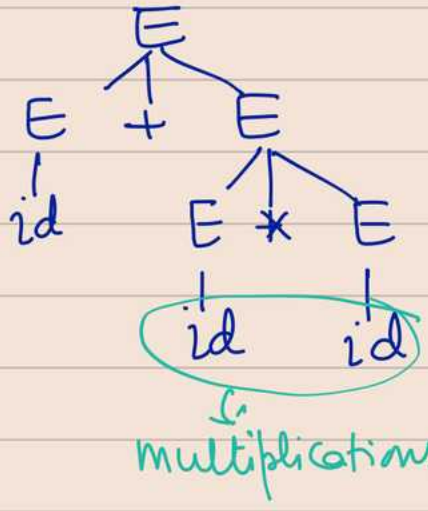
A CFG is ambiguous if one or more terminal strings have multiple leftmost derivations from the start symbol. Equivalently, multiple rightmost derivations or multiple parse trees.

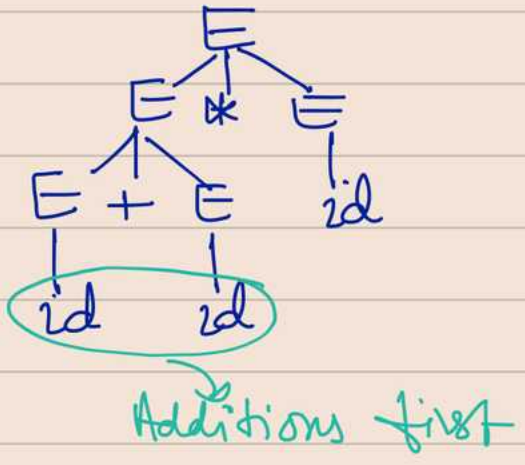
Example :

(1) $G = \quad E \rightarrow E+E \mid E*E \mid id$

Consider derivation of id+id*id

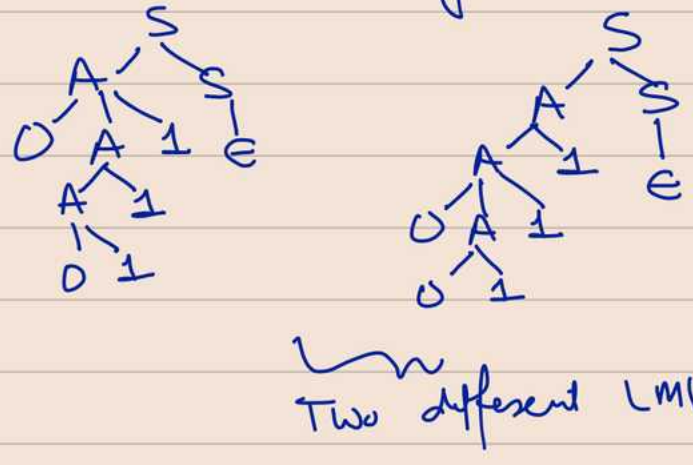
Two different leftmost derivations (LMDs) are :





Example (2) : $S \rightarrow AS \mid \epsilon \quad A \rightarrow A1 \mid 0A1 \mid 01$

Consider string 00111



Notes

(1) A CFL L is inherently ambiguous if every CFG for L is ambiguous

ex

$S \rightarrow AS \mid \in$

$A \rightarrow 0A1 \mid B$

$B \rightarrow B1 \mid 01$

(2) Ambiguity is a property of the gramma

(3) Determine whether a CFG is ambiguous is Undecidable.

(4) Given CFG G, Find G^1 such that $L(G) = L(G^1)$ and G^1 is unambiguous, There is no algorithm that can solve this problem.

Context free grammars (CFG):

Write CFG for the following languages

① $L = \{ a^n b^n \mid n \geq 1 \}$

$S \rightarrow ab \mid aSb$

min = ab

general = aaabbb

② $L = \{ a^n b^m \mid n < m,\ n, m \geq 1 \}$

$S \rightarrow AB$

$A \rightarrow aAb \mid ab$

$B \rightarrow bB \mid b$

min = abb

general = $a^m b^{m+K}$, $m \geq 1$ $K > 1$

③ $L = \{ a^n b^{2n} \mid n \geq 1 \}$

$S \rightarrow abb \mid aSbb$

min = abb

general = aaabbbbbb

④ $L = \{ a^n b^m c^{n+m},\ n, m \geq 1 \}$

$S \rightarrow aSc \mid aBc$

$B \rightarrow bBc \mid bc$

$a^n b^m c^m c^n$

⑤ $L = \{ ww^R \mid w \in \{a,b\}^* \}$

$S \rightarrow aSa \mid bSb \mid \in$

min = $\in$

general = abbbba

⑥ $L = \{ x \mid x \in \{0,1\}^*, n_0(x) = n_1(x) \}$

min = $\in$

gen = 0101

$S \to 0S1 \mid 1S0 \mid SS \mid \in$

⑦ $L = \{ a^i b^j c^k \mid j > i+k \}$

$a^i \; b^i \; b^{k+m} \; c^k$

$S \to ABC$

$A \to aAb \mid \in$

$C \to bCc \mid \in$

$B \to bB \mid b$.

⑧ $L = (01)^* \mid (11+0)^*$ ← Regular expression

$S \to A \mid B$

$A \to 01A \mid \in$

$B \to 11B \mid 0B \mid \in$.

L = set of all palindromes over $\{0,1\}^*$.

$S \rightarrow 0S0 \mid 1S1 \mid \epsilon \mid 0 \mid 1$

Chomsky Normal Form (CNF):

A context-free grammar G is in Chomsky normal form if every production is of the form

$$A \to a \quad \text{or} \quad A \to BC$$

Note: $S \to \Lambda$ is also allowed if $\Lambda \in L(G)$. However, when $\Lambda \in L(G)$, we assume that S does not appear on the R.H.S of any production

Reduction of CFG to CNF:

For every CFG, there is an equivalent grammar in CNF.

Step 1: Eliminate null and unit productions

Step 2: Eliminate terminals in R.H.S (which are Not in CNF)

Like if $A \to X_1 X_2 \ldots X_n$ with some terminal on R.H.S. if X_i is a terminal say a_i, add a new nonterminal C_{a_i} to V_N' and add $C_{a_i} \to a_i$ to P'.

Step 3: Restrict the number of nonterminals in R.H.S by two.

Ex: Reduce the following grammar G to CNF.

$G \equiv S \to aAD,\ A \to aB \mid bAB,\ B \to b,\ D \to d.$

Solution:

$S \to C_a AD \quad A \to C_a B \mid C_b AB,$
$B \to b,\ D \to d,\ C_a \to a,\ C_b \to b$

Now restrict the number of NT's by 2

$S \to C_a C_1$ and $C_1 \to AD$
$A \to C_b C_2$ and $C_2 \to AB$

Final productions:

$S \to C_a C_1,\ C_1 \to AD,$
$A \to C_a B \mid C_b C_2$
$C_2 \to AB$
$B \to b$
$D \to d$
$C_a \to a$
$C_b \to b$

Brief overview of the left recursion and its removal:

A production of a CFG is left recursive if it has the form

$$A \to A\alpha$$

where $A \in V_N$, $\alpha \in (V_N \cup \Sigma)^*$.

Removal:

$$A \to A\alpha \mid \beta \equiv A \to \beta A' \;\Big\}$$
$$A' \to \alpha A' \mid \in \;\Big\}$$

Left recursion is removed. A' is a New Nonterminal

in general:

$$A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

$$\equiv$$

$$A \to \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A'$$
$$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \in$$

Remove null production

without $\in$, this can also be re-written as

$$A \to \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A' \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$
$$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_m$$

Greibach Normal Form (GNF):

A CFG $G=(V_N, \Sigma, P, S)$ is said to be in GNF if every production is of the form $A \to a\alpha$, where $a \in \Sigma$ and $\alpha \in V_N^*$.

Note: $L(G_{GNF}) = L(G) - \{\in\}$ or add $S \to \in$

Procedure:

Step 1: Grammar Simplification.
(Convert given CFG → CNF (recommended))
Eliminate null, unit, useless, and bring CFG in CNF

Step 2: Rename Variables (Nonterminals) like $A_1, A_2, \cdots, A_n$ starting with $S=A$ $\quad \gamma \in V_N^*$

Step 3: Enforce the ordering condition
We want productions only of the form
$A_i \to A_j \gamma$ with $j > i$
if $A_i \to A_j \gamma$ and $j < i$ then
We substitute the productions of A_j.
Repeat until $A_i \to A_p \gamma$, $p > i$

Step 4: Remove left recursion
if we get $A_K \to A_K \gamma$, eliminate by removing left recursion (Here new nonterminals are added as Z_i's)

Step 5: Make every rule start with a terminal
Whenever $A_i \to A_j \gamma$, substitute the rules of A_j until the first symbol becomes a terminal

Identify productions which are not in any one of the following form

$A_i \to A_j \alpha_k \quad j > i$
$Z_i \to A_j \alpha_k \quad j \leq n$
$A_i \to a \alpha_k$
$a \in \Sigma$
$\alpha_k \in V_N^*$

↓
Standard form

In summary the conversion process is:

1. convert CFG → CNF
2. Rename variables $A_1, \dots, A_n$ (with $S = A_1$)
3. for each A_i:
   - eliminate $A_i \to A_j \gamma$, $j < i$ by substitution
   - Remove left recursion
4. substitute until every rule is $A \to a\alpha$

Example:

$G \equiv$ $S \to XA \mid BB$
$B \to b \mid SB$
$X \to b$
$A \to a$.

1. G is already in CNF
2. Relabel the variables
   $S = A_1$, $X = A_2$, $A = A_3$, $B = A_4$

$A_1 \to A_2A_3 \mid A_4A_4$
$A_4 \to b \mid A_1A_4$
$A_2 \to b$
$A_3 \to a$.

3. Identify productions which are not in the standard form

$A_4 \to b \mid A_1A_4$

→ Not in standard form

Identify productions which are not in any one of the following form

$A_i \to A_j x_k \quad j > i$
$Z_i \to A_j x_k \quad j \leq n$
$A_i \to a x_k$
$a \in \Sigma$
$x_k \in V_N^*$

↳ standard form

(4) $A_4 \to b \mid A_1 A_4$

Substitute A_1

$A_4 \to b \mid A_2 A_3 A_4 \mid A_4 A_4 A_4$

Substitute A_2

$A_4 \to b \mid b A_3 A_4 \mid A_4 A_4 A_4$

Need to remove left recursion

$A_4 \to b A_4 A_4 Z_1 \mid b Z_1$

$Z_1 \to A_4 A_4 Z_1 \mid \in$

Remove $\in$

$A_4 \to b A_4 A_4 Z_1 \mid b Z_1 \mid b A_4 A_4 \mid b$

$Z_1 \to A_4 A_4 Z_1 \mid A_4 A_4$

At this stage our grammar looks like

$A_1 \to A_2 A_3 \mid A_4 A_4$

$A_4 \to b A_3 A_4 \mid b \mid b A_3 A_4 Z_1 \mid b Z_1$

$Z_1 \to A_4 A_4 \mid A_4 A_4 Z_1$

$A_2 \to b$

$A_3 \to a$

} All are in standard form, but not in GNF

(5) A_2, A_3 A_4 productions are in GNF

for $A_1 \to A_2 A_3 \mid A_4 A_4$

Substitute for A_2 and A_4 to convert it to GNF, similarly for $Z \to A_4 A_4 \mid A_4 A_4 Z$

Substitute for A_4 to convert it into GNF

Final grammar ↴

$A_1 \to bA_3 \mid bA_3A_4A_4 \mid bA_4 \mid bA_3A_4ZA_4 \mid bZA_4$

$A_4 \to bA_3A_4 \mid b \mid bA_3A_4Z \mid bZ$

$Z \to bA_3A_4A_4 \mid bA_4 \mid bA_3A_4ZA_4 \mid bZA_4 \mid bA_3A_4A_4Z \mid bA_4Z \mid bA_3A_4ZA_4Z \mid bZA_4Z$

$A_2 \to b$

$A_3 \to a$

(19) productions.

Simplification of CFG:

Ex: $G \equiv$ $S \to AB$
$A \to a$
$B \to b$
$B \to C$
$E \to c \mid \wedge$

Following are the issues with this grammar

1. C does not derive any terminal string
2. E and c do not appear in any sentential form
3. $E \to \wedge$ is a null production
4. $B \to C$ simply replaces B by C

→ Non terminals not deriving terminal strings

→ Symbols not appearing in any sentential form

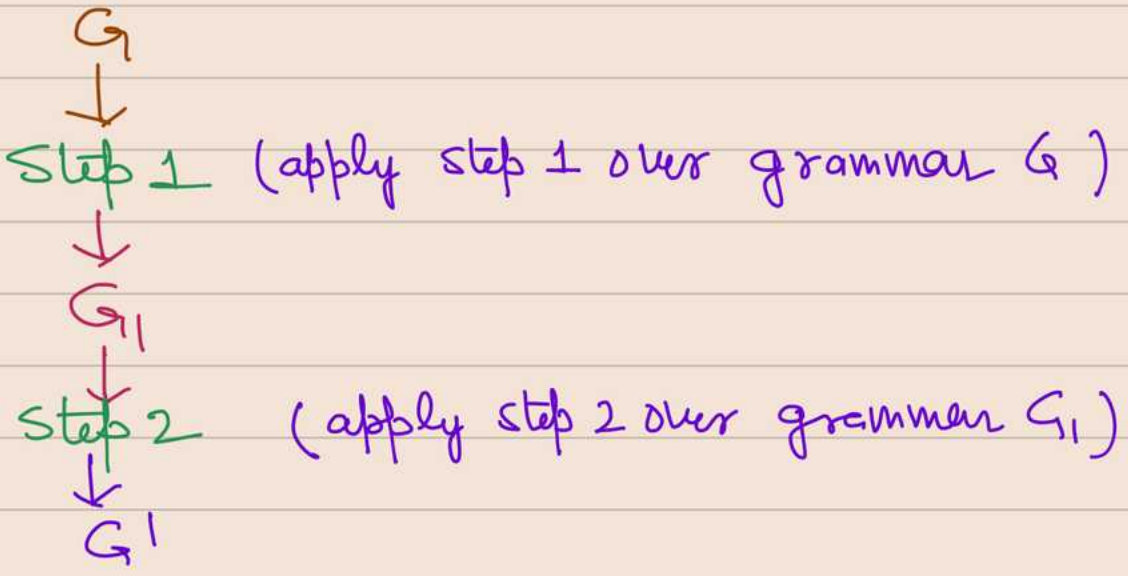
→ Null productions.

→ Unit productions.

Construction of Reduced Grammars:

Step 1: We construct a grammar G_1 equivalent to the given grammar G so that every Nonterminal in G_1 derives some terminal string (either direct or indirect)

Step 2: We construct a grammar G' equivalent to G_1 so that every symbol in G' appears in some sentential form of G' which is equivalent to G_1 and hence G' is the required reduced grammar



Note: Order of step 1 and step 2 matters. if change the order, then final grammar not be the reduced grammar.

Step 1: For every CFG $G = (V_N, \Sigma, P, S)$, we can construct an equivalent grammar $G_1 = (V_N', \Sigma, P', S)$, such that each Non-terminal in G_1 derives some terminal string:

Procedure:

① $W_1 = \{A \in V_N \mid \exists\ A \to \alpha,\ \alpha \in \Sigma^*\}$

② $W_{i+1} = W_i \cup \{A \in V_N \mid \exists\ A \to \alpha, \alpha \in (W_i \cup \Sigma)^*\}$

Stops when $W_k = W_{k+1}$

⇒ ③ $V_N' = W_k$.

④ $P' = \{A \to \alpha \mid A, \alpha \in (V_N' \cup \Sigma)^*\}$

Example:

$G \equiv \quad S \to AB,\ A \to a,\ B \to b,\ B \to C,\ E \to c$

Find G' such that every Nonterminal in G' derives some terminal string.

Soln: $W_1 = \{A, B, E\}$

$W_2 = \{A, B, E, S\}$

$W_3 = W_2$

⇒ $V_N' = \{A, B, E, S\}$

$P' = \{S \to AB,\ A \to a,\ B \to b,\ E \to c\}$

Step 2: For every CFG $G_1 = (V_N, \Sigma, P, S)$, we can construct an equivalent grammar $G' = (V_N', \Sigma', P', S)$, such that every symbol in $V_N' \cup \Sigma'$ appears in some sentential form:

for every X in $V_N' \cup \Sigma'$, there exists α such that $S \overset{*}{\Rightarrow} \alpha$, and X is a symbol in the string α

Procedure:

① $W_1 = \{S\}$

② $W_{i+1} = W_i \cup \{X \in V_N \cup \Sigma \mid \exists\, A \to \alpha$ with $A \in W_i$ and α containing the symbol $X\}$

③ $V_N' = V_N \cap W_K$, $\Sigma' = \Sigma \cap W_K$

④ $P' = \{A \to \alpha \mid A \in W_K\}$

Ex: $G_1 \equiv S \to AB,\ A \to a,\ B \to b,\ E \to c,$

Soln: $W_1 = \{S\}$

$W_2 = \{S, A, B\}$

$W_3 = \{S, A, B, a, b\}$

$V_N' = S, A, B \qquad \Sigma' = a, b$

$P' = S \to AB,\ A \to a,\ B \to b.$

Example: Find a reduced grammar equivalent to the grammar G, whose productions are

$S \to AB \mid CA,\ B \to BC \mid AB,\ A \to a,\ C \to aB \mid b$

Soln:

Step 1:

$W_1 = \{A, C\}$

$W_2 = \{A, C, S\}$

$W_3 = W_2$

$\Rightarrow V_N' = A, C, S.$

$P' = S \to CA,\ A \to a,\ C \to b$

Step 2: (Note: use Step 2, over the O/P of step 1)

$W_1 = \{S\}$

$W_2 = \{S, C, A\}$

$W_3 = \{S, C, A, a, b\}$

$W_3 = W_4$

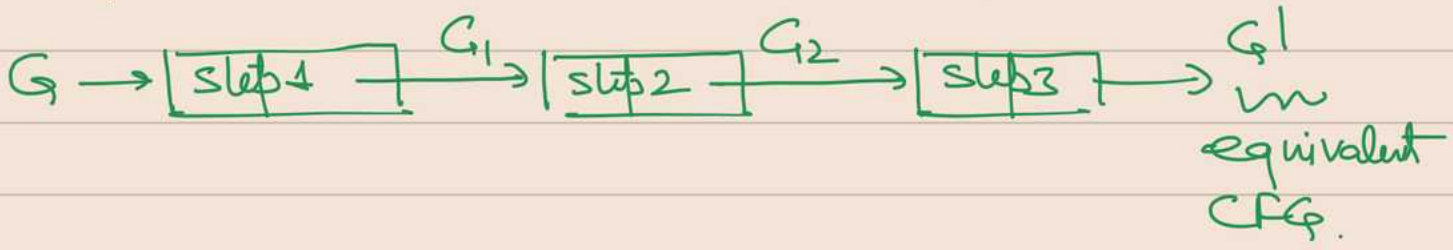
$\Rightarrow V_N' = S, C, A$

$P' = S \to CA,\ A \to a,\ C \to b$

if G is a context-free grammar, we can construct an equivalent grammar G' which is reduced and has no null productions or unit productions.

Note: We construct equivalent grammar G_1 in the following way, and these steps must be followed in this order only:

Step 1 : Eliminate null productions
Step 2 : Eliminate unit productions
Step 3 : Construct a reduced grammar



Elimination of Null productions :

Nullable Nonterminal: A nonterminal N is nullable if there is a production $N \to \Lambda$

Or

there is a derivation that starts at N and leads to Λ

$N \Rightarrow \dots \Rightarrow \Lambda$

Procedure :

1. Delete all Λ-productions
2. Add the following productions, for every production $X \to$ old string, add new productions of the form

   $X \to$ New string, where new string is formed by deleting all possible subsets of nullable nonterminals, except $X \to \Lambda$ (even if all the characters of old string are nullable)

Ex : $S \to a \mid Xb \mid aYa$

$X \to Y \mid \Lambda$

$Y \to b \mid X$

Procedure : $N = \{X, Y\}$ // Set of Nullable NT's

$S \to a \mid Xb \mid aYa \mid b \mid aa$

$X \to Y$

$Y \to b \mid X$

Elimination of Unit productions:

A production of the form

Nonterminal → One Nonterminal

is called a Unit production.

Procedure:

For every pair of Nonterminals A, and B if the CFG has a unit production $A \to B$ or if there is a chain of unit productions leading from A to B, such that

$$A \Rightarrow X_1 \Rightarrow X_2 \Rightarrow \cdots \Rightarrow B$$

where X_1, X_2, are some nonterminals, we then introduce new productions according to the following rule:

if the nonunit productions from B are

$$B \to s_1 | s_2 | s_3 | \ldots$$

where s_1, s_2, and s_3 are strings, create the productions

$$A \to s_1 | s_2 | s_3 | \ldots$$

We do the same for all such pairs of A's and B's simultaneously. We can then eliminate all Unit productions.